

book_to_slide_BY_sections_V13_

July 19, 2025

1 Set up Paths

```
[37]: # Cell 1: Setup and Configuration
import os
import re
import logging
import warnings
from docx import Document
import pdfplumber
import ollama
from tenacity import retry, stop_after_attempt, wait_exponential, RetryError
import json
import logging

# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
↳ %(message)s')
logger = logging.getLogger(__name__)

# Extract Unit ID from the filename
# --- Helper Functions ---
def print_header(text: str, char: str = "="):
    """Prints a centered header to the console."""
    print("\n" + char * 80)
    print(text.center(80))
    print(char * 80)

def extract_uo_id_from_filename(filename: str) -> str:
    match = re.match(r'^[A-Z]+\d+', os.path.basename(filename))
    if match:
        return match.group(0)
    raise ValueError(f"Could not extract a valid Unit ID from filename:
↳ '{filename}'")

# To define outputs folder
TEST_MODE = True
```

```

# --- 1. CORE SETTINGS ---
# Set this to True for EPUB, False for PDF. This controls the entire notebook's
↳flow.
PROCESS_EPUB = True # for EPUB
# PROCESS_EPUB = False # for PDF

# Control process for testing
EXTRACT_UO = False
CREATE_RAG_BOOK = False

#=====INPUTS=====

# --- 2. INPUT FILE NAMES ---
# The name of the Unit Outline file (e.g., DOCX, PDF)
UNIT_OUTLINE_FILENAME = "ICT312 Digital Forensic_Final.docx" # epub
# UNIT_OUTLINE_FILENAME = "ICT311 Applied Cryptography.docx" # pdf

# The names of the book files
EPUB_BOOK_FILENAME = "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide
↳to Computer Forensics and Investigations_ Processing Digital
↳Evidence-Cengage Learning (2018).epub"
PDF_BOOK_FILENAME = "(Chapman & Hall_CRC Cryptography and Network Security
↳Series) Jonathan Katz, Yehuda Lindell - Introduction to Modern
↳Cryptography-CRC Press (2020).pdf"

# --- 3. DIRECTORY STRUCTURE ---
# Define the base path to your project to avoid hardcoding long paths everywhere
PROJECT_BASE_DIR = "/home/sebas_dev_linux/projects/course_generator"

CONFIG_DIR = os.path.join(PROJECT_BASE_DIR, "configs")
SETTINGS_DECK_PATH = os.path.join(CONFIG_DIR, "settings_deck.json")
#TEACHING_FLOWS_PATH = os.path.join(CONFIG_DIR, "teaching_flows.json") # 
↳future add

# to check the layouts
LAYOUT_MAPPING_PATH = os.path.join(CONFIG_DIR, "layout_mapping_test_Mod.json")
↳# using the modified one

# New output path for the processed settings
PROCESSED_SETTINGS_PATH = os.path.join(CONFIG_DIR, "processed_settings.json")

# Define subdirectories relative to the base path
DATA_DIR_INPUT = os.path.join(PROJECT_BASE_DIR, "data")

```

```

INPUT_UO_DIR = os.path.join(DATA_DIR_INPUT, "UO")
INPUT_BOOKS_DIR = os.path.join(DATA_DIR_INPUT, "books")

#=====OUTPUTS=====

try:
    UNIT_ID = extract_uo_id_from_filename(UNIT_OUTLINE_FILENAME)
except ValueError as e:
    print(f"Error: {e}")
    UNIT_ID = "UNKNOWN_ID"

# Define outputs

if TEST_MODE:
    DATA_DIR_OUTPUT = os.path.join(PROJECT_BASE_DIR, "data_output/test_Output")
else:
    DATA_DIR_OUTPUT = os.path.join(PROJECT_BASE_DIR, "data_output")

os.makedirs(DATA_DIR_OUTPUT, exist_ok=True)
PARSE_DATA_DIR = os.path.join(PROJECT_BASE_DIR, DATA_DIR_OUTPUT, "1.␣
↳Parse_data")
os.makedirs(PARSE_DATA_DIR, exist_ok=True)

# Construct full paths for clarity

OUTPUT_PARSED_UO_DIR = os.path.join(PARSE_DATA_DIR, f"Parse_UO/{UNIT_ID}")
os.makedirs(OUTPUT_PARSED_UO_DIR, exist_ok=True)
OUTPUT_PARSED_TOC_DIR = os.path.join(PARSE_DATA_DIR, f"Parse_TOC_books/
↳{UNIT_ID}")
os.makedirs(OUTPUT_PARSED_TOC_DIR, exist_ok=True)
OUTPUT_DB_DIR = os.path.join(DATA_DIR_OUTPUT, f"0. DataBase_Chroma/{UNIT_ID}")
os.makedirs(OUTPUT_DB_DIR, exist_ok=True)

# to Save the individual FINAL plan to a file
PLAN_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"2. generated_plans/{UNIT_ID}")
os.makedirs(PLAN_OUTPUT_DIR, exist_ok=True)

#to Save the individual FINAL Content to a file
CONTENT_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"3. generated_content/
↳{UNIT_ID}")
os.makedirs(CONTENT_OUTPUT_DIR, exist_ok=True)

```

```

CONTENT_LLM_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"4.↳
↳generated_content_llm/{UNIT_ID}")
os.makedirs(CONTENT_LLM_OUTPUT_DIR, exist_ok=True)

SLIDE_TEMPLATE_PATH = "/home/sebas_dev_linux/projects/course_generator/data/
↳slide_style/slide_style_test_2.pptx"
FINAL_PRESENTATION_DIR = os.path.join(DATA_DIR_OUTPUT, f"5. final_presentations/
↳{UNIT_ID}")
os.makedirs(FINAL_PRESENTATION_DIR, exist_ok=True)

# --- 4. LLM & EMBEDDING CONFIGURATION ---
LLM_PROVIDER = "ollama" # Can be "ollama", "openai", "gemini"
OLLAMA_HOST = "http://localhost:11434"
OLLAMA_MODEL = "qwen3:8b" # "qwen3:8b", #"mistral:latest"
EMBEDDING_MODEL_OLLAMA = "nomic-embed-text"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 100

# --- 5. DYNAMICALLY GENERATED PATHS & IDs (DO NOT EDIT THIS SECTION) ---
# This section uses the settings above to create all the necessary variables↳
↳for later cells.

# Full path to the unit outline file
FULL_PATH_UNIT_OUTLINE = os.path.join(INPUT_UO_DIR, UNIT_OUTLINE_FILENAME)

# Determine which book and output paths to use based on the PROCESS_EPUB flag
if PROCESS_EPUB:
    BOOK_PATH = os.path.join(INPUT_BOOKS_DIR, EPUB_BOOK_FILENAME)
    PRE_EXTRACTED_TOC_JSON_PATH = os.path.join(OUTPUT_PARSED_TOC_DIR,↳
↳f"{UNIT_ID}_epub_table_of_contents.json")
else:
    BOOK_PATH = os.path.join(INPUT_BOOKS_DIR, PDF_BOOK_FILENAME)
    PRE_EXTRACTED_TOC_JSON_PATH = os.path.join(OUTPUT_PARSED_TOC_DIR,↳
↳f"{UNIT_ID}_pdf_table_of_contents.json")

# Define paths for the vector database
file_type_suffix = 'epub' if PROCESS_EPUB else 'pdf'
CHROMA_PERSIST_DIR = os.path.join(OUTPUT_DB_DIR,↳
↳f"{UNIT_ID}_chroma_db_toc_chunks_{file_type_suffix}")
CHROMA_COLLECTION_NAME = f"book_toc_chunks_{file_type_suffix}"

```

```

# Define path for the parsed unit outline
PARSED_UO_JSON_PATH = os.path.join(OUTPUT_PARSED_UO_DIR, f"{os.path.
↳splitext(UNIT_OUTLINE_FILENAME)[0]}_parsed.json")

# --- Sanity Check Printout ---
print("--- CONFIGURATION SUMMARY ---")
print(f"Processing Mode: {' EPUB' if PROCESS_EPUB else ' PDF'}")
print(f" Unit ID: {UNIT_ID}")
print(f" Unit Outline Path: {FULL_PATH_UNIT_OUTLINE}")
print(f" Book Path: {BOOK_PATH}")
print(f" Parsed UO Output Path: {PARSED_UO_JSON_PATH}")
print(f" Parsed ToC Output Path: {PRE_EXTRACTED_TOC_JSON_PATH}")
print(f" Vector DB Path: {CHROMA_PERSIST_DIR}")
print(f" Vector DB Collection: {CHROMA_COLLECTION_NAME}")
print(f" Plan Develop output: {PLAN_OUTPUT_DIR}")
print(f" Plan Fetched output: {CONTENT_OUTPUT_DIR}")
print(f" Plan Generated LLM output: {CONTENT_LLM_OUTPUT_DIR}")
print(f" Final slides: {FINAL_PRESENTATION_DIR}")
print("--- SETUP COMPLETE ---")

```

--- CONFIGURATION SUMMARY ---

Processing Mode: EPUB

Unit ID: ICT312

Unit Outline Path:

/home/sebas_dev_linux/projects/course_generator/data/UO/ICT312 Digital Forensic_Final.docx

Book Path: /home/sebas_dev_linux/projects/course_generator/data/books/Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations Processing Digital Evidence-Cengage Learning (2018).epub

Parsed UO Output Path:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/1. Parse_data/Parse_UO/ICT312/ICT312 Digital Forensic_Final_parsed.json

Parsed ToC Output Path:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/1. Parse_data/Parse_TOC_books/ICT312/ICT312_epub_table_of_contents.json

Vector DB Path:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/0. DataBase_Chroma/ICT312/ICT312_chroma_db_toc_chunks_epub

Vector DB Collection: book_toc_chunks_epub

Plan Develop output:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2. generated_plans/ICT312

Plan Fetched output:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3. generated_content/ICT312

Plan Generated LLM output:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4. generated_content_llm/ICT312

```
Final slides:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/5.
final_presentations/ICT312
--- SETUP COMPLETE ---
```

2 System Prompt

```
[12]: UNIT_OUTLINE_SYSTEM_PROMPT_TEMPLATE = """
You are an expert academic assistant tasked with parsing a university unit
↪outline document and extracting key information into a structured JSON
↪format.

The input will be the raw text content of a unit outline. Your goal is to
↪identify and extract the following details and structure them precisely as
↪specified in the JSON schema below. Note: do not change any key name

**JSON Output Schema:**

```json
{{
 "unitInformation": {{
 "unitCode": "string | null",
 "unitName": "string | null",
 "creditPoints": "integer | null",
 "unitRationale": "string | null",
 "prerequisites": "string | null"
 }},
 "learningOutcomes": [
 "string"
],
 "assessments": [
 {{
 "taskName": "string",
 "description": "string",
 "dueWeek": "string | null",
 "weightingPercent": "integer | null",
 "learningOutcomesAssessed": "string | null"
 }}
],
 "weeklySchedule": [
 {{
 "week": "string",
 "contentTopic": "string",
 "requiredReading": "string | null"
 }}
],

```

```

 "requiredReadings": [
 "string"
],
 "recommendedReadings": [
 "string"
]
}

```

Instructions for Extraction:

Unit Information: Locate Unit Code, Unit Name, Credit Points. Capture 'Unit Overview / Rationale' as unitRationale. Identify prerequisites.

Learning Outcomes: Extract each learning outcome statement.

Assessments: Each task as an object. Capture full task name, description, Due Week, Weighting % (number), and Learning Outcomes Assessed.

weeklySchedule: Each week as an object. Capture Week, contentTopic, and requiredReading.

Required and Recommended Readings: List full text for each.

**\*\*Important Considerations for the LLM\*\*:**

Pay close attention to headings and table structures.

If information is missing, use null for string/integer fields, or an empty list [] for array fields.

Do no change keys in the template given

Ensure the output is ONLY the JSON object, starting with {{{{ and ending with }}}}. No explanations or conversational text before or after the JSON.

Now, parse the following unit outline text:

```
--- UNIT_OUTLINE_TEXT_START ---
```

```
{outline_text}
```

```
--- UNIT_OUTLINE_TEXT_END ---
```

```
"""
```

[13]: *# Place this in a new cell after your imports, or within Cell 3 before the functions.*

*# This code is based on the schema from your screenshot on page 4.*

```

from pydantic import BaseModel, Field, ValidationError
from typing import List, Optional
import time

```

*# Define Pydantic models that match your JSON schema*

```

class UnitInformation(BaseModel):
 unitCode: Optional[str] = None
 unitName: Optional[str] = None
 creditPoints: Optional[int] = None
 unitRationale: Optional[str] = None
 prerequisites: Optional[str] = None

```

```

class Assessment(BaseModel):

```

```

 taskName: str
 description: str
 dueWeek: Optional[str] = None
 weightingPercent: Optional[int] = None
 learningOutcomesAssessed: Optional[str] = None

class WeeklyScheduleItem(BaseModel):
 week: str
 contentTopic: str
 requiredReading: Optional[str] = None

class ParsedUnitOutline(BaseModel):
 unitInformation: UnitInformation
 learningOutcomes: List[str]
 assessments: List[Assessment]
 weeklySchedule: List[WeeklyScheduleItem]
 requiredReadings: List[str]
 recommendedReadings: List[str]

```

### 3 Helper functions

```

[14]: # Cell 10: Configuration and Scoping for Content Generation (Corrected)

import os
import json
import logging

Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

--- Global Test Overrides (for easy testing) ---
TEST_OVERRIDE_WEEKS = None
TEST_OVERRIDE_FLOW_ID = None
TEST_OVERRIDE_SESSIONS_PER_WEEK = None
TEST_OVERRIDE_DISTRIBUTION_STRATEGY = None

def process_and_load_configurations():
 """
 PHASE 1: Loads configurations, calculates a PRELIMINARY time-based slide
 budget,
 and saves the result as 'processed_settings.json' for the Planning Agent.
 """

```

```

"""
print_header("Phase 1: Configuration and Scoping Process", char="-")

--- Load all input files ---
logger.info("Loading all necessary configuration and data files...")
try:
 os.makedirs(CONFIG_DIR, exist_ok=True)
 with open(PARSED_UO_JSON_PATH, 'r', encoding='utf-8') as f:
 unit_outline = json.load(f)
 with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
 book_toc = json.load(f)
 with open(SETTINGS_DECK_PATH, 'r', encoding='utf-8') as f:
 settings_deck = json.load(f)
 # with open(TEACHING_FLOWS_PATH, 'r', encoding='utf-8') as f:
 teaching_flows = json.load(f)
 logger.info("All files loaded successfully.")
except FileNotFoundError as e:
 logger.error(f"FATAL: A required configuration file was not found: {e}")
 return None

--- Pre-process and Refine Settings ---
logger.info("Pre-processing settings_deck for definitive plan...")
processed_settings = json.loads(json.dumps(settings_deck))

unit_info = unit_outline.get("unitInformation", {})
processed_settings['course_id'] = unit_info.get("unitCode",
 "UNKNOWN_COURSE")
processed_settings['unit_name'] = unit_info.get("unitName", "Unknown Unit_
 Name")

--- Apply test overrides IF they are not None ---
logger.info("Applying overrides if specified...")
This block now correctly sets the teaching_flow_id based on the
 interactive flag.
if TEST_OVERRIDE_FLOW_ID is not None:
 processed_settings['teaching_flow_id'] = TEST_OVERRIDE_FLOW_ID
 logger.info(f"OVERRIDE: teaching_flow_id set to_
 '{TEST_OVERRIDE_FLOW_ID}'")
else:
 # If no override, use the 'interactive' boolean from the file as the
 source of truth.
 is_interactive = processed_settings.get('interactive', False)
 if is_interactive:
 processed_settings['teaching_flow_id'] = 'apply_topic_interactive'
 else:
 processed_settings['teaching_flow_id'] = 'standard_lecture'

```

```

 logger.info(f"Loaded from settings: 'interactive' is {is_interactive}.\n
↳Set teaching_flow_id to '{processed_settings['teaching_flow_id']}'".)

 # The 'interactive' flag is now always consistent with the teaching_flow_id.
 processed_settings['interactive'] = "interactive" in
↳processed_settings['teaching_flow_id'].lower()

 if TEST_OVERRIDE_SESSIONS_PER_WEEK is not None:
 processed_settings['week_session_setup']['sessions_per_week'] =
↳TEST_OVERRIDE_SESSIONS_PER_WEEK
 logger.info(f"OVERRIDE: sessions_per_week set to
↳{TEST_OVERRIDE_SESSIONS_PER_WEEK}")

 if TEST_OVERRIDE_DISTRIBUTION_STRATEGY is not None:
 processed_settings['week_session_setup']['distribution_strategy'] =
↳TEST_OVERRIDE_DISTRIBUTION_STRATEGY
 logger.info(f"OVERRIDE: distribution_strategy set to
↳'{TEST_OVERRIDE_DISTRIBUTION_STRATEGY}'")

 if TEST_OVERRIDE_WEEKS is not None:
 processed_settings['generation_scope']['weeks'] = TEST_OVERRIDE_WEEKS
 logger.info(f"OVERRIDE: generation_scope weeks set to
↳{TEST_OVERRIDE_WEEKS}")

 # --- DYNAMIC SLIDE BUDGET CALCULATION (Phase 1) ---
 logger.info("Calculating preliminary slide budget based on session time...")

 params = processed_settings.get('parameters_slides', {})
 SLIDES_PER_HOUR = params.get('slides_per_hour', 18)

 duration_hours = processed_settings['week_session_setup'].
↳get('session_time_duration_in_hour', 1.0)
 sessions_per_week = processed_settings['week_session_setup'].
↳get('sessions_per_week', 1)

 logger.info(f" Duration Hours {duration_hours}.")
 logger.info(f" Sessions per week {sessions_per_week}.")

 slides_content_per_session = int(duration_hours * SLIDES_PER_HOUR)
 target_total_slides = slides_content_per_session * sessions_per_week

 processed_settings['slide_count_strategy']['target_total_slides'] =
↳target_total_slides

```

```

 processed_settings['slide_count_strategy']['slides_content_per_session'] = \
↪ slides_content_per_session
 logger.info(f"Preliminary weekly content slide target calculated:\
↪ {target_total_slides} slides.")

 # --- Resolve Generation Scope if not overridden ---
 if TEST_OVERRIDE_WEEKS is None and processed_settings.
↪ get('generation_scope', {}).get('weeks') == "all":
 num_weeks = len(unit_outline.get('weeklySchedule', []))
 processed_settings['generation_scope']['weeks'] = list(range(1, \
↪ num_weeks + 1))

 # --- Save the processed settings to disk ---
 logger.info(f"Saving preliminary processed configuration to:\
↪ {PROCESSED_SETTINGS_PATH}")
 with open(PROCESSED_SETTINGS_PATH, 'w', encoding='utf-8') as f:
 json.dump(processed_settings, f, indent=2)
 logger.info("File saved successfully.")

 # --- Assemble master config for optional preview ---
 master_config = {
 "processed_settings": processed_settings,
 "unit_outline": unit_outline,
 "book_toc": book_toc,
 # "teaching_flows": teaching_flows
 }

 print_header("Phase 1 Configuration Complete", char="-")
 logger.info("Master configuration object is ready for the Planning Agent.")
 return master_config

```

### 3.0.1 Component: Definitive PowerPoint Layout Inspector

**Component:** Definitive PowerPoint Layout Inspector **Primary Purpose** The `inspect_and_generate_layout_config` function serves as a critical pre-processing utility for the automated presentation generation system. Its primary purpose is to bridge the gap between a visual PowerPoint template and the programmatic logic of the content generation agents. It achieves this by performing a deep inspection of a given PowerPoint (.pptx) template file and auto-generating a detailed, structured, and human-readable JSON configuration file (`layout_mapping.json`). This configuration file acts as the “API documentation” for the presentation template, allowing both human users and a Large Language Model (LLM) to understand and utilize the available slide layouts effectively. **Key Functions and GoOf course.** Here is a formal description of the purpose and functionality of the “Definitive PowerPoint Layout Inspector” script. This description is suitable for project documentation, a README file, or for explaining its role to other developers.

**Primary Purpose** The `inspect_and_generate_layout_config` function serves as a critical pre-processing utility for the automated presentation generation system. Its primary purpose is to **bridge the gap between a visual PowerPoint template and the programmatic logic of the content generation agents.**

It achieves this by performing a deep inspection of a given PowerPoint (`.pptx`) template file and auto-generating a detailed, structured, and human-readable JSON configuration file (`layout_mapping.json`). This configuration file acts as the “API documentation” for the presentation template, allowing both human users and a Large Language Model (LLM) to understand and utilize the available slide layouts effectively.

## Key Functions and Goals

### 1. Comprehensive Layout Discovery:

- The script guarantees that **every single slide layout** present in the PowerPoint template’s Slide Master is detected and analyzed. This prevents a common problem where unused or unconventionally named layouts might be missed by simpler scripts.

### 2. Detailed Placeholder Analysis:

- For each layout, the script extracts an exhaustive list of all its placeholders. For every placeholder, it records crucial metadata:
  - **type**: The functional role of the placeholder (e.g., `TITLE`, `BODY`, `OBJECT`, `TABLE`, `PICTURE`).
  - **name**: The unique name given to the placeholder in the PowerPoint interface (e.g., “Title 1”, “Content Placeholder 2”).
  - **idx**: The internal identification number of the placeholder.
  - **position and size**: The physical coordinates (`left`, `top`) and dimensions (`width`, `height`), converted to an intuitive unit (inches) for easy comprehension of the layout’s visual structure.

### 3. Intelligent Capability Summarization:

- The script’s core innovation is its ability to generate a **machine-readable capabilities summary** for each layout. Instead of just listing raw data, it synthesizes the placeholder information into a concise description of what the layout is designed for. For example:
  - `{"title_support": "standard_title", "body_layout": "2_column"}`
  - `{"title_support": "centered_title_with_subtitle", "body_layout": "no_body"}`
  - `{"specific_content_types": ["TABLE", "CHART"]}`
- This summary is specifically designed to be passed to an LLM as part of a prompt, enabling the LLM to make an informed, logical choice about the best layout for presenting a given piece of information.

### 4. User-Friendly Configuration:

- While providing a detailed summary for the LLM, the script also generates a simplified `user_selections` section. This allows a human operator to easily map the system’s semantic slide types (e.g., “Agenda”, “Summary”) to a specific layout index, providing a robust fallback and manual override capability.

## How It Solves Critical Problems

- **Eliminates Ambiguity:** By capturing the name, index, and position of every placeholder, it solves the problem of layouts with multiple placeholders of the same type (e.g., two content

boxes). The system can now programmatically target the “left column” vs. the “right column”.

- **Decouples Logic from Design:** The presentation generation agent no longer needs hard-coded assumptions about the template’s design. All the logic for choosing and populating layouts is driven by the generated JSON file. This means the visual template can be updated or completely replaced without requiring changes to the core Python code.
- **Empowers the LLM:** It transforms a visual, unstructured design asset (the .pptx file) into a structured, well-defined set of “tools” (the layouts) that an LLM can reason about. This is the key to enabling more advanced tasks where the LLM doesn’t just fill in content, but also makes decisions about the *visual structure* of the presentation.

In summary, the Definitive PowerPoint Layout Inspector is the foundational component that makes the entire presentation generation process intelligent, configurable, and robust. It translates the abstract design of a template into concrete, actionable data.

```
[15]: # this process the layouts from the slides provide to use them to process the
 ↪ plan (This require be change to the course of the system)
 # https://aistudio.google.com/prompts/18YaU5pG96eFMbM1l6yeG1v9j63PkLc5H

import logging
import os
import json
import logging
import random
from pptx import Presentation
from pptx.util import Inches
from pptx.enum.shapes import MSO_SHAPE_TYPE
from pptx.enum.shapes import PP_PLACEHOLDER
Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
 ↪ %(message)s')
logger = logging.getLogger(__name__)
def print_header(text: str, char: str = "="):
 """Prints a centered header to the console."""
 print("\n" + char * 80)
 print(text.center(80))
 print(char * 80)

def generate_layout_capabilities(layout_name: str, placeholders: list) -> dict:

 essential_placeholders = [p for p in placeholders if p['type'] !=
 ↪ 'SLIDE_NUMBER']
 capabilities = {"title_support": "none", "body_layout": "none",
 ↪ "specific_content_types": []}
 has_center_title = any(p['type'] == 'CENTER_TITLE' for p in
 ↪ essential_placeholders)
 has_standard_title = any(p['type'] == 'TITLE' for p in
 ↪ essential_placeholders)
```

```

has_subtitle = any(p['type'] == 'SUBTITLE' for p in essential_placeholders)
if has_center_title: capabilities["title_support"] = "centered_title"
elif has_standard_title: capabilities["title_support"] = "standard_title"
if has_subtitle: capabilities["title_support"] += "_with_subtitle"
body_placeholders = [p for p in essential_placeholders if p['type'] not in
↳('TITLE', 'CENTER_TITLE', 'SUBTITLE')]
if len(body_placeholders) == 0: capabilities["body_layout"] = "no_body"
elif len(body_placeholders) == 1: capabilities["body_layout"] =
↳"single_column"
elif len(body_placeholders) > 1:
 body_placeholders.sort(key=lambda p: p['left'])
 if len(body_placeholders) > 1 and body_placeholders[1]['left'] >
↳(body_placeholders[0]['left'] + body_placeholders[0]['width'] * 0.5):
 capabilities["body_layout"] = f"{len(body_placeholders)}_column"
 else: capabilities["body_layout"] = "stacked_sections"
specific_types = {p['type'] for p in body_placeholders if p['type'] not in
↳('BODY', 'OBJECT')}
if specific_types: capabilities["specific_content_types"] =
↳sorted(list(specific_types))
return capabilities

def inspect_and_generate_layout_config(template_path: str, output_path: str):
 """
 Inspects a template, generates a machine-readable capabilities summary, and
 ↳creates
 a complete and definitive JSON configuration file for user and LLM use.
 """
 try:
 prs = Presentation(template_path)
 except Exception as e:
 logger.error(f"Could not open or parse the presentation template at
↳'{template_path}'. Error: {e}")
 return

 # --- 1. Analyze all Layouts ---

 available_layouts = []
 for i, layout in enumerate(prs.slide_layouts):
 placeholder_details = []
 for p in layout.placeholders:
 placeholder_details.append({
 "idx": p.placeholder_format.idx, "type": p.placeholder_format.
↳type.name,
 "name": p.name, "left": round(p.left.inches, 2), "top": round(p.
↳top.inches, 2),

```

```

 "width": round(p.width.inches, 2), "height": round(p.height.
↪inches, 2)
 })
 capabilities = generate_layout_capabilities(layout.name,
↪placeholder_details)
 layout_info = {"layout_index": i, "layout_name": layout.name,
↪"capabilities": capabilities, "placeholders": placeholder_details}
 available_layouts.append(layout_info)

 # --- 2. Create the Complete User-Facing Configuration Structure ---
 def find_default_by_capability(layouts, capability_key, capability_value,
↪fallback_index=1):
 for layout in layouts:
 if layout['capabilities'].get(capability_key) == capability_value:
 return layout['layout_index']
 # If no perfect match is found, return a safe fallback index
 return fallback_index if len(layouts) > fallback_index else 0

 # ** This map is now complete, explicitly including all required keys **
 user_selection_map = {
 "Title": { # title and subtitle
 "description": "Main title slide of the deck.",
 "selected_layout_index":
↪find_default_by_capability(available_layouts, "title_support",
↪"centered_title_with_subtitle")
 },
 "Agenda": { # title and object
 "description": "Agenda/Table of Contents slide.",
 "selected_layout_index":
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
 },
 "Content": { # title and object
 "description": "Default slide for a standard topic with bullet
↪points.",
 "selected_layout_index":
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
 },
 "Content_Two_Column": { # title and 2 objects
 "description": "Slide for side-by-side content, like comparisons.",
 "selected_layout_index":
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
 },
 "Content_child": { ## # title, subtitle and object
 "description": "Default slide for a standard topic with bullet
↪points.",

```

```

 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
 },
 "Content_Two_Column_child": { ## title, subtitle and 2 objects
 "description": "Slide for side-by-side content, like comparisons.",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
 },
 "Application": {# title and object
 "description": "Slide for interactive questions ('Let's Apply This!
↪').",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
 },
 "Application_Two_Column ": { # title and 2 objects
 "description": "Slide for side-by-side content ('Let's Apply This!
↪').",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
 },
 "Summary": { # title and object
 "description": "The final summary slide of the deck.",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
 },
 "End": { # title
 "description": "The final 'Thank You / Questions?' slide.",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "title_support",␣
↪"centered_title", fallback_index=7)
 },
 "Divider": { # title and object
 "description": "Slide that divide sections general use base on the␣
↪agenda.",
 "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "title_support",␣
↪"centered_title", fallback_index=7)
 }
}

config_to_save = {
 ///_INSTRUCTIONS: "This file describes the available slide layouts.␣
↪The 'available_layouts' section is for an LLM to read. The 'user_selections'␣
↪section is for you to edit. Please verify the 'selected_layout_index' for␣
↪each slide type.",
 "template_file": os.path.basename(template_path),

```

```

 "user_selections": user_selection_map,
 "available_layouts": available_layouts
 }

 # --- 3. Save the Configuration File ---
 try:
 with open(output_path, 'w', encoding='utf-8') as f:
 json.dump(config_to_save, f, indent=4)
 print_header("Configuration Generated Successfully", char="=")
 print(f"A new, human-readable configuration file has been saved to:
↪\n{output_path}")
 print("\nPlease open this file to review the classifications and edit
↪your layout selections.")
 except Exception as e:
 logger.error(f"Failed to write the layout configuration file to
↪'{output_path}'. Error: {e}")

--- Execution ---
SLIDE_TEMPLATE_PATH = "/home/sebas_dev_linux/projects/course_generator/data/
↪slide_style/slide_style_test_2.pptx"
LAYOUT_MAPPING_PATH = "/home/sebas_dev_linux/projects/course_generator/
↪configs/layout_mapping_test.json"
You run this function to generate the config file.
Make sure SLIDE_TEMPLATE_PATH and LAYOUT_MAPPING_PATH are defined.
inspect_and_generate_layout_config(SLIDE_TEMPLATE_PATH, LAYOUT_MAPPING_PATH)

```

#### 4 Extrac Unit outline details to process following steps - output raw json with UO details

```

[16]: # Cell 3: Parse Unit Outline

--- Helper Functions for Parsing ---
def extract_text_from_file(filepath: str) -> str:
 _, ext = os.path.splitext(filepath.lower())
 if ext == '.docx':
 doc = Document(filepath)
 full_text = [p.text for p in doc.paragraphs]
 for table in doc.tables:
 for row in table.rows:
 full_text.append(" | ".join(cell.text for cell in row.cells))
 return '\n'.join(full_text)
 elif ext == '.pdf':
 with pdfplumber.open(filepath) as pdf:
 return "\n".join(page.extract_text() for page in pdf.pages if page.
↪extract_text())

```

```

else:
 raise TypeError(f"Unsupported file type: {ext}")

def parse_llm_json_output(content: str) -> dict:
 try:
 match = re.search(r'\{.*\}', content, re.DOTALL)
 if not match: return None
 return json.loads(match.group(0))
 except (json.JSONDecodeError, TypeError):
 return None

@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=2, max=10))
def call_ollama_with_retry(client, prompt):
 logger.info(f"Calling Ollama model '{OLLAMA_MODEL}'...")
 response = client.chat(
 model=OLLAMA_MODEL,
 messages=[{"role": "user", "content": prompt}],
 format="json",
 options={"temperature": 0.0}
)
 if not response or 'message' not in response or not response['message'].
 get('content'):
 raise ValueError("Ollama returned an empty or invalid response.")
 return response['message']['content']

--- Main Orchestration Function for this Cell ---
def parse_and_save_outline_robust(
 input_filepath: str,
 output_filepath: str,
 prompt_template: str,
 max_retries: int = 3
):
 logger.info(f"Starting to robustly process Unit Outline: {input_filepath}")

 if not os.path.exists(input_filepath):
 logger.error(f"Input file not found: {input_filepath}")
 return

 try:
 outline_text = extract_text_from_file(input_filepath)
 if not outline_text.strip():
 logger.error("Extracted text is empty. Aborting.")
 return
 except Exception as e:
 logger.error(f"Failed to extract text from file: {e}", exc_info=True)
 return

```

```

client = ollama.Client(host=OLLAMA_HOST)
current_prompt = prompt_template.format(outline_text=outline_text)

for attempt in range(max_retries):
 logger.info(f"Attempt {attempt + 1}/{max_retries} to parse outline.")

 try:
 # Call the LLM
 llm_output_str = call_ollama_with_retry(client, current_prompt)

 # Find the JSON blob in the response
 json_blob = parse_llm_json_output(llm_output_str) # Your existing
↪helper

 if not json_blob:
 raise ValueError("LLM did not return a parsable JSON object.")

 # *** THE KEY VALIDATION STEP ***
 # Try to parse the dictionary into your Pydantic model.
 # This will raise a `ValidationError` if keys are wrong, types are
↪wrong, or fields are missing.
 parsed_data = ParsedUnitOutline.model_validate(json_blob)

 # If successful, save the validated data and exit the loop
 logger.info("Successfully validated JSON structure against Pydantic
↪model.")

 os.makedirs(os.path.dirname(output_filepath), exist_ok=True)
 with open(output_filepath, 'w', encoding='utf-8') as f:
 # Use .model_dump_json() for clean, validated output
 f.write(parsed_data.model_dump_json(indent=2))

 logger.info(f"Successfully parsed and saved Unit Outline to:
↪{output_filepath}")
 return # Exit function on success

 except ValidationError as e:
 logger.warning(f"Validation failed on attempt {attempt + 1}. Error:
↪{e}")

 # Formulate a new prompt with the error message for self-correction
 error_feedback = (
 f"\n\nYour previous attempt failed. You MUST correct the
↪following errors:\n"
 f"{e}\n\n"
 f"Please regenerate the entire JSON object, ensuring it
↪strictly adheres to the schema "
 f"and corrects these specific errors. Do not change any key
↪names."

```

```

)
 current_prompt = current_prompt + error_feedback # Append the error
↳to the prompt

 except Exception as e:
 # Catch other errors like network issues from call_ollama_with_retry
 logger.error(f"An unexpected error occurred on attempt {attempt + 1}: {e}", exc_info=True)
 # You might want to wait before retrying for non-validation errors
 time.sleep(5)

 logger.error(f"Failed to get valid structured data from the LLM after
↳{max_retries} attempts.")

--- In your execution block, call the new function ---
parse_and_save_outline(...) becomes:

if EXTRACT_UO:
 parse_and_save_outline_robust(
 input_filepath=FULL_PATH_UNIT_OUTLINE,
 output_filepath=PARSED_UO_JSON_PATH,
 prompt_template=UNIT_OUTLINE_SYSTEM_PROMPT_TEMPLATE
)

```

## 5 Extract TOC from epub or PDF

```

[17]: # Cell 4: Extract Book Table of Contents (ToC) with Pre-assigned IDs & Links in
↳Order

from ebooklib import epub, ITEM_NAVIGATION
from bs4 import BeautifulSoup
import fitz # PyMuPDF
import json
import os
from typing import List, Dict
import urllib.parse # Needed to clean up links

=====
1. HELPER FUNCTIONS (MODIFIED TO INCLUDE ID ASSIGNMENT AND LINK EXTRACTION)
=====

def clean_epub_href(href: str) -> str:
 """Removes URL fragments and decodes URL-encoded characters."""
 if not href: return ""
 # Remove fragment identifier (e.g., '#section1')

```

```

cleaned_href = href.split('#')[0]
Decode any URL-encoded characters (e.g., %20 -> space)
return urllib.parse.unquote(cleaned_href)

--- EPUB Extraction Logic ---
def parse_navpoint(navpoint: BeautifulSoup, counter: List[int], level: int = 0) → Dict:
 """Recursively parses EPUB 2 navPoints and assigns a toc_id and link_filename."""
 title = navpoint.navLabel.text.strip()
 if not title: return None

 # --- MODIFICATION: Extract the linked filename ---
 content_tag = navpoint.find('content', recursive=False)
 link_filename = clean_epub_href(content_tag['src']) if content_tag else ""

 node = {
 "level": level,
 "toc_id": counter[0],
 "title": title,
 "link_filename": link_filename, # Add the cleaned link
 "children": []
 }
 counter[0] += 1

 for child_navpoint in navpoint.find_all('navPoint', recursive=False):
 child_node = parse_navpoint(child_navpoint, counter, level + 1)
 if child_node: node["children"].append(child_node)

 return node

def parse_li(li_element: BeautifulSoup, counter: List[int], level: int = 0) → Dict:
 """Recursively parses EPUB 3 elements and assigns a toc_id and link_filename."""
 a_tag = li_element.find('a', recursive=False)
 if a_tag:
 title = a_tag.get_text(strip=True)
 if not title: return None

 # --- MODIFICATION: Extract the linked filename ---
 link_filename = clean_epub_href(a_tag.get('href'))

 node = {
 "level": level,
 "toc_id": counter[0],
 "title": title,

```

```

 "link_filename": link_filename, # Add the cleaned link
 "children": []
 }
 counter[0] += 1

 nested_ol = li_element.find('ol', recursive=False)
 if nested_ol:
 for sub_li in nested_ol.find_all('li', recursive=False):
 child_node = parse_li(sub_li, counter, level + 1)
 if child_node: node["children"].append(child_node)
 return node
 return None

def extract_epub_toc(epub_path, output_json_path):
 print(f"Processing EPUB ToC for: {epub_path}")
 toc_data = []
 book = epub.read_epub(epub_path)
 id_counter = [0]

 for nav_item in book.get_items_of_type(ITEM_NAVIGATION):
 soup = BeautifulSoup(nav_item.get_content(), 'xml')
 # Logic to handle both EPUB 2 (NCX) and EPUB 3 (XHTML)
 if nav_item.get_name().endswith('.ncx'):
 print("INFO: Found EPUB 2 (NCX) Table of Contents. Parsing...")
 navmap = soup.find('navMap')
 if navmap:
 for navpoint in navmap.find_all('navPoint', recursive=False):
 node = parse_navpoint(navpoint, id_counter, level=0)
 if node: toc_data.append(node)
 else: # Assumes EPUB 3
 print("INFO: Found EPUB 3 (XHTML) Table of Contents. Parsing...")
 toc_nav = soup.select_one('nav[epub:type="toc"]')
 if toc_nav:
 top_ol = toc_nav.find('ol', recursive=False)
 if top_ol:
 for li in top_ol.find_all('li', recursive=False):
 node = parse_li(li, id_counter, level=0)
 if node: toc_data.append(node)
 if toc_data: break

 if toc_data:
 os.makedirs(os.path.dirname(output_json_path), exist_ok=True)
 with open(output_json_path, 'w', encoding='utf-8') as f:
 json.dump(toc_data, f, indent=2, ensure_ascii=False)
 print(f" Successfully wrote EPUB ToC with IDs and links to: ↵
↵{output_json_path}")
 else:

```

```

 print(" WARNING: No ToC data extracted from EPUB.")

--- PDF Extraction Logic (Unchanged) ---
def build_pdf_hierarchy_with_ids(toc_list: List) -> List[Dict]:
 root = []
 parent_stack = {-1: {"children": root}}
 id_counter = [0]
 for level, title, page in toc_list:
 normalized_level = level - 1
 node = {"level": normalized_level, "toc_id": id_counter[0], "title": title.strip(), "page": page, "children": []}
 id_counter[0] += 1
 parent_node = parent_stack.get(normalized_level - 1)
 if parent_node: parent_node["children"].append(node)
 parent_stack[normalized_level] = node
 return root

def extract_pdf_toc(pdf_path, output_json_path):
 print(f"Processing PDF ToC for: {pdf_path}")
 try:
 doc = fitz.open(pdf_path)
 toc = doc.get_toc()
 hierarchical_toc = []
 if not toc: print(" WARNING: This PDF has no embedded bookmarks (ToC).")
 else:
 print(f"INFO: Found {len(toc)} bookmark entries. Building hierarchy and assigning IDs...")
 hierarchical_toc = build_pdf_hierarchy_with_ids(toc)
 os.makedirs(os.path.dirname(output_json_path), exist_ok=True)
 with open(output_json_path, 'w', encoding='utf-8') as f:
 json.dump(hierarchical_toc, f, indent=2, ensure_ascii=False)
 print(f" Successfully wrote PDF ToC with assigned IDs to: {output_json_path}")
 except Exception as e: print(f"An error occurred during PDF ToC extraction: {e}")

=====
2. EXECUTION BLOCK
=====

if PROCESS_EPUB:
 extract_epub_toc(BOOK_PATH, PRE_EXTRACTED_TOC_JSON_PATH)
else:
 extract_pdf_toc(BOOK_PATH, PRE_EXTRACTED_TOC_JSON_PATH)

```

Processing EPUB ToC for:  
/home/sebas\_dev\_linux/projects/course\_generator/data/books/Bill Nelson, Amelia

Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_  
Processing Digital Evidence-Cengage Learning (2018).epub  
INFO: Found EPUB 2 (NCX) Table of Contents. Parsing..  
Successfully wrote EPUB ToC with IDs and links to:  
/home/sebas\_dev\_linux/projects/course\_generator/data\_output/test\_Output/1.  
Parse\_data/Parse\_TOC\_books/ICT312/ICT312\_epub\_table\_of\_contents.json

## 6 Hirachical DB base on TOC

### 6.1 Process Book

```
[18]: # Cell 5: Create Hierarchical Vector Database (with Sequential ToC ID and Chunk
 ↪ ID)
 # This cell processes the book, enriches it with hierarchical and sequential
 ↪ metadata,
 # chunks it, and creates the final vector database.

import os
import json
import shutil
import logging
from typing import List, Dict, Any, Tuple
from langchain_core.documents import Document
from langchain_community.document_loaders import PyPDFLoader,
 ↪ UnstructuredEPubLoader
from langchain_ollama.embeddings import OllamaEmbeddings
from langchain_chroma import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter

Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
 ↪ %(message)s')
logger = logging.getLogger(__name__)

--- Helper: Clean metadata values for ChromaDB ---
def clean_metadata_for_chroma(value: Any) -> Any:
 """Sanitizes metadata values to be compatible with ChromaDB."""
 if isinstance(value, list): return ", ".join(map(str, value))
 if isinstance(value, dict): return json.dumps(value)
 if isinstance(value, (str, int, float, bool)) or value is None: return value
 return str(value)

--- Core Function to Process Book with Pre-extracted ToC ---
def process_book_with_extracted_toc(
 book_path: str,
 extracted_toc_json_path: str,
 chunk_size: int,
```

```

 chunk_overlap: int
) -> Tuple[List[Document], List[Dict[str, Any]]]:

 logger.info(f"Processing book '{os.path.basename(book_path)}' using ToC
↳from '{os.path.basename(extracted_toc_json_path)}'.")

 # 1. Load the pre-extracted hierarchical ToC
 try:
 with open(extracted_toc_json_path, 'r', encoding='utf-8') as f:
 hierarchical_toc = json.load(f)
 if not hierarchical_toc:
 logger.error(f"Pre-extracted ToC at '{extracted_toc_json_path}' is
↳empty or invalid.")
 return [], []
 logger.info(f"Successfully loaded pre-extracted ToC with
↳{len(hierarchical_toc)} top-level entries.")
 except Exception as e:
 logger.error(f"Error loading pre-extracted ToC JSON: {e}",
↳exc_info=True)
 return [], []

 # 2. Load all text elements/pages from the book
 all_raw_book_docs: List[Document] = []
 _, file_extension = os.path.splitext(book_path.lower())

 if file_extension == ".epub":
 loader = UnstructuredEPubLoader(book_path, mode="elements",
↳strategy="fast")
 try:
 all_raw_book_docs = loader.load()
 logger.info(f"Loaded {len(all_raw_book_docs)} text elements from
↳EPUB.")
 except Exception as e:
 logger.error(f"Error loading EPUB content: {e}", exc_info=True)
 return [], hierarchical_toc
 elif file_extension == ".pdf":
 loader = PyPDFLoader(book_path)
 try:
 all_raw_book_docs = loader.load()
 logger.info(f"Loaded {len(all_raw_book_docs)} pages from PDF.")
 except Exception as e:
 logger.error(f"Error loading PDF content: {e}", exc_info=True)
 return [], hierarchical_toc
 else:
 logger.error(f"Unsupported book file format: {file_extension}")
 return [], hierarchical_toc

```

```

if not all_raw_book_docs:
 logger.error("No text elements/pages loaded from the book.")
 return [], hierarchical_toc

3. Create enriched LangChain Documents by matching ToC to content
final_documents_with_metadata: List[Document] = []

Flatten the ToC, AND add a unique sequential ID for sorting and
validation.
flat_toc_entries: List[Dict[str, Any]] = []

def _add_ids_and_flatten_recursive(nodes: List[Dict[str, Any]],
current_titles_path: List[str], counter: List[int]):
 """
 Recursively traverses ToC nodes to flatten them and assign a unique,
sequential toc_id.
 """
 for node in nodes:
 toc_id = counter[0]
 counter[0] += 1
 title = node.get("title", "").strip()
 if not title: continue
 new_titles_path = current_titles_path + [title]
 entry = {
 "titles_path": new_titles_path,
 "level": node.get("level"),
 "full_title_for_matching": title,
 "toc_id": toc_id
 }
 if "page" in node: entry["page"] = node["page"]
 flat_toc_entries.append(entry)
 if node.get("children"):
 _add_ids_and_flatten_recursive(node.get("children", []),
new_titles_path, counter)

 toc_id_counter = [0]
 _add_ids_and_flatten_recursive(hierarchical_toc, [], toc_id_counter)
 logger.info(f"Flattened ToC and assigned sequential IDs to
{len(flat_toc_entries)} entries.")

Logic for PDF metadata assignment
if file_extension == ".pdf" and any("page" in entry for entry in
flat_toc_entries):
 logger.info("Assigning metadata to PDF pages based on ToC page numbers..")

```

```

 flat_toc_entries.sort(key=lambda x: x.get("page", -1) if x.get("page")
↪is not None else -1)
 for page_doc in all_raw_book_docs:
 page_num_0_indexed = page_doc.metadata.get("page", -1)
 page_num_1_indexed = page_num_0_indexed + 1
 assigned_metadata = {"source": os.path.basename(book_path),
↪"page_number": page_num_1_indexed}
 best_match_toc_entry = None
 for toc_entry in flat_toc_entries:
 toc_page = toc_entry.get("page")
 if toc_page is not None and toc_page <= page_num_1_indexed:
 if best_match_toc_entry is None or toc_page >
↪best_match_toc_entry.get("page", -1):
 best_match_toc_entry = toc_entry
 elif toc_page is not None and toc_page > page_num_1_indexed:
 break
 if best_match_toc_entry:
 for i, title_in_path in
↪enumerate(best_match_toc_entry["titles_path"]):
 assigned_metadata[f"level_{i+1}_title"] = title_in_path
 assigned_metadata['toc_id'] = best_match_toc_entry.get('toc_id')
 else:
 assigned_metadata["level_1_title"] = "Uncategorized PDF Page"
 cleaned_meta = {k: clean_metadata_for_chroma(v) for k, v in
↪assigned_metadata.items()}
 final_documents_with_metadata.append(Document(page_content=page_doc.
↪page_content, metadata=cleaned_meta))

 # Logic for EPUB metadata assignment
 elif file_extension == ".epub":
 logger.info("Assigning metadata to EPUB elements by matching ToC titles
↪in text...")
 toc_titles_for_search = [entry for entry in flat_toc_entries if entry.
↪get("full_title_for_matching")]
 current_hierarchy_metadata = {}
 for element_doc in all_raw_book_docs:
 element_text = element_doc.page_content.strip() if element_doc.
↪page_content else ""
 if not element_text: continue
 for toc_entry in toc_titles_for_search:
 if element_text == toc_entry["full_title_for_matching"]:
 current_hierarchy_metadata = {"source": os.path.
↪basename(book_path)}
 for i, title_in_path in enumerate(toc_entry["titles_path"]):
 current_hierarchy_metadata[f"level_{i+1}_title"] =
↪title_in_path

```

```

 current_hierarchy_metadata['toc_id'] = toc_entry.
↪get('toc_id')
 if "page" in toc_entry:
↪current_hierarchy_metadata["epub_toc_page"] = toc_entry["page"]
 break
 if not current_hierarchy_metadata:
 doc_metadata_to_assign = {"source": os.path.
↪basename(book_path), "level_1_title": "EPUB Preamble", "toc_id": -1}
 else:
 doc_metadata_to_assign = current_hierarchy_metadata.copy()
 cleaned_meta = {k: clean_metadata_for_chroma(v) for k, v in
↪doc_metadata_to_assign.items()}
 final_documents_with_metadata.
↪append(Document(page_content=element_text, metadata=cleaned_meta))

 else: # Fallback
 final_documents_with_metadata = all_raw_book_docs

 if not final_documents_with_metadata:
 logger.error("No documents were processed or enriched with hierarchical_
↪metadata.")
 return [], hierarchical_toc

 logger.info(f"Total documents prepared for chunking:
↪{len(final_documents_with_metadata)}")

 text_splitter = RecursiveCharacterTextSplitter(
 chunk_size=chunk_size,
 chunk_overlap=chunk_overlap,
 length_function=len
)
 final_chunks = text_splitter.split_documents(final_documents_with_metadata)
 logger.info(f"Split into {len(final_chunks)} final chunks, inheriting_
↪hierarchical metadata.")

 # --- MODIFICATION START: Add a unique, sequential chunk_id to each chunk_
↪---
 logger.info("Assigning sequential chunk_id to all final chunks...")
 for i, chunk in enumerate(final_chunks):
 chunk.metadata['chunk_id'] = i
 logger.info(f"Assigned chunk_ids from 0 to {len(final_chunks) - 1}.")
 # --- MODIFICATION END ---

 return final_chunks, hierarchical_toc

--- Main Execution Block for this Cell ---

```

```

if CREATE_RAG_BOOK:
 if not os.path.exists(PRE_EXTRACTED_TOC_JSON_PATH):
 logger.error(f"CRITICAL: Pre-extracted ToC file not found at_
↳ '{PRE_EXTRACTED_TOC_JSON_PATH}'".)
 logger.error("Please run the 'Extract Book Table of Contents (ToC)'_
↳ cell (Cell 4) first.")
 else:
 final_chunks_for_db, toc_reloaded = process_book_with_extracted_toc(
 book_path=BOOK_PATH,
 extracted_toc_json_path=PRE_EXTRACTED_TOC_JSON_PATH,
 chunk_size=CHUNK_SIZE,
 chunk_overlap=CHUNK_OVERLAP
)

 if final_chunks_for_db:
 if os.path.exists(CHROMA_PERSIST_DIR):
 logger.warning(f"Deleting existing ChromaDB directory:_
↳ '{CHROMA_PERSIST_DIR}'")
 shutil.rmtree(CHROMA_PERSIST_DIR)

 logger.info(f"Initializing embedding model_
↳ '{EMBEDDING_MODEL_OLLAMA}' and creating new vector database...")
 embedding_model = OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA)

 vector_db = Chroma.from_documents(
 documents=final_chunks_for_db,
 embedding=embedding_model,
 persist_directory=CHROMA_PERSIST_DIR,
 collection_name=CHROMA_COLLECTION_NAME
)

 reloaded_db = Chroma(persist_directory=CHROMA_PERSIST_DIR,_
↳ embedding_function=embedding_model, collection_name=CHROMA_COLLECTION_NAME)
 count = reloaded_db._collection.count()

 print("-" * 50)
 logger.info(f" Vector DB created successfully at:_
↳ '{CHROMA_PERSIST_DIR}'")
 logger.info(f" Collection '{CHROMA_COLLECTION_NAME}' contains_
↳ {count} documents.")
 print("-" * 50)
 else:
 logger.error(" Failed to generate chunks. Vector DB not created.")

```

[19]: # Cell 5a: Inspecting EPUB Documents and Metadata BEFORE Chunking

```

import json
import os
import logging
from langchain_community.document_loaders import UnstructuredEPubLoader
from langchain_core.documents import Document

--- Setup Logger for this inspection cell ---
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
↳%(message)s')

def inspect_epub_preprocessing():
 """
 This function replicates the pre-chunking logic from Cell 5 for EPUB files
 to show the list of large documents with their assigned ToC metadata.
 """
 if not PROCESS_EPUB:
 print("This inspection cell is for EPUB processing. Please set
↳PROCESS_EPUB = True in Cell 1.")
 return

 print_header("EPUB Pre-Processing Inspection", char="~")

 # --- 1. Load the necessary data (replicating start of Cell 5) ---
 logger.info("Loading pre-extracted ToC and raw EPUB elements...")
 try:
 with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
 hierarchical_toc = json.load(f)

 loader = UnstructuredEPubLoader(BOOK_PATH, mode="elements",
↳strategy="fast")
 all_raw_book_docs = loader.load()
 logger.info(f"Successfully loaded {len(all_raw_book_docs)} raw text
↳elements from the EPUB.")
 except Exception as e:
 logger.error(f"Failed to load necessary files: {e}")
 return

 # --- 2. Flatten the ToC (replicating logic from Cell 5) ---
 logger.info("Flattening the hierarchical ToC for matching...")
 flat_toc_entries = []
 def _add_ids_and_flatten_recursive(nodes, current_titles_path, counter):
 for node in nodes:
 toc_id = counter[0]
 counter[0] += 1
 title = node.get("title", "").strip()
 if not title: continue

```

```

 new_titles_path = current_titles_path + [title]
 entry = {
 "titles_path": new_titles_path,
 "level": node.get("level"),
 "full_title_for_matching": title,
 "toc_id": toc_id
 }
 flat_toc_entries.append(entry)
 if node.get("children"):
 _add_ids_and_flatten_recursive(node.get("children", []),
↪new_titles_path, counter)

 _add_ids_and_flatten_recursive(hierarchical_toc, [], [0])
 logger.info(f"Flattened ToC into {len(flat_toc_entries)} entries.")

 # --- 3. The Core Matching Logic for EPUB (the part you want to see) ---
 logger.info("Assigning metadata to EPUB elements by matching ToC titles...")

 final_documents_with_metadata = []
 toc_titles_for_search = [entry for entry in flat_toc_entries if entry.
↪get("full_title_for_matching")]
 current_hierarchy_metadata = {}

 for element_doc in all_raw_book_docs:
 element_text = element_doc.page_content.strip() if element_doc.
↪page_content else ""
 if not element_text:
 continue

 # Check if this element is a heading that matches a ToC entry
 is_heading = False
 for toc_entry in toc_titles_for_search:
 if element_text == toc_entry["full_title_for_matching"]:
 # It's a heading! Update the current context.
 current_hierarchy_metadata = {"source": os.path.
↪basename(BOOK_PATH)}
 for i, title_in_path in enumerate(toc_entry["titles_path"]):
 current_hierarchy_metadata[f"level_{i+1}_title"] =
↪title_in_path
 current_hierarchy_metadata['toc_id'] = toc_entry.get('toc_id')
 is_heading = True
 break # Found the match, no need to search further

 # Assign metadata
 if not current_hierarchy_metadata:
 # Content before the first ToC entry (e.g., cover, title page)

```

```

 doc_metadata_to_assign = {"source": os.path.basename(BOOK_PATH),
↪ "level_1_title": "EPUB Preamble", "toc_id": -1}
 else:
 doc_metadata_to_assign = current_hierarchy_metadata.copy()

 final_documents_with_metadata.
↪ append(Document(page_content=element_text, metadata=doc_metadata_to_assign))

 logger.info(f"Processing complete. Generated_
↪ {len(final_documents_with_metadata)} documents with assigned metadata.")

 # --- 4. Print the result for inspection ---
 print_header("INSPECTION RESULTS: Documents Before Chunking", char "=")
 print(f"Total documents created: {len(final_documents_with_metadata)}\n")

 for i, doc in enumerate(final_documents_with_metadata[:100]): # Print first_
↪ 30 to avoid flooding the output
 print(f"--- Document [{i+1}] ---")
 print(f" Assigned Metadata: {doc.metadata}")
 print(f" Content (Un-chunked Element):")
 print(f" >> '{doc.page_content}'")
 print("-" * 25 + "\n")

 # --- Execute the inspection ---
 inspect_epub_preprocessing()

```

2025-07-18 23:25:35,354 - INFO - Loading pre-extracted ToC and raw EPUB elements...

~~~~~  
EPUB Pre-Processing Inspection  
~~~~~

[WARNING] Could not load translations for en-US

data file translations/en.yaml not found

2025-07-18 23:25:39,678 - WARNING - Could not load translations for en-US

data file translations/en.yaml not found

[WARNING] The term Abstract has no translation defined.

2025-07-18 23:25:39,679 - WARNING - The term Abstract has no translation defined.

2025-07-18 23:25:42,923 - INFO - Successfully loaded 11815 raw text elements from the EPUB.

2025-07-18 23:25:42,923 - INFO - Flattening the hierarchical ToC for matching...

2025-07-18 23:25:42,924 - INFO - Flattened ToC into 877 entries.

2025-07-18 23:25:42,924 - INFO - Assigning metadata to EPUB elements by matching ToC titles...

2025-07-18 23:25:43,114 - INFO - Processing complete. Generated 11483 documents with assigned metadata.

```
=====
 INSPECTION RESULTS: Documents Before Chunking
=====
```

Total documents created: 11483

--- Document [1] ---

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
```

Content (Un-chunked Element):

```
>> 'Guide to Computer Forensics and Investigations: Processing Digital
Evidence'
```

--- Document [2] ---

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
```

Content (Un-chunked Element):

```
>> 'Copyright Statement'
```

--- Document [3] ---

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
```

Content (Un-chunked Element):

```
>> 'Guide to Computer Forensics and Investigations: Processing Digital
Evidence'
```

--- Document [4] ---

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
```

Content (Un-chunked Element):

```
>> 'COPYRIGHT © 2019, 2016 Cengage Learning, Inc.'
```

--- Document [5] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

>> 'ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced or distributed in any form or by any means, except as permitted by U.S. copyright law, without the prior written permission of the copyright owner.'

-----

--- Document [6] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

>> 'For product information and technology assistance, contact us at Cengage Customer & Sales Support, 1-800-354-9706 or support.cengage.com.'

-----

--- Document [7] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

>> 'For permission to use material from this text or product, submit all requests online at [www.cengage.com/permissions](http://www.cengage.com/permissions).'

-----

--- Document [8] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

>> 'SOURCE FOR ILLUSTRATIONS: Copyright © Cengage.'

-----

--- Document [9] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

```

>> ', Microsoft® is a registered trademark of the Microsoft Corporation.'

--- Document [10] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'Library of Congress Control Number: 2018936389'

--- Document [11] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'ISBN: 978-1-337-56894-4'

--- Document [12] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'Cengage'

--- Document [13] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> '20 Channel Center Street'

--- Document [14] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'Boston MA 02210'

```

```

--- Document [15] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'USA'

--- Document [16] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'Cengage is a leading provider of customized learning solutions with
employees residing in nearly 40 different countries and sales in more than 125
countries around the world. Find your local representative at www.cengage.com.'

--- Document [17] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'Cengage products are represented in Canada by Nelson Education, Ltd.'

--- Document [18] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'To learn more about Cengage platforms and services, visit
www.cengage.com.'

--- Document [19] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
Content (Un-chunked Element):
>> 'To register or access your online learning solution or purchase materials
for your course, visit www.cengagebrain.com.'

```

--- Document [20] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'EPUB Preamble', 'toc\_id': -1}

Content (Un-chunked Element):

>> 'Notice to the Reader Publisher does not warrant or guarantee any of the products described herein or perform any independent analysis in connection with any of the product information contained herein. Publisher does not assume, and expressly disclaims, any obligation to obtain and include information other than that provided to it by the manufacturer. The reader is expressly warned to consider and adopt all safety precautions that might be indicated by the activities described herein and to avoid all potential hazards. By following the instructions contained herein, the reader willingly assumes all risks in connection with such instructions. The publisher makes no representations or warranties of any kind, including but not limited to, the warranties of fitness for particular purpose or merchantability, nor are any such representations implied with respect to the material set forth herein, and the publisher takes no responsibility with respect to such material. The publisher shall not be liable for any special, consequential, or exemplary damages resulting, in whole or part, from the readers' use of, or reliance upon, this material.'

-----

--- Document [21] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}

Content (Un-chunked Element):

>> 'Preface'

-----

--- Document [22] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}

Content (Un-chunked Element):

>> 'Guide to Computer Forensics and Investigations is now in its sixth edition. As digital technology and cyberspace have evolved from their early roots as basic communication platforms into the hyper-connected world we live in today, so has the demand for people who have the knowledge and skills to investigate legal and technical issues involving computers and digital technology. My sincere compliments to the authors and publishing staff who have made this textbook such a remarkable resource for thousands of students and practitioners worldwide.'

-----

--- Document [23] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}  
Content (Un-chunked Element):

>> 'Computers, the Internet, and the world's digital ecosystem are all instrumental in how we conduct our daily lives. When the founding fathers of the modern computing era were designing the digital infrastructure as we know it today, security and temporal accountability issues were not at the top of their list of things to do. The technological advancement of these systems over the past 10 years has changed the way we learn, socialize, and conduct business. Finding digital data that can be used as evidence to incriminate or exonerate a suspect accused in a legal or administrative proceeding is not an easy task.'

-----

--- Document [24] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}  
Content (Un-chunked Element):

>> 'Cyberthreats have become pervasive in modern society. They range from simple computer viruses to complex ransomware and cyber extortion schemes. The ability to conduct sophisticated digital forensics investigations has become a requirement in both the government and commercial sectors. Currently, the organizations and agencies whose job it is to investigate both criminal and civil matters involving the use of rapidly developing digital technology often struggle to keep up with the ever-changing digital landscape. Additionally, finding trained and qualified people to conduct these types of inquiries has been challenging.'

-----

--- Document [25] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}  
Content (Un-chunked Element):

>> 'Today, an entire industry has evolved for the purpose of investigating events occurring in cyberspace to include incidents involving international and corporate espionage, massive data breaches, and even cyberterrorism. The opportunities for employment in this field are expanding every day. Professionals in this exciting field of endeavor are now in high demand and are expected to have multiple skill sets in areas such as malware analysis, cloud computing, social media, and mobile device forensics.'

-----

--- Document [26] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Preface', 'toc\_id': 3}

```
Content (Un-chunked Element):
>> 'Guide to Computer Forensics and Investigations can now be found in both
academic and professional environments as a reliable source of current technical
information and practical exercises concerning investigations involving the
latest digital technologies. It's my belief that this book, combined with an
enthusiastic and knowledgeable facilitator, makes for a fascinating course of
instruction.'
```

-----

```
--- Document [27] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
```

```
Content (Un-chunked Element):
>> 'As I have stated to many of my students in the past, it's not just laptop
computers and servers that harbor the binary code of ones and zeros, but an
infinite array of digital devices. If one of these devices retains evidence of a
crime, it's up to newly trained and educated digital detectives to find the
evidence in a forensically sound manner. This book will assist both students and
practitioners in accomplishing this goal.'
```

-----

```
--- Document [28] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
```

```
Content (Un-chunked Element):
>> 'Respectfully,'
```

-----

```
--- Document [29] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
```

```
Content (Un-chunked Element):
>> 'John A. Sgromolo'
```

-----

```
--- Document [30] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
```

```
Content (Un-chunked Element):
>> 'As a Senior Special Agent, John was one of the founding members of the
NCIS Computer Crime Investigations Group. John left government service to run
his own company, Digital Forensics, Inc., and has taught hundreds of law
enforcement and corporate students nationwide in the art and science of digital
forensics investigations. Currently, he serves as a senior consultant for
```

Verizon's Global Security Services, where he helps manage the Threat Intel Response Service.'

-----

--- Document [31] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Introduction'

-----

--- Document [32] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Computer forensics, now most commonly called "digital forensics," has been a professional field for many years, but most well-established experts in the field have been self-taught. The growth of the Internet and the worldwide proliferation of computers have increased the need for digital investigations. Computers can be used to commit crimes, and crimes can be recorded on computers, including company policy violations, embezzlement, e-mail harassment, murder, leaks of proprietary information, and even terrorism. Law enforcement, network administrators, attorneys, and private investigators now rely on the skills of professional digital forensics experts to investigate criminal and civil cases.'

-----

--- Document [33] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'This book is not intended to provide comprehensive training in digital forensics. It does, however, give you a solid foundation by introducing digital forensics to those who are new to the field. Other books on digital forensics are targeted to experts; this book is intended for novices who have a thorough grounding in computer and networking basics.'

-----

--- Document [34] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

```
Content (Un-chunked Element):
>> 'The new generation of digital forensics experts needs more initial
training because operating systems, computer and mobile device hardware, and
forensics software tools are changing more quickly. This book covers current and
past operating systems and a range of hardware, from basic workstations and
high-end network servers to a wide array of mobile devices. Although this book
focuses on a few forensics software tools, it also reviews and discusses other
currently available tools.'
```

```

--- Document [35] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
>> 'The purpose of this book is to guide you toward becoming a skilled digital
forensics investigator. A secondary goal is to help you pass related
certification exams. As the field of digital forensics and investigations
matures, keep in mind that certifications will change. You can find more
information on certifications in Chapter 2 and Appendix A.'
```

```

--- Document [36] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
>> 'Intended Audience'
```

```

--- Document [37] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
>> 'Although this book can be used by people with a wide range of backgrounds,
it's intended for those with A+ and Network+ certifications or the equivalent. A
networking background is necessary so that you understand how computers operate
in a networked environment and can work with a network administrator when
needed. In addition, you must know how to use a computer from the command line
and how to use common operating systems, including Windows, Linux, and macOS,
and their related hardware.'
```

```

--- Document [38] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
```

```
>> 'This book can be used at any educational level, from technical high
schools and community colleges to graduate students. Current professionals in
the public and private sectors can also use this book. Each group will approach
investigative problems from a different perspective, but all will benefit from
the coverage.'
```

```

```

```
--- Document [39] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
```

```
>> 'What's New in This Edition'
```

```

```

```
--- Document [40] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
```

```
>> 'The chapter flow of this book is organized so that you're first exposed to
what happens in a forensics lab and how to set one up before you get into the
nuts and bolts. Coverage of several GUI tools has been added to give you a
familiarity with some widely used software. In addition, Chapter 11 has
additional coverage of social media forensics, Chapter 12 has been expanded to
include more information on smartphones and tablets, and Chapter 13 on forensics
procedures for information stored in the cloud has been updated. Corrections
have been made to this edition based on feedback from users, and all software
tools and Web sites have been updated to reflect what's current at the time of
publication. Finally, a new digital lab manual is being offered in MindTap for
Guide to Computer Forensics and Investigationsto go with the sixth edition
textbook.'
```

```

```

```
--- Document [41] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
```

```
Content (Un-chunked Element):
```

```
>> 'Chapter Descriptions'
```

```

--- Document [42] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Here is a summary of the topics covered in each chapter of this book:'

--- Document [43] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 1 , "Understanding the Digital Forensics Profession and
Investigations," introduces you to the history of digital forensics and explains
how the use of electronic evidence developed. It also reviews legal issues and
compares public and private sector cases. This chapter also explains how to take
a systematic approach to preparing a digital investigation, describes how to
conduct an investigation, and summarizes requirements for workstations and
software.'
```

```

--- Document [44] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 2 , "The Investigator's Office and Laboratory," outlines physical
requirements and equipment for digital forensics labs, from small private
investigators' labs to the regional FBI lab. It also covers certifications for
digital investigators and building a business case for a forensics lab.'
```

```

--- Document [45] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 3 , "Data Acquisition," explains how to prepare to acquire data
from a suspect's drive and discusses available Linux and GUI acquisition tools.
This chapter also discusses acquiring data from RAID systems and gives you an
overview of tools for remote acquisitions.'
```

-----  
--- Document [46] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 4 , "Processing Crime and Incident Scenes," explains search warrants and the nature of a typical digital forensics case. It discusses when to use outside professionals, how to assemble a team, and how to evaluate a case and explains the correct procedures for searching and seizing evidence. This chapter also introduces you to calculating hashes to verify data you collect.'

-----

--- Document [47] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 5 , "Working with Windows and CLI Systems," discusses the most common operating systems. You learn what happens and what files are altered during computer startup and how file systems deal with deleted and slack space. In addition, this chapter covers some options for decrypting drives encrypted with whole disk encryption and explains the purpose of using virtual machines.'

-----

--- Document [48] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 6 , "Current Digital Forensics Tools," explores current digital forensics software and hardware tools, including those that might not be readily available, and evaluates their strengths and weaknesses.'

-----

--- Document [49] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 7 , "Linux and Macintosh File Systems," continues the operating system discussion from Chapter 5 by examining Macintosh and Linux OSs and file systems. It also gives you practice in using Linux forensics tools.'

```

--- Document [50] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 8 , "Recovering Graphics Files," explains how to recover graphics
files and examines data compression, carving data, reconstructing file
fragments, and steganography and copyright issues.'
```

```

--- Document [51] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 9 , "Digital Forensics Analysis and Validation," covers
determining what data to collect and analyze and refining investigation plans.
It also explains validation with hex editors and forensics software and data-
hiding techniques.'
```

```

--- Document [52] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 10 , "Virtual Machine Forensics, Live Acquisitions, and Network
Forensics," covers tools and methods for conducting forensic analysis of virtual
machines, performing live acquisitions, reviewing network logs for evidence, and
using network-monitoring tools to detect unauthorized access. It also examines
using Linux tools and the Honeynet Project's resources.'
```

```

--- Document [53] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
Content (Un-chunked Element):
>> 'Chapter 11 , "E-mail and Social Media Investigations," examines e-mail
crimes and violations and reviews some specialized e-mail and social media
forensics tools. It also explains how to approach investigating social media
communications and handling the challenges this content poses.'
```

-----  
--- Document [54] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 12 , "Mobile Device Forensics and The Internet of Anything," covers investigation techniques and acquisition procedures for smartphones, other mobile devices, Internet of Anything devices, and sensors. You learn where data might be stored or backed up and what tools are available for these investigations.'

-----

--- Document [55] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 13 , "Cloud Forensics," summarizes the legal and technical challenges in conducting cloud forensics. It also describes how to acquire cloud data and explains how remote acquisition tools can be used in cloud investigations.'

-----

--- Document [56] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 14 , "Report Writing for High-Tech Investigations," discusses the importance of report writing in digital forensics examinations; offers guidelines on report content, structure, and presentation; and explains how to generate report findings with forensics software tools.'

-----

--- Document [57] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 15 , "Expert Testimony in Digital Investigations," explores the role of an expert witness or a fact witness, including developing a curriculum vitae, understanding the trial process, and preparing forensics evidence for

testimony. It also offers guidelines for testifying in court and at depositions and hearings.'

-----  
--- Document [58] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter 16 , "Ethics for the Expert Witness," provides guidance in the principles and practice of ethics for digital forensics investigators and examines other professional organizations' codes of ethics.'

-----  
--- Document [59] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Appendix A , "Certification Test References," provides information on the National Institute of Standards and Technology (NIST) testing processes for validating digital forensics tools and covers digital forensics certifications and training programs.'

-----  
--- Document [60] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Appendix B , "Digital Forensics References," lists recommended books, journals, e-mail lists, and Web sites for additional information and further study. It also covers the latest ISO 27000 standards that apply to digital forensics.'

-----  
--- Document [61] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Appendix C , "Digital Forensics Lab Considerations," provides more information on considerations for forensics labs, including certifications, ergonomics, structural design, and communication and fire-suppression systems.'

It also covers applicable ISO standards.'

-----  
--- Document [62] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Appendix D , "Legacy File System and Forensics Tools," reviews FAT file system basics and Mac legacy file systems and explains using DOS forensics tools, creating forensic boot media, and using scripts. It also has an overview of the hexadecimal numbering system and how it's applied to digital information.'

-----  
--- Document [63] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Features'

-----  
--- Document [64] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'To help you fully understand digital forensics, this book includes many features designed to enhance your learning experience:'

-----  
--- Document [65] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}

Content (Un-chunked Element):

>> 'Chapter objectives-Each chapter begins with a detailed list of the concepts to be mastered in that chapter. This list gives you a quick reference to the chapter's contents and is a useful study aid.'

-----  
--- Document [66] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher

```

Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Figures and tables-Screenshots are used as guidelines for stepping through
commands and forensics tools. For tools not included with the book or that
aren't offered in free demo versions, figures have been added when possible to
illustrate the tool's interface. Tables are used throughout the book to present
information in an organized, easy-to-grasp manner.'

--- Document [67] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Chapter summaries-Each chapter's material is followed by a summary of the
concepts introduced in that chapter. These summaries are a helpful way to review
the ideas covered in each chapter.'

--- Document [68] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Key terms-Following the chapter summary, all new terms introduced in the
chapter with boldfaced text are gathered together in the Key Terms list. This
list encourages a more thorough understanding of the chapter's key concepts and
is a useful reference.'

--- Document [69] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Review questions-The end-of-chapter assessment begins with a set of review
questions that reinforce the main concepts in each chapter. These questions help
you evaluate and apply the material you have learned.'

--- Document [70] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital

```

```

Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Hands-on projects-Although understanding the theory behind digital
technology is important, nothing can improve on real-world experience. To this
end, each chapter offers several hands-on projects with software supplied as
free downloads on the student companion site and in MindTap. You can explore a
variety of ways to acquire and even hide evidence. For the conceptual chapters,
research projects are supplied.'

--- Document [71] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Case projects-At the end of each chapter are case projects. To do these
projects, you must draw on real-world common sense as well as your knowledge of
the technical topics covered to that point in the book. Your goal for each
project is to come up with answers to problems similar to those you'll face as a
working digital forensics investigator.'

--- Document [72] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Software and student data files-Student data files are available for
download from the student companion site and the MindTap for this book and are
used for activities and projects in the chapters. Demo and freeware software
used in this book can be downloaded from the Web sites specified in activities
and projects or in "Digital Forensics Software" later in this introduction.'

--- Document [73] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
 Content (Un-chunked Element):
 >> 'Student companion site-To access the student companion site, go to
www.cengagebrain.com and search for the sixth edition by entering the title,
author's name, or ISBN. On the product page, click the Free Materials tab, and
then click Save to MyHome. Then you can sign in as a returning student or choose
to create a new account. After you've logged on, you can begin accessing your

```

free study tools.'

-----  
--- Document [74] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Text and Graphic Conventions'  
-----

--- Document [75] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'When appropriate, additional information and exercises have been added to this book to help you better understand the topic at hand. The following icons used in this book alert you to additional materials:'  
-----

--- Document [76] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Note'  
-----

--- Document [77] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Note Box'  
-----

--- Document [78] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'The Note icon draws your attention to additional helpful material related

to the subject being covered.'

-----  
--- Document [79] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Tip'

-----  
--- Document [80] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Tips based on the authors' experiences offer extra information about how to attack a problem or what to do in real-world situations.'

-----  
--- Document [81] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Caution'

-----  
--- Document [82] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'Caution Box'

-----  
--- Document [83] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Introduction', 'toc\_id': 4}  
Content (Un-chunked Element):  
>> 'The Caution icon warns you about potential mistakes or problems and explains how to avoid them.'

```

--- Document [84] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Hands-On Projects',
'toc_id': 53}
 Content (Un-chunked Element):
 >> 'Hands-On Projects'

--- Document [85] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Hands-On Projects',
'toc_id': 53}
 Content (Un-chunked Element):
 >> 'The hands-on icon indicates that the projects following it give you a
chance to practice using software tools and get hands-on experience.'

--- Document [86] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
 Content (Un-chunked Element):
 >> 'Case Projects'

--- Document [87] ---
 Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
 Content (Un-chunked Element):
 >> 'This icon marks the start of projects that require you to apply common
sense and knowledge to solving problems involving that chapter's concepts.'

```

--- Document [88] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'Student Resources'

--- Document [89] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'MindTap for Guide to Computer Forensics and Investigations helps you learn on your terms.'

--- Document [90] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'Instant access in your pocket: Take advantage of the MindTap Mobile App to learn on your terms. Read or listen to textbooks and study with the aid of instructor notifications, flashcards, and practice quizzes.'

--- Document [91] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'MindTap helps you create your own potential. Gear up for ultimate success: Track your scores and stay motivated toward your goals. Whether you have more work to do or are ahead of the curve, you'll know where you need to focus your efforts. The MindTap Green Dot will charge your confidence along the way.'

-----  
--- Document [92] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'MindTap helps you own your progress. Make your textbook yours; no one knows what works for you better than you. Highlight key text, add notes, and create custom flashcards. When it's time to study, everything you've flagged or noted can be gathered into a guide you can organize.'

-----

--- Document [93] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'Lab Manual'

-----

--- Document [94] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

>> 'A new digital lab manual is being offered in the MindTap for Guide to Computer Forensics and Investigations to give you additional hands-on experience.'

-----

--- Document [95] ---

Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations\_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level\_1\_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level\_2\_title': 'Chapter Review', 'level\_3\_title': 'Case Projects', 'toc\_id': 54}

Content (Un-chunked Element):

```

>> 'Lab Requirements'

--- Document [96] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
Content (Un-chunked Element):
>> 'The hands-on projects in this book help you apply what you have learned
about digital forensics techniques. The following sections list the minimum
requirements for completing all the projects in this book. In addition to the
items listed, you must be able to download and install demo versions of
software.'

--- Document [97] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
Content (Un-chunked Element):
>> 'Note'

--- Document [98] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
Content (Un-chunked Element):
>> 'Note Box'

--- Document [99] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
Content (Un-chunked Element):

```

```
>> 'Magnet AXIOM has a 30-day trial for download. If you aren't purchasing its
academic license or are planning to do the labs at home, it's recommended that
you wait until Chapter 11 to begin using this tool so that your trial doesn't
expire before you finish the projects. In addition, wait until Chapter 11 before
doing Hands-On Project 6-5, installing Magnet AXIOM.'
```

```

```

```
--- Document [100] ---
```

```
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
```

```
Content (Un-chunked Element):
```

```
>> 'Note'
```

```

```

### 6.1.1 Full Database Health & Hierarchy Diagnostic Report

```
[20]: # Cell 5.1: Full Database Health & Hierarchy Diagnostic Report (V5 - with
↳Content Preview)
```

```
import os
import json
import logging
import random
from typing import List, Dict, Any

You might need to install pandas if you haven't already
try:
 import pandas as pd
 pandas_available = True
except ImportError:
 pandas_available = False

try:
 from langchain_chroma import Chroma
 from langchain_ollama.embeddings import OllamaEmbeddings
 from langchain_core.documents import Document
 langchain_available = True
except ImportError:
 langchain_available = False

Setup Logger
```

```

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -\n
↳%(message)s')
logger = logging.getLogger(__name__)

--- HELPER FUNCTIONS ---
def print_header(text: str, char: str = "="):
 """Prints a centered header to the console."""
 print("\n" + char * 80)
 print(text.center(80))
 print(char * 80)

def count_total_chunks(node: Dict) -> int:
 """Recursively counts all chunks in a node and its children."""
 total = node.get('_chunks', 0)
 for child_node in node.get('_children', {}).values():
 total += count_total_chunks(child_node)
 return total

def print_hierarchy_report(node: Dict, indent_level: int = 0):
 """
 Recursively prints the reconstructed hierarchy, sorting by sequential ToC
 ↳ID.
 """
 sorted_children = sorted(
 node.get('_children', {}).items(),
 key=lambda item: item[1].get('_toc_id', float('inf'))
)

 for title, child_node in sorted_children:
 prefix = " " * indent_level + "|-- "
 total_chunks_in_branch = count_total_chunks(child_node)
 direct_chunks = child_node.get('_chunks', 0)
 toc_id = child_node.get('_toc_id', 'N/A')
 print(f"{prefix}{title} [ID: {toc_id}] (Total Chuck in branch:\n
↳{total_chunks_in_branch}, Direct Chunk: {direct_chunks})")
 print_hierarchy_report(child_node, indent_level + 1)

def find_testable_sections(node: Dict, path: str, testable_list: List):
 """
 Recursively find sections with a decent number of "direct" chunks to test
 ↳sequence on.
 """
 if node.get('_chunks', 0) > 10 and not node.get('_children'):
 testable_list.append({
 "path": path,
 "toc_id": node.get('_toc_id'),
 "chunk_count": node.get('_chunks')

```

```

 })

 for title, child_node in node.get('_children', {}).items():
 new_path = f"{path} -> {title}" if path else title
 find_testable_sections(child_node, new_path, testable_list)

--- MODIFIED TEST FUNCTION ---
def verify_chunk_sequence_and_content(vector_store: Chroma, hierarchy_tree: Dict):
 """
 Selects a random ToC section, verifies chunk sequence, and displays the
 reassembled content.
 """
 print_header("Chunk Sequence & Content Integrity Test", char="-")
 logger.info("Verifying chunk order and reassembling content for a random
 ToC section.")

 # 1. Find a good section to test
 testable_sections = []
 find_testable_sections(hierarchy_tree, "", testable_sections)

 if not testable_sections:
 logger.warning("Could not find a suitable section with enough chunks to
 test. Skipping content test.")
 return

 random_section = random.choice(testable_sections)
 test_toc_id = random_section['toc_id']
 section_title = random_section['path'].split(' -> ')[-1]

 logger.info(f"Selected random section for testing:
 {random_section['path']} (toc_id: {test_toc_id})")

 # 2. Retrieve all documents (content + metadata) for that toc_id
 try:
 # Use .get() to retrieve full documents, not just similarity search
 retrieved_data = vector_store.get(
 where={"toc_id": test_toc_id},
 include=["metadatas", "documents"]
)

 # Combine metadatas and documents into LangChain Document objects
 docs = [Document(page_content=doc, metadata=meta) for doc, meta in
 zip(retrieved_data['documents'], retrieved_data['metadatas'])]

```

```

 logger.info(f"Retrieved {len(docs)} document chunks for toc_id_
↳ {test_toc_id}.")

 if len(docs) < 1:
 logger.warning("No chunks found in the selected section. Skipping.")
 return

 # 3. Sort the documents by chunk_id
 # Handle cases where chunk_id might be missing for robustness
 docs.sort(key=lambda d: d.metadata.get('chunk_id', -1))

 chunk_ids = [d.metadata.get('chunk_id') for d in docs]
 if None in chunk_ids:
 logger.error("TEST FAILED: Some retrieved chunks are missing a_
↳ 'chunk_id'.")
 return

 # 4. Verify sequence
 is_sequential = all(chunk_ids[i] == chunk_ids[i-1] + 1 for i in_
↳ range(1, len(chunk_ids)))

 # 5. Reassemble and print content
 full_content = "\n".join([d.page_content for d in docs])

 print("\n" + "-"*25 + " CONTENT PREVIEW " + "-"*25)
 print(f"Title: {section_title} [toc_id: {test_toc_id}]")
 print(f"Chunk IDs: {chunk_ids}")
 print("-" * 70)
 print(full_content)
 print("-" * 23 + " END CONTENT PREVIEW " + "-"*23 + "\n")

 if is_sequential:
 logger.info(" TEST PASSED: Chunk IDs for the section are_
↳ sequential and content is reassembled.")
 else:
 logger.warning("TEST PASSED (with note): Chunk IDs are not_
↳ perfectly sequential but are in increasing order.")
 logger.warning("This is acceptable. Sorting by chunk_id_
↳ successfully restored narrative order.")

 except Exception as e:
 logger.error(f"TEST FAILED: An error occurred during chunk sequence_
↳ verification: {e}", exc_info=True)

--- MAIN DIAGNOSTIC FUNCTION ---

```

```

def run_full_diagnostics():
 if not langchain_available:
 logger.error("LangChain components not installed. Skipping diagnostics.
↪")
 return
 if not pandas_available:
 logger.warning("Pandas not installed. Some reports may not be available.
↪")

 print_header("Full Database Health & Hierarchy Diagnostic Report")

 # 1. Connect to the Database
 logger.info("Connecting to the vector database...")
 if not os.path.exists(CHROMA_PERSIST_DIR):
 logger.error(f"FATAL: Chroma DB directory not found at_
↪{CHROMA_PERSIST_DIR}.")
 return

 vector_store = Chroma(
 persist_directory=CHROMA_PERSIST_DIR,
 embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
 collection_name=CHROMA_COLLECTION_NAME
)
 logger.info("Successfully connected to the database.")

 # 2. Retrieve ALL Metadata
 total_docs = vector_store._collection.count()
 if total_docs == 0:
 logger.warning("Database is empty. No diagnostics to run.")
 return

 logger.info(f"Retrieving metadata for all {total_docs} chunks...")
 metadatas = vector_store.get(limit=total_docs,
↪include=["metadatas"])['metadatas']
 logger.info("Successfully retrieved all metadata.")

 # 3. Reconstruct the Hierarchy Tree
 logger.info("Reconstructing hierarchy from chunk metadata...")
 hierarchy_tree = { '_children': {} }
 chunks_without_id = 0

 for meta in metadatas:
 toc_id = meta.get('toc_id')
 if toc_id is None or toc_id == -1:
 chunks_without_id += 1
 node_title = meta.get('level_1_title', 'Orphaned Chunks')
 if node_title not in hierarchy_tree['_children']:

```

```

 hierarchy_tree['_children'][node_title] = {'_children': {},
↳ '_chunks': 0, '_toc_id': float('inf')}
 hierarchy_tree['_children'][node_title]['_chunks'] += 1
 continue

 current_node = hierarchy_tree
 for level in range(1, 7):
 level_key = f'level_{level}_title'
 title = meta.get(level_key)
 if not title: break
 if title not in current_node['_children']:
 current_node['_children'][title] = {'_children': {}, '_chunks':
↳ 0, '_toc_id': float('inf')}
 current_node = current_node['_children'][title]

 current_node['_chunks'] += 1
 current_node['_toc_id'] = min(current_node['_toc_id'], toc_id)

 logger.info("Hierarchy reconstruction complete.")

4. Print Hierarchy Report
 print_header("Reconstructed Hierarchy Report (Book Order)", char="-")
 print_hierarchy_report(hierarchy_tree)

5. Run Chunk Sequence and Content Test
 verify_chunk_sequence_and_content(vector_store, hierarchy_tree)

6. Final Summary
 print_header("Diagnostic Summary", char="-")
 print(f"Total Chunks in DB: {total_docs}")

 if chunks_without_id > 0:
 logger.warning(f"Found {chunks_without_id} chunks MISSING a valid
↳ '_toc_id'. Check 'Orphaned' sections.")
 else:
 logger.info("All chunks contain valid 'toc_id' metadata. Sequential
↳ integrity is maintained.")

 print_header("Diagnostic Complete")

--- Execute Diagnostics ---
 if 'CHROMA_PERSIST_DIR' in locals() and langchain_available:
 run_full_diagnostics()
 else:
 logger.error("Skipping diagnostics: Global variables not defined or
↳ LangChain not available.")

```

```
2025-07-18 23:25:43,136 - INFO - Connecting to the vector database...
2025-07-18 23:25:43,141 - ERROR - FATAL: Chroma DB directory not found at
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/0.
DataBase_Chroma/ICT312/ICT312_chroma_db_toc_chunks_epub.
```

```
=====
Full Database Health & Hierarchy Diagnostic Report
=====
```

```
[21]: # Cell 6: Verify Content Retrieval for a Specific toc_id with Reassembled Text

import os
import json
import logging
from langchain_chroma import Chroma
from langchain_ollama.embeddings import OllamaEmbeddings

--- Logger Setup ---
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
↳%(message)s')

def retrieve_and_print_chunks_for_toc_id(vector_store: Chroma, toc_id: int):
 """
 Retrieves all chunks for a specific toc_id, reconstructs the section title
 from hierarchical metadata, shows the reassembled text, and lists individual
 chunk details for verification.
 """
 try:
 # Use the 'get' method with a 'where' filter to find all chunks for the
↳toc_id
 results = vector_store.get(
 where={"toc_id": toc_id},
 include=["documents", "metadatas"]
)

 if not results or not results.get('ids'):
 logger.warning(f"No chunks found in the database for toc_id =
↳{toc_id}")
 print("=" * 80)
 print(f"VERIFICATION FAILED: No content found for toc_id: {toc_id}")
 print("=" * 80)
 return

 documents = results['documents']
 metadatas = results['metadatas']
```

```

--- FIX START: Reconstruct the hierarchical section title from
↳ metadata ---
We assume all chunks for the same toc_id share the same titles.
We will inspect the metadata of the first chunk to get the title.
section_title = "Unknown or Uncategorized Section"
if metadatas:
 first_meta = metadatas[0]

 # Find all 'level_X_title' keys in the metadata
 level_titles = []
 for key, value in first_meta.items():
 if key.startswith("level_") and key.endswith("_title"):
 try:
 # Extract the level number (e.g., 1 from
↳ 'level_1_title') for sorting
 level_num = int(key.split('_')[1])
 level_titles.append((level_num, value))
 except (ValueError, IndexError):
 # Ignore malformed keys, just in case
 continue

 # Sort the titles by their level number (1, 2, 3...)
 level_titles.sort()

 # Join the sorted titles to create a breadcrumb-style title
 if level_titles:
 title_parts = [title for num, title in level_titles]
 section_title = " > ".join(title_parts)

--- FIX END ---

--- Print a clear header with the reconstructed section title ---
print("=" * 80)
print(f"VERIFYING SECTION: '{section_title}' (toc_id: {toc_id})")
print("=" * 80)
logger.info(f"Found {len(documents)} chunks in the database for this
↳ section.")

Sort chunks by their chunk_id to ensure they are in the correct order
↳ for reassembly
sorted_items = sorted(zip(documents, metadatas), key=lambda item:
↳ item[1].get('chunk_id', 0))

--- Reassemble and print the full text for the section ---
all_chunk_texts = [item[0] for item in sorted_items]
reassembled_text = "\n".join(all_chunk_texts)

print("\n" + "#" * 28 + " Reassembled Text " + "#" * 28)

```

```

print(reassembled_text)
print("#" * 80)

--- Print individual chunk details for in-depth verification ---
print("\n" + "-" * 24 + " Retrieved Chunk Details " + "-" * 25)
for i, (doc, meta) in enumerate(sorted_items):
 print(f"\n[Chunk {i+1} of {len(documents)} | chunk_id: {meta.
↪get('chunk_id', 'N/A')}]")
 content_preview = doc.replace('\n', ' ').strip()
 print(f" Content Preview: '{content_preview[:250]}...'")
 print(f" Metadata: {json.dumps(meta, indent=2)}")

print("\n" + "=" * 80)
print(f"Verification complete for section '{section_title}'.")
print("=" * 80)

except Exception as e:
 logger.error(f"An error occurred during retrieval for toc_id {toc_id}:␣
↪{e}", exc_info=True)

=====
EXECUTION BLOCK (No changes needed here)
=====

--- IMPORTANT: Set the ID of the section you want to test here ---
Example: ToC ID 10 might be "An Overview of Digital Forensics"
Example: ToC ID 11 might be "Digital Forensics and Other Related Disciplines"
TOC_ID_TO_TEST = 9# Change this to an ID you know exists from your ToC

Assume these variables are defined in a previous cell from your notebook
CHROMA_PERSIST_DIR = "./chroma_db_with_metadata"
EMBEDDING_MODEL_OLLAMA = "nomic-embed-text"
CHROMA_COLLECTION_NAME = "forensics_handbook"

Check if the database directory exists before attempting to connect
if 'CHROMA_PERSIST_DIR' in locals() and os.path.exists(CHROMA_PERSIST_DIR):
 logger.info(f"Connecting to the existing vector database at␣
↪'{CHROMA_PERSIST_DIR}'...")

try:
 vector_store = Chroma(
 persist_directory=CHROMA_PERSIST_DIR,
 embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
 collection_name=CHROMA_COLLECTION_NAME
)

```

```

 # Run the verification function
 retrieve_and_print_chunks_for_toc_id(vector_store, TOC_ID_TO_TEST)

 except Exception as e:
 logger.error(f"Failed to initialize Chroma or run retrieval. Error:␣
↪{e}")
 logger.error("Please ensure your embedding model and collection names␣
↪are correct.")

else:
 logger.error("Database directory not found or 'CHROMA_PERSIST_DIR' variable␣
↪is not set.")
 logger.error("Please run the previous cell (Cell 5) to create the database␣
↪first.")

```

```

2025-07-18 23:25:43,150 - ERROR - Database directory not found or
'CHROMA_PERSIST_DIR' variable is not set.
2025-07-18 23:25:43,151 - ERROR - Please run the previous cell (Cell 5) to
create the database first.
2025-07-18 23:25:43,151 - ERROR - Please run the previous cell (Cell 5) to
create the database first.

```

## 6.2 Test Data Base for content development

Require Description

[22]: *# Cell 7: Verify Vector Database (Final Version with Rich Diagnostic Output)*

```

import os
import json
import re
import random
import logging
from typing import List, Dict, Any, Tuple, Optional

Third-party imports
try:
 from langchain_chroma import Chroma
 from langchain_ollama.embeddings import OllamaEmbeddings
 from langchain_core.documents import Document
 langchain_available = True
except ImportError:
 langchain_available = False

Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
↪%(message)s')
logger = logging.getLogger(__name__)

```

```

--- HELPER FUNCTIONS ---

def print_results(query_text: str, results: list, where_filter: Optional[Dict]
 ↳= None):
 """
 Richly prints query results, showing the query, filter, and retrieved
 ↳documents.
 """
 print("\n" + "-"*10 + " DIAGNOSTIC: RETRIEVAL RESULTS " + "-"*10)
 print(f"QUERY: '{query_text}'")
 if where_filter:
 print(f"FILTER: {json.dumps(where_filter, indent=2)}")

 if not results:
 print("--> No documents were retrieved for this query and filter.")
 print("-" * 55)
 return

 print(f"--> Found {len(results)} results. Displaying top {min(len(results),
 ↳3)}:")
 for i, doc in enumerate(results[:3]):
 print(f"\n[RESULT {i+1}]")
 content_preview = doc.page_content.replace('\n', ' ').strip()
 print(f" Content : '{content_preview[:200]}...")
 print(f" Metadata: {json.dumps(doc.metadata, indent=2)}")
 print("-" * 55)

--- HELPER FUNCTIONS FOR FINDING DATA (UNCHANGED) ---
def find_deep_entry(nodes: List[Dict], current_path: List[str] = []) ->
 ↳Optional[Tuple[Dict, List[str]]]:
 shuffled_nodes = random.sample(nodes, len(nodes))
 for node in shuffled_nodes:
 if node.get('level', 0) >= 2 and node.get('children'): return node,
 ↳current_path + [node['title']]
 if node.get('children'):
 path = current_path + [node['title']]
 deep_entry = find_deep_entry(node['children'], path)
 if deep_entry: return deep_entry
 return None

def find_chapter_title_by_number(toc_data: List[Dict], chap_num: int) ->
 ↳Optional[List[str]]:
 def search_nodes(nodes, num, current_path):

```

```

 for node in nodes:
 path = current_path + [node['title']]
 if re.match(rf"(Chapter\s)?{num}[.:\s]", node.get('title', ''), re.
↳IGNORECASE): return path
 if node.get('children'):
 found_path = search_nodes(node['children'], num, path)
 if found_path: return found_path
 return None
 return search_nodes(toc_data, chap_num, [])

--- ENHANCED TEST CASES with DIAGNOSTIC OUTPUT ---

def basic_retrieval_test(db, outline):
 print_header("Test 1: Basic Retrieval", char="-")
 try:
 logger.info("Goal: Confirm the database is live and contains_
↳thematically relevant content.")
 logger.info("Strategy: Perform a simple similarity search using the_
↳course's 'unitName'.")
 query_text = outline.get("unitInformation", {}).get("unitName",_
↳"introduction")

 logger.info(f"Action: Searching for query: '{query_text}'...")
 results = db.similarity_search(query_text, k=1)

 print_results(query_text, results) # <--- SHOW THE EVIDENCE

 logger.info("Verification: Check if at least one document was returned.
↳")
 assert len(results) > 0, "Basic retrieval query returned no results."

 logger.info(" Result: TEST 1 PASSED. The database is online and_
↳responsive.")
 return True
 except Exception as e:
 logger.error(f" Result: TEST 1 FAILED. Reason: {e}")
 return False

def deep_hierarchy_test(db, toc):
 print_header("Test 2: Deep Hierarchy Retrieval", char="-")
 try:
 logger.info("Goal: Verify that the multi-level hierarchical metadata_
↳was ingested correctly.")
 logger.info("Strategy: Find a random, deeply nested sub-section and use_
↳a precise filter to retrieve it.")

```

```

 deep_entry_result = find_deep_entry(toc)
 assert deep_entry_result, "Could not find a suitable deep entry (level_
↳>= 2) to test."
 node, path = deep_entry_result
 query = node['title']

 logger.info(f" - Selected random deep section: {' -> '.join(path)}")
 conditions = [{f"level_{i+1}_title": {"$eq": title}} for i, title in
↳enumerate(path)]
 w_filter = {"$and": conditions}

 logger.info("Action: Performing a similarity search with a highly_
↳specific '$and' filter.")
 results = db.similarity_search(query, k=1, filter=w_filter)

 print_results(query, results, w_filter) # <--- SHOW THE EVIDENCE

 logger.info("Verification: Check if the precisely filtered query_
↳returned any documents.")
 assert len(results) > 0, "Deeply filtered query returned no results."

 logger.info(" Result: TEST 2 PASSED. Hierarchical metadata is_
↳structured correctly.")
 return True
 except Exception as e:
 logger.error(f" Result: TEST 2 FAILED. Reason: {e}")
 return False

def advanced_alignment_test(db, outline, toc):
 print_header("Test 3: Advanced Unit Outline Alignment", char="-")
 try:
 logger.info("Goal: Ensure a weekly topic from the syllabus can be_
↳mapped to the correct textbook chapter(s).")
 logger.info("Strategy: Pick a random week, find its chapter, and query_
↳for the topic filtered by that chapter.")
 week_to_test = random.choice(outline['weeklySchedule'])
 logger.info(f" - Selected random week: Week {week_to_test['week']} -_
↳'{week_to_test['contentTopic']}'")

 reading = week_to_test.get('requiredReading', '')
 chap_nums_str = re.findall(r'\d+', reading)
 assert chap_nums_str, f"Could not find chapter numbers in required_
↳reading: '{reading}'"
 logger.info(f" - Extracted required chapter number(s):_
↳{chap_nums_str}")

```

```

 chapter_paths = [find_chapter_title_by_number(toc, int(n)) for n in
↪chap_nums_str]
 chapter_paths = [path for path in chapter_paths if path is not None]
 assert chapter_paths, f"Could not map chapter numbers {chap_nums_str}
↪to a valid ToC path."

 level_1_titles = list(set([path[0] for path in chapter_paths]))
 logger.info(f" - Mapped to top-level ToC entries: {level_1_titles}")

 or_filter = [{"level_1_title": {"$eq": title}} for title in
↪level_1_titles]
 w_filter = {"$or": or_filter} if len(or_filter) > 1 else or_filter[0]
 query = week_to_test['contentTopic']

 logger.info("Action: Searching for the weekly topic, filtered by the
↪mapped chapter(s).")
 results = db.similarity_search(query, k=5, filter=w_filter)

 print_results(query, results, w_filter) # <--- SHOW THE EVIDENCE

 logger.info("Verification: Check if at least one returned document is
↪from the correct chapter.")
 assert len(results) > 0, "Alignment query returned no results for the
↪correct section/chapter."

 logger.info(" Result: TEST 3 PASSED. The syllabus can be reliably
↪aligned with the textbook content.")
 return True
 except Exception as e:
 logger.error(f" Result: TEST 3 FAILED. Reason: {e}")
 return False

def content_sequence_test(db, outline):
 print_header("Test 4: Content Sequence Verification", char="-")
 try:
 logger.info("Goal: Confirm that chunks for a topic can be re-ordered to
↪form a coherent narrative.")
 logger.info("Strategy: Retrieve several chunks for a random topic and
↪verify their 'chunk_id' is sequential.")
 topic_query = random.choice(outline['weeklySchedule'])['contentTopic']

 logger.info(f"Action: Performing similarity search for topic:
↪'{topic_query}' to get a set of chunks.")
 results = db.similarity_search(topic_query, k=10)

 print_results(topic_query, results) # <--- SHOW THE EVIDENCE

```

```

docs_with_id = [doc for doc in results if 'chunk_id' in doc.metadata]
assert len(docs_with_id) > 3, "Fewer than 4 retrieved chunks have a
↳ 'chunk_id' to test."

chunk_ids = [doc.metadata['chunk_id'] for doc in docs_with_id]
sorted_ids = sorted(chunk_ids)

logger.info(f" - Retrieved and sorted chunk IDs: {sorted_ids}")
logger.info("Verification: Check if the sorted list of chunk_ids is
↳ strictly increasing.")
is_ordered = all(sorted_ids[i] >= sorted_ids[i-1] for i in range(1,
↳ len(sorted_ids)))
assert is_ordered, "The retrieved chunks' chunk_ids are not in
↳ ascending order when sorted."

logger.info(" Result: TEST 4 PASSED. Narrative order can be
↳ reconstructed using 'chunk_id'.")
return True
except Exception as e:
 logger.error(f" Result: TEST 4 FAILED. Reason: {e}")
 return False

--- MAIN VERIFICATION EXECUTION ---
def run_verification():
 print_header("Database Verification Process")

 if not langchain_available:
 logger.error("LangChain libraries not found. Aborting tests.")
 return

 required_files = {
 "Chroma DB": CHROMA_PERSIST_DIR,
 "ToC JSON": PRE_EXTRACTED_TOC_JSON_PATH,
 "Parsed Outline": PARSED_UO_JSON_PATH
 }
 for name, path in required_files.items():
 if not os.path.exists(path):
 logger.error(f"Required '{name}' not found at '{path}'. Please run
↳ previous cells.")
 return

 with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
 toc_data = json.load(f)
 with open(PARSED_UO_JSON_PATH, 'r', encoding='utf-8') as f:
 unit_outline_data = json.load(f)

```

```

logger.info("Connecting to DB and initializing components...")
embeddings = OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA)
vector_store = Chroma(
 persist_directory=CHROMA_PERSIST_DIR,
 embedding_function=embeddings,
 collection_name=CHROMA_COLLECTION_NAME
)

results_summary = [
 basic_retrieval_test(vector_store, unit_outline_data),
 deep_hierarchy_test(vector_store, toc_data),
 advanced_alignment_test(vector_store, unit_outline_data, toc_data),
 content_sequence_test(vector_store, unit_outline_data)
]

passed_count = sum(filter(None, results_summary))
failed_count = len(results_summary) - passed_count

print_header("Verification Summary")
print(f"Total Tests Run: {len(results_summary)}")
print(f"Passed: {passed_count}")
print(f"Failed: {failed_count}")
print_header("Verification Complete", char="=")

--- Execute Verification ---
Assumes global variables from Cell 1 are available in the notebook's scope
run_verification()

```

2025-07-18 23:25:43,166 - ERROR - Required 'Chroma DB' not found at  
 '/home/sebas\_dev\_linux/projects/course\_generator/data\_output/test\_Output/0.  
 DataBase\_Chroma/ICT312/ICT312\_chroma\_db\_toc\_chunks\_epub'. Please run previous  
 cells.

```

=====
 Database Verification Process
=====

```

## 7 Content Generation

### 7.1 Planning Agent

```

[23]: # PlanningAgent
 # Cell 8: The Data-Driven Planning Agent (Final Hierarchical Version)

import os
import json

```

```

import re
import math
import logging
from typing import List, Dict, Any, Optional, Tuple

Setup Logger and LangChain components
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
try:
 from langchain_chroma import Chroma
 from langchain_ollama.embeddings import OllamaEmbeddings
 langchain_available = True
except ImportError:
 langchain_available = False

def print_header(text: str, char: str = "="):
 """Prints a centered header to the console."""
 print("\n" + char * 80)
 print(text.center(80))
 print(char * 80)

class PlanningAgent:
 """
 An agent that creates a hierarchical content plan, adaptively partitions
 content
 into distinct lecture decks, and allocates presentation time.
 """
 def __init__(self, master_config: Dict, vector_store: Optional[Any] = None):
 self.config = master_config['processed_settings']
 self.unit_outline = master_config['unit_outline']
 self.book_toc = master_config['book_toc']
 self.flat_toc_with_ids = self._create_flat_toc_with_ids()
 self.vector_store = vector_store
 logger.info("Data-Driven PlanningAgent initialized successfully.")

 def _create_flat_toc_with_ids(self) -> List[Dict]:
 """Creates a flattened list of the ToC for easy metadata lookup."""
 flat_list = []
 def flatten_recursive(nodes, counter):
 for node in nodes:
 node_id = counter[0]; counter[0] += 1
 flat_list.append({'toc_id': node_id, 'title': node.get('title', ''),
 'node': node})
 if node.get('children'):
 flatten_recursive(node.get('children'), counter)
 flatten_recursive(self.book_toc, [0])

```

```

 return flat_list

def _assign_sequence_ids_recursively(self, node: Dict, counter: list):
 """
 Recursively walks a content node, assigning a unique, sequential ID
 to the node, its children, and its interactive activity in a
 ↪depth-first order.
 """
 # 1. Assign sequence ID to the parent content node itself.
 node['seq_id'] = counter[0]
 counter[0] += 1

 # 2. Recurse through all children first.
 if 'children' in node and node['children']:
 for child in node['children']:
 self._assign_sequence_ids_recursively(child, counter)

 # 3. Assign an ID to the interactive activity LAST, so it appears after
 ↪the content and its children.
 if 'interactive_activity' in node:
 node['interactive_activity']['seq_id'] = counter[0]
 counter[0] += 1

def _identify_relevant_chapters(self, weekly_schedule_item: Dict) ->
↪List[int]:
 """Extracts chapter numbers precisely from the 'requiredReading' string.
 ↪"""
 reading_str = weekly_schedule_item.get('requiredReading', '')
 match = re.search(r'Chapter(s)?', reading_str, re.IGNORECASE)
 if not match: return []
 search_area = reading_str[match.start():]
 chap_nums_str = re.findall(r'\d+', search_area)
 if chap_nums_str:
 return sorted(list(set(int(n) for n in chap_nums_str)))
 return []

def _find_chapter_node(self, chapter_number: int) -> Optional[Dict]:
 """Finds the ToC node for a specific chapter number."""
 for item in self.flat_toc_with_ids:
 if re.match(rf"Chapter\s{chapter_number}(?:\D|$)", item['title']):
 return item['node']
 return None

def _build_topic_plan_tree(self, toc_node: Dict) -> Dict:
 """
 Recursively builds a hierarchical plan tree from any ToC node,
 annotating it with direct and total branch chunk counts.

```

```

 """
 node_metadata = next((item for item in self.flat_toc_with_ids if
↪item['node'] is toc_node), None)
 if not node_metadata: return {}

 retrieved_docs = self.vector_store.get(where={'toc_id':
↪node_metadata['toc_id']})
 direct_chunk_count = len(retrieved_docs.get('ids', []))

 plan_node = {
 "title": node_metadata['title'],
 "toc_id": node_metadata['toc_id'],
 "chunk_count": direct_chunk_count,
 "total_chunks_in_branch": 0,
 "slides_allocated": 0,
 "children": []
 }

 child_branch_total = 0
 for child_node in toc_node.get('children', []):
 if any(ex in child_node.get('title', '').lower() for ex in
↪["review", "introduction", "summary", "key terms"]):
 continue
 child_plan_node = self._build_topic_plan_tree(child_node)
 if child_plan_node:
 plan_node['children'].append(child_plan_node)
 child_branch_total += child_plan_node.
↪get('total_chunks_in_branch', 0)

 plan_node['total_chunks_in_branch'] = direct_chunk_count +
↪child_branch_total
 return plan_node

 # In PlanningAgent Class...

 def _allocate_slides_to_tree(self, plan_tree: Dict, content_slides_budget:
↪int):
 """
 (FINAL, REORDERED FOR CLARITY) Performs a multi-pass process to
↪allocate content slides,
 add activities, sum totals, and reorders the keys in each node for
↪maximum readability.
 """
 if not plan_tree or content_slides_budget <= 0:
 return plan_tree

```

```

--- Pass 1: Allocate Content Slides ---
def allocate_content_recursively(node, budget):
 node['budget_slides_content'] = round(budget)
 node['direct_slides_content'] = 0
 if not node.get('children'):
 node['direct_slides_content'] = round(budget)
 return
 total_branch_chunks = node.get('total_chunks_in_branch', 0)
 own_content_slides = 0
 if total_branch_chunks > 0:
 own_content_slides = round(budget * (node.get('chunk_count', 0) /
 ↪ total_branch_chunks))
 node['direct_slides_content'] = own_content_slides
 remaining_budget_for_children = budget - own_content_slides
 children_total_chunks = total_branch_chunks - node.
 ↪get('chunk_count', 0)
 if children_total_chunks <= 0: return
 for child in node.get('children', []):
 child_budget = remaining_budget_for_children * (child.
 ↪get('total_chunks_in_branch', 0) / children_total_chunks)
 allocate_content_recursively(child, child_budget)

 allocate_content_recursively(plan_tree, content_slides_budget)

--- Pass 2: Add Interactive Activities ---
def add_interactive_nodes(node, depth, interactive_deep):
 if not node: return
 if self.config.get('interactive', False):
 if interactive_deep:
 if depth == 2: node['interactive_activity'] = {"title":
 ↪f"{node.get('title')} (Deep-Dive Activity)", "toc_id": node.get('toc_id'),
 ↪f"slides_allocated": 1}
 if depth == 1: node['interactive_activity'] = {"title":
 ↪f"{node.get('title')} (General Activity)", "toc_id": node.get('toc_id'),
 ↪f"slides_allocated": 1}
 else:
 if depth == 1: node['interactive_activity'] = {"title":
 ↪f"{node.get('title')} (Interactive Activity)", "toc_id": node.get('toc_id'),
 ↪f"slides_allocated": 1}
 for child in node.get('children', []):
 add_interactive_nodes(child, depth + 1, interactive_deep)

 add_interactive_nodes(plan_tree, 1, self.config.get('interactive_deep',
 ↪False))

--- Pass 3: Sum All Slides Up the Tree ---

```

```

def sum_slides_upwards(node):
 total_slides = node.get('direct_slides_content', 0)
 total_slides += node.get('interactive_activity', {}).
↳get('slides_allocated', 0)
 if node.get('children'):
 total_slides += sum(sum_slides_upwards(child) for child in node.
↳get('children', []))
 node['total_slides_in_branch'] = total_slides
 return total_slides

sum_slides_upwards(plan_tree)

--- NEW: Pass 4: Reorder keys for final clarity ---
def reorder_keys_for_readability(node: Dict) -> Dict:
 if not node:
 return None

 # Define the desired order of keys
 key_order = [
 "title",
 "toc_id",
 "chunk_count",
 "total_chunks_in_branch",
 "budget_slides_content",
 "direct_slides_content",
 "total_slides_in_branch",
 "children",
 "interactive_activity"
]

 # Rebuild the dictionary in the specified order
 reordered_node = {key: node[key] for key in key_order if key in
↳node}

 # Recursively reorder children
 if 'children' in reordered_node:
 reordered_node['children'] =
↳[reorder_keys_for_readability(child) for child in reordered_node['children']]

 return reordered_node

return reorder_keys_for_readability(plan_tree)

def create_content_plan_for_week(self, week_number: int) -> Optional[Dict]:
 """Orchestrates the adaptive planning and partitioning process."""
 print_header(f"Planning Week {week_number}", char="*")

```

```

weekly_schedule_item = self.unit_outline['weeklySchedule'][week_number_
↪- 1]

chapter_numbers = self._identify_relevant_chapters(weekly_schedule_item)
if not chapter_numbers: return None

num_decks = self.config['week_session_setup'].get('sessions_per_week',
↪1)

 # 1. Build a full plan tree for each chapter to get its weight.
 chapter_plan_trees = [self._build_topic_plan_tree(self.
↪_find_chapter_node(cn)) for cn in chapter_numbers if self.
↪_find_chapter_node(cn)]
 total_weekly_chunks = sum(tree.get('total_chunks_in_branch', 0) for
↪tree in chapter_plan_trees)

 # 2. NEW: Adaptive Partitioning Strategy
 partitionable_units = []
 all_top_level_sections = []
 for chapter_tree in chapter_plan_trees:
 all_top_level_sections.extend(chapter_tree.get('children', []))

 num_top_level_sections = len(all_top_level_sections)

 # Always prefer to split by top-level sections if there are enough to
↪distribute.
 if num_top_level_sections >= num_decks:
 logger.info(f"Partitioning strategy: Distributing
↪{num_top_level_sections} top-level sections across {num_decks} decks.")
 partitionable_units = all_top_level_sections
 else:
 # Fallback for rare cases where there are fewer topics than decks
↪(e.g., 1 chapter with 1 section, but 2 decks).
 logger.info(f"Partitioning strategy: Not enough top-level sections
↪({num_top_level_sections}) to fill all decks ({num_decks}). Distributing
↪whole chapters instead.")
 partitionable_units = chapter_plan_trees

 # 3. Partition the chosen units into decks using a bin-packing algorithm
 decks = [[] for _ in range(num_decks)]
 deck_weights = [0] * num_decks
 sorted_units = sorted(partitionable_units, key=lambda x: x.
↪get('toc_id', 0))

 for unit in sorted_units:
 lightest_deck_index = deck_weights.index(min(deck_weights))

```

```

 decks[lightest_deck_index].append(unit)
 deck_weights[lightest_deck_index] += unit.
↪get('total_chunks_in_branch', 0)

 # 4. Plan each deck
 content_slides_per_week = self.config['slide_count_strategy'].
↪get('target_total_slides', 25)
 final_deck_plans = []
 for i, deck_content_trees in enumerate(decks):
 deck_number = i + 1
 deck_chunk_weight = sum(tree.get('total_chunks_in_branch', 0) for
↪tree in deck_content_trees)
 deck_slide_budget = round((deck_chunk_weight / total_weekly_chunks)
↪* content_slides_per_week) if total_weekly_chunks > 0 else 0

 logger.info(f"--- Planning Deck {deck_number}/{num_decks} | Topics:
↪{[t['title'] for t in deck_content_trees]} | Weight: {deck_chunk_weight}
↪chunks | Slide Budget: {deck_slide_budget} ---")

 # The allocation function is recursive and works on any tree or
↪sub-tree
 planned_content = [self._allocate_slides_to_tree(tree,
↪round(deck_slide_budget * (tree.get('total_chunks_in_branch', 0) /
↪deck_chunk_weight))) if deck_chunk_weight > 0 else tree for tree in
↪deck_content_trees]

 final_deck_plans.append({
 "deck_number": deck_number,
 "deck_title": f"{self.config.get('unit_name', 'Course')} - Week
↪{week_number}, Lecture {deck_number}",
 "session_content": planned_content
 })

 return {
 "week": week_number,
 "overall_topic": weekly_schedule_item.get('contentTopic'),
 "deck_plans": final_deck_plans
 }

 def finalize_and_calculate_time_plan(self, draft_plan: Dict, config: Dict)
↪-> Dict:
 """
 Takes a draft plan and enriches it by:
 1. Calculating detailed slide counts and time allocations for every
↪node.

```

```

2. Adding framework sections and wrapping content.
3. Calculating and adding summaries for decks and the entire week.
4. Assigning a final sequence ID (seq_id) for slide generation.
5. Reordering all keys for maximum readability.
"""
final_plan = json.loads(json.dumps(draft_plan))

--- Time Constants from Config ---
params = config.get('parameters_slides', {})
TIME_PER_CONTENT = params.get('time_per_content_slides_min', 3)
TIME_PER_INTERACTIVE = params.get('time_per_interactive_slide_min', 5)
TIME_FOR_FRAMEWORK_DECK = params.get('time_for_framework_slides_min', 6)
FRAMEWORK_SLIDES_PER_DECK = 4

--- Recursive Helper Functions ---
def _calculate_time_and_reorder(node: Dict):
 # 1. Recurse to the bottom first to perform a bottom-up calculation
 children_total_time = 0
 if 'children' in node and node['children']:
 for child in node['children']:
 _calculate_time_and_reorder(child) # Recursive call
 children_total_time += child.get('time_allocation_minutes', 0)
 ↪{}.get('total_branch_time', 0)

 # 2. Calculate this node's direct time
 direct_content_time = node.get('direct_slides_content', 0) * ↪
 ↪TIME_PER_CONTENT
 interactive_time = node.get('interactive_activity', {}).
 ↪get('slides_allocated', 0) * TIME_PER_INTERACTIVE

 # 3. Calculate this node's total branch time
 branch_total_time = direct_content_time + interactive_time + ↪
 ↪children_total_time

 # 4. Create the time allocation object
 time_alloc = {
 "direct_content_time": direct_content_time,
 "direct_interactive_time": interactive_time,
 "total_branch_time": branch_total_time
 }
 node['time_allocation_minutes'] = time_alloc

 # 5. Reorder all keys for this node to ensure final clarity
 key_order = [
 "title",
 "toc_id",

```

```

 "chunk_count",
 "total_chunks_in_branch",
 "budget_slides_content",
 "direct_slides_content",
 "total_slides_in_branch",
 "time_allocation_minutes",
 "children",
 "interactive_activity"

]

 reordered_node = {key: node[key] for key in key_order if key in
↪node}

 # Clear the original node and update it with the reordered keys
 node.clear()
 node.update(reordered_node)

 # --- Main Processing Loop for Decks ---
 for deck in final_plan.get("deck_plans", []):
 session_content_blocks = deck.pop("session_content", [])

 # Perform the combined time calculation and reordering pass
 for block in session_content_blocks:
 _calculate_time_and_reorder(block)

 # Create Framework Sections
 week_number, deck_number = final_plan.get("week"), deck.
↪get("deck_number")

 title_section = {"section_type": "Title", "content": { "unit_name":
↪config.get('unit_name', 'Course'), "unit_code": config.get('course_id', ''),
↪"week_topic": final_plan.get('overall_topic', ''), "deck_title": f"Week
↪{week_number}, Lecture {deck_number}"}}

 agenda_section = {"section_type": "Agenda", "content": {"title":
↪"Today's Agenda", "items": [item.get('title', 'Untitled Topic') for item in
↪session_content_blocks]}}

 summary_section = {"section_type": "Summary", "content": {"title":
↪"Summary & Key Takeaways", "placeholder": "Auto-generate based on covered
↪topics."}}

 end_section = {"section_type": "End", "content": {"title": "Thank
↪You", "text": "Questions?"}}

 main_content_block = {"section_type": "Content", "content_blocks":
↪session_content_blocks}

 final_sections_for_deck = [title_section, agenda_section,
↪main_content_block, summary_section, end_section]

```

```

*** NEW: Assign Sequential IDs for the entire deck ***

seq_counter = [0] # Use a list for pass-by-reference behavior
for section in final_sections_for_deck:
 if section.get("section_type") == "Content":
 # For the main content, iterate through its blocks and
 ↪start the recursion
 for block in section.get("content_blocks", []):
 self._assign_sequence_ids_recursively(block,
 ↪seq_counter)
 else:
 # For simple framework slides, just assign the next ID
 section['seq_id'] = seq_counter[0]
 seq_counter[0] += 1

*** END NEW CODE ***

Calculate Deck Summaries
total_content_slides = sum(b.get('total_slides_in_branch', 0) - b.
↪get('interactive_activity', {}).get('slides_allocated', 0) for b in
↪session_content_blocks)
total_interactive_slides = sum(b.get('interactive_activity', {}).
↪get('slides_allocated', 0) for b in session_content_blocks)

deck_content_time = sum(b.get('time_allocation_minutes', {}).
↪get('total_branch_time', 0) for b in session_content_blocks)

deck['total_slides_in_deck'] = FRAMEWORK_SLIDES_PER_DECK + sum(b.
↪get('total_slides_in_branch', 0) for b in session_content_blocks)
deck['slide_count_breakdown'] = {"framework":
↪FRAMEWORK_SLIDES_PER_DECK, "content": total_content_slides, "interactive":
↪total_interactive_slides}
deck['time_breakdown_minutes'] = {"framework":
↪TIME_FOR_FRAMEWORK_DECK, "content_and_interactive": deck_content_time,
↪"total_deck_time": TIME_FOR_FRAMEWORK_DECK + deck_content_time}
deck['sections'] = final_sections_for_deck
if 'deck_title' in deck: del deck['deck_title']

--- Calculate Grand Totals for the Week ---

```

```

 weekly_slide_summary = {"total_slides_for_week": 0,
↪ "total_framework_slides": 0, "total_content_slides": 0,
↪ "total_interactive_slides": 0, "number_of_decks": len(final_plan.
↪ get("deck_plans", []))}

 weekly_time_summary = {"total_time_for_week_minutes": 0,
↪ "total_framework_time": 0, "total_content_and_interactive_time": 0}

 for deck in final_plan.get("deck_plans", []):
 weekly_slide_summary['total_slides_for_week'] += deck.
↪ get('total_slides_in_deck', 0)
 for key, value in deck.get('slide_count_breakdown', {}).items():
↪ weekly_slide_summary[f"total_{key}_slides"] += value
 weekly_time_summary['total_time_for_week_minutes'] += deck.
↪ get('time_breakdown_minutes', {}).get('total_deck_time', 0)
 weekly_time_summary['total_framework_time'] += deck.
↪ get('time_breakdown_minutes', {}).get('framework', 0)
 weekly_time_summary['total_content_and_interactive_time'] += deck.
↪ get('time_breakdown_minutes', {}).get('content_and_interactive', 0)

 # --- Construct Final Ordered Plan ---
 final_ordered_plan = {
 "week": final_plan.get("week"),
 "overall_topic": final_plan.get("overall_topic"),
 "weekly_slide_summary": weekly_slide_summary,
 "weekly_time_summary_minutes": weekly_time_summary,
 "deck_plans": final_plan.get("deck_plans", [])
 }

 return final_ordered_plan

 # --- NEW FUNCTION TO GENERATE MASTER SUMMARY ---
 def generate_and_save_master_plan(self, weekly_plans: List[Dict], config:
↪ Dict):
 """
 Aggregates summaries from all weekly plans into a single master plan,
↪ file,
 including new grand total metrics.
 """
 print_header("Phase 4: Generating Master Unit Plan", char="#")

 # Initialize the master plan structure with the new fields
 master_plan = {
 "unit_code": config.get('course_id', 'UNKNOWN'),
 "unit_name": config.get('unit_name', 'Unknown Unit'),
 "grand_total_summary": {
 "total_slides_for_unit": 0,

```

```

 "total_framework_slides": 0,
 "total_content_slides": 0,
 "total_interactive_slides": 0,
 "total_number_of_decks": 0,
 "total_time_for_unit_minutes": 0,
 "total_time_for_unit_in_hour": 0, # New
 "average_deck_time_in_min": 0,
 "average_deck_time_in_hour": 0
 },
 "weekly_summaries": []
}

grand_totals = master_plan["grand_total_summary"]

Loop through each weekly plan to aggregate data
for plan in sorted(weekly_plans, key=lambda p: p.get('week', 0)):
 # Extract the high-level summary for this week
 summary_entry = {
 "week": plan.get("week"),
 "overall_topic": plan.get("overall_topic"),
 "slide_summary": plan.get("weekly_slide_summary"),
 "time_summary_minutes": plan.get("weekly_time_summary_minutes")
 }
 master_plan["weekly_summaries"].append(summary_entry)

 # Add this week's totals to the grand totals
 slide_summary = plan.get("weekly_slide_summary", {})
 time_summary = plan.get("weekly_time_summary_minutes", {})

 grand_totals["total_slides_for_unit"] += slide_summary.
↳get("total_slides_for_unit", 0)
 grand_totals["total_framework_slides"] += slide_summary.
↳get("total_framework_slides", 0)
 grand_totals["total_content_slides"] += slide_summary.
↳get("total_content_slides", 0)
 grand_totals["total_interactive_slides"] += slide_summary.
↳get("total_interactive_slides", 0)
 grand_totals["total_number_of_decks"] += slide_summary.
↳get("number_of_decks", 0)
 grand_totals["total_time_for_unit_minutes"] += time_summary.
↳get("total_time_for_unit_minutes", 0)

 # --- NEW: Calculate the final derived grand totals after the loop ---
 if grand_totals["total_time_for_unit_minutes"] > 0:
 grand_totals["total_time_for_unit_in_hour"] =
↳round(grand_totals["total_time_for_unit_minutes"] / 60, 2)

```

```

 if grand_totals["total_number_of_decks"] > 0:
 grand_totals["average_deck_time_in_min"] =
↪round(grand_totals["total_time_for_unit_minutes"] /
↪grand_totals["total_number_of_decks"], 2)

 if grand_totals["total_number_of_decks"] > 0:
 grand_totals["average_deck_time_in_hour"] =
↪round((grand_totals["total_time_for_unit_minutes"] /
↪grand_totals["total_number_of_decks"])) / 60, 2)

 master_filename = f"{config.get('course_id', 'UNIT')}_master_plan_unit.
↪json"
 output_path = os.path.join(PLAN_OUTPUT_DIR, master_filename)

 try:
 with open(output_path, 'w') as f:
 json.dump(master_plan, f, indent=2)
 logger.info(f"Successfully generated and saved Master Unit Plan to:
↪{output_path}")
 print("\n--- Preview of Master Plan ---")
 print(json.dumps(master_plan, indent=2))
 return True
 except Exception as e:
 logger.error(f"Failed to save Master Unit Plan: {e}", exc_info=True)

```

## 7.2 Content Generator Agent

```

[24]: # CONTENT_SLIDE_GENERATION_PROMPT = ""
You are an expert university lecturer and instructional designer. Your task
↪is to create the content for a single, specific PowerPoint slide as part of
↪a larger topic.

CONTEXT:
- **Main Topic:** "{main_topic_title}"
- **Overall Raw Text for this Topic:** ``{topic_raw_content}``
- **Generation Task:** You are generating slide **{current_slide_num}** of a
↪total of **{total_slides_for_topic}** slides for this topic.
- **Content Covered on Previous Slides:** {generation_history}

INSTRUCTIONS:
1. Based on all the context, determine the most logical sub-topic for the
↪*next* slide ({current_slide_num}). Do NOT repeat content from previous
↪slides.

```

```

2. Create the content for this single slide. You can use one or two "Content
↳Objects" to structure the slide (e.g., use two objects for a comparison).
3. Your output MUST be a single, well-formed JSON object. Do NOT add any
↳text or explanations outside of the JSON object.

JSON OUTPUT STRUCTURE & RULES:
{{
"title": "string | The main title for the slide, consistent with the main
↳topic.",
"subtitle": "string | A more specific subtitle for this particular slide.
↳Use null if not needed.",
"objects": [
{{
"content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
"content_purpose": "string | CHOOSE ONE: 'description', 'explanation',
↳'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',
↳'case_study', 'example'.",
"data": "object | The structured data for the slide. See examples below.
↳"
}}
]
}}

DATA STRUCTURE EXAMPLES:
- For "bullet_points": "data": ["Point 1", {{ "text": "Point 2", "children":
↳["Sub-point 2.1"] }}]
- For "table": "data": {{ "headers": ["Col A"], "rows": [["Data 1"]] }}

Now, generate the JSON for slide {current_slide_num}.
""

CONTENT_SLIDE_GENERATION_PROMPT = ""
You are an expert university lecturer and instructional designer. Your task
↳is to process a large block of raw text and intelligently divide it into a
↳sequence of PowerPoint slides.

CONTEXT:
- **Main Topic:** "{main_topic_title}"
- **The ENTIRE Raw Text for this Topic:** ``{topic_raw_content}```
- **Total Slides to Create:** You must cover all the key concepts in the raw
↳text over a total of **{total_slides_for_topic}** slides.
- **Current Task:** You are generating slide **{current_slide_num}** of
↳**{total_slides_for_topic}**.
- **Topics Already Covered on Previous Slides:** {generation_history}

YOUR GOAL AND INSTRUCTIONS:

```

```

1. Analyze the ENTIRE raw text. Identify the distinct sub-topics within
↳ it.
2. Review the topics already covered in the `generation_history`.
3. Identify the NEXT logical sub-topic from the raw text that has NOT been
↳ covered yet. Your primary goal is to ensure comprehensive coverage without
↳ repetition.
4. Create the content for a single slide that is focused only on this
↳ new, uncovered sub-topic.
5. Your output MUST be a single, well-formed JSON object. Do not add any
↳ text or explanations outside the JSON object.

JSON OUTPUT STRUCTURE & RULES:
{{
"title": "string | The main title for the slide, consistent with the main
↳ topic.",
"subtitle": "string | A more specific subtitle for this particular slide.
↳ Use null if not needed.",
"objects": [
{{
"content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
"content_purpose": "string | CHOOSE ONE: 'description', 'explanation',
↳ 'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',
↳ 'case_study', 'example'.",
"data": "object | The structured data for the slide. See examples below.
↳ "
}}
]
}}

DATA STRUCTURE EXAMPLES:
- For "bullet_points": "data": ["Point 1", {"text": "Point 2", "children":
↳ ["Sub-point 2.1"]}]]
- For "table": "data": {"headers": ["Col A"], "rows": [["Data 1"]]]

Now, generate the JSON for slide {current_slide_num}.
""

INTERACTIVE_ACTIVITY_PROMPT = ""
You are an engaging university lecturer creating an interactive slide to test
↳ student understanding.

CONTEXT:

```

```

- Topic for this Activity: "{topic_title}"
- Raw Text Content for this Topic: ``{topic_raw_content}``

INSTRUCTIONS:
1. Create a single, insightful multiple-choice question that assesses a key
 ↪ concept from the provided raw text.
2. The question must have exactly 4 options. Provide the correct answer and
 ↪ a brief, clear explanation.
3. Your output MUST be a single, well-formed JSON object that adheres
 ↪ strictly to the structure below.

JSON OUTPUT STRUCTURE:
{{
"title": "Let's Apply This!",
"subtitle": "Knowledge Check: {topic_title}",
"objects": [
{{
"content_type": "multiple_choice_question",
"content_purpose": "knowledge_check",
"data": {{
"question_text": "string | The question to be asked.",
"options": [
{{"label": "A", "text": "string"}}},
{{"label": "B", "text": "string"}}},
{{"label": "C", "text": "string"}}},
{{"label": "D", "text": "string"}}}
],
"correct_answer": {{
"label": "string | e.g., 'B'",
"explanation": "string | A brief explanation of why this answer is
 ↪ correct."
}}
}}
]
}}

Now, generate the JSON for the interactive activity.
""

#
 ↪ =====
NEW, CONTEXT-AWARE SUMMARY GENERATION PROMPT
This prompt provides the LLM with the content of all slides for a rich
 ↪ summary.

```

```

#
↳ =====

SUMMARY_GENERATION_PROMPT = ""
You are an expert university lecturer crafting the final, conclusive summary
↳ slide for a lecture.

CONTEXT:
- **Overall Lecture Topic:** "{lecture_topic}"
- **Detailed Content of Slides Presented:**
``json
{slide_content_details}
INSTRUCTIONS:
Analyze the detailed content of all the slides provided in the JSON context.
Synthesize the most critical points into 3-5 concise, high-level takeaways.
↳ Do not just list the subtitles. Capture the core lesson of each major
↳ section.
Your output MUST be a single, well-formed JSON object that adheres to the
↳ standard slide structure below. This ensures it can be rendered correctly.

JSON OUTPUT STRUCTURE:
{{
"title": "Summary & Key Takeaways",
"subtitle": null,
"objects": [
{{
"content_type": "bullet_points",
"content_purpose": "description",
"data": [
"string | Key takeaway 1.",
"string | Key takeaway 2.",
"string | Key takeaway 3.",
"string | Key takeaway 4.",
"string | Key takeaway 5."
]
}}
]
}}
]
}}
Now, generate the JSON for the final summary slide.
""

```

[33]: # Content Generator

```

""
Improvement to avoid repetition split the data provide? --> search sentences
↳ or by paragraphs

```

To create more interactive activities could run a program to random select the  
 ↳ prompt or 2 steps analysis - check data then select the prompt to run / more  
 ↳ capable llm probably handle repetition  
 make stronger structure retriever to secure structure for presenter  
 """

```
PROMPT FOR THE SIMPLE, SINGLE-SLIDE CASE (when direct_slides_content == 1)
This prompt does NOT have 'slide_subtitle_to_generate'
SINGLE_STEP_CONTENT_SLIDE_GENERATION_PROMPT = """
You are an expert university lecturer and instructional designer. Your task is
↳ to process a large block of raw text and intelligently divide it into a
↳ sequence of PowerPoint slides.

CONTEXT:
- **Main Topic:** "{main_topic_title}"
- **The ENTIRE Raw Text for this Topic:** ``{topic_raw_content}``
- **Total Slides to Create:** You must cover all the key concepts in the raw
↳ text over a total of **{total_slides_for_topic}** slides.
- **Current Task:** You are generating slide **{current_slide_num}** of
↳ {total_slides_for_topic}.
- **Topics Already Covered on Previous Slides:** {generation_history}

YOUR GOAL AND INSTRUCTIONS:
1. **Analyze the ENTIRE raw text.** Identify the distinct sub-topics within it.
2. **Review the topics already covered** in the `generation_history`.
3. **Identify the NEXT logical sub-topic from the raw text that has NOT been
↳ covered yet.** Your primary goal is to ensure comprehensive coverage without
↳ repetition.
4. **Create the content for a single slide** that is focused only on this
↳ new, uncovered sub-topic.
5. Your output MUST be a single, well-formed JSON object. Do not add any
↳ text or explanations outside the JSON object.

JSON OUTPUT STRUCTURE & RULES:
{{
 "title": "string | The main title for the slide, consistent with the main
↳ topic.",
 "subtitle": "string | A more specific subtitle for this particular slide. Use
↳ null if not needed.",
 "objects": [
 {{
 "content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
 "content_purpose": "string | CHOOSE ONE: 'description', 'explanation',
↳ 'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',
↳ 'case_study', 'example'.",
 "data": "object | The structured data for the slide. See examples below."
 }}
]
}}
```

```

 }}
]
}}

DATA STRUCTURE EXAMPLES:
- For "bullet_points": "data": [{"Point 1", {"text": "Point 2", "children": [
 ↳ ["Sub-point 2.1"]}}]}
- For "table": "data": {"headers": ["Col A"], "rows": [["Data 1"]]}

Now, generate the JSON for slide {current_slide_num}.
"""

PROMPT FOR THE ADVANCED, MULTI-SLIDE CASE (when direct_slides_content > 1)
This is the prompt you currently have, just renamed.
MULTIPLE_STEP_CONTENT_SLIDE_GENERATION_PROMPT = """
You are an expert university lecturer creating the content for a single,
↳ specific PowerPoint slide based on a pre-defined plan.

CONTEXT:
- **Main Topic:** "{main_topic_title}"
- **The ENTIRE Raw Text for this Topic:** ``{topic_raw_content}``
- **Current Task:** You are generating slide **{current_slide_num}** of
 ↳ **{total_slides_for_topic}**.
- **Slide Focus:** This slide should be exclusively about
 ↳ **"{slide_subtitle_to_generate}"**.
- **Topics Already Covered on Previous Slides:** {generation_history}

YOUR GOAL AND INSTRUCTIONS:
1. **Refer to the ENTIRE raw text** to find the information related to
 ↳ **"{slide_subtitle_to_generate}"**.
2. **Create the content for this single slide.**
3. **Do not repeat content from previous slides** as detailed in the history.
4. Your output **MUST** be a single, well-formed JSON object.

JSON OUTPUT STRUCTURE & RULES:
{{
 "title": "{main_topic_title}",
 "subtitle": "{slide_subtitle_to_generate}",
 "objects": [
 {{
 "content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
 "content_purpose": "string | CHOOSE ONE: 'description', 'explanation',
 ↳ 'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',
 ↳ 'case_study', 'example'.",
 "data": "object | The structured data for the slide."
 }}
]
}}

```

```
]
}}
```

Now, generate the JSON for slide {current\_slide\_num}.  
"""

```
INTERACTIVE_ACTIVITY_PROMPT = """
```

```
You are an engaging university lecturer creating an interactive slide to test_
↳student understanding.
```

```
CONTEXT:
```

```
- **Topic for this Activity:** "{topic_title}"
- **Raw Text Content for this Topic:** ``{topic_raw_content}``
```

```
INSTRUCTIONS:
```

1. Create a single, insightful multiple-choice question that assesses a key\_  
↳concept from the provided raw text.
2. The question must have exactly 4 options. Provide the correct answer and a\_  
↳brief, clear explanation.
3. Your output **\*\*MUST\*\*** be a single, well-formed JSON object that adheres\_  
↳strictly to the structure below.

```
JSON OUTPUT STRUCTURE:
```

```
{
 "title": "Let's Apply This!",
 "subtitle": "Knowledge Check: {topic_title}",
 "objects": [
 {
 "content_type": "multiple_choice_question",
 "content_purpose": "knowledge_check",
 "data": {
 "question_text": "string | The question to be asked.",
 "options": [
 {"label": "A", "text": "string"},
 {"label": "B", "text": "string"},
 {"label": "C", "text": "string"},
 {"label": "D", "text": "string"}
],
 "correct_answer": {
 "label": "string | e.g., 'B'",
 "explanation": "string | A brief explanation of why this answer is_
↳correct."
 }
 }
 }
]
}
```

```

}}

Now, generate the JSON for the interactive activity.
"""

=====
NEW, CONTEXT-AWARE SUMMARY GENERATION PROMPT
This prompt provides the LLM with the content of all slides for a rich
 ↳summary.
=====

SUMMARY_GENERATION_PROMPT = """
You are an expert university lecturer crafting the final, conclusive summary
 ↳slide for a lecture.

CONTEXT:
- **Overall Lecture Topic:** "{lecture_topic}"
- **Detailed Content of Slides Presented:**
  ```json
  {slide_content_details}
  INSTRUCTIONS:
  Analyze the detailed content of all the slides provided in the JSON context.
  Synthesize the most critical points into 3-5 concise, high-level takeaways. Do
    ↳not just list the subtitles. Capture the core lesson of each major section.
  Your output MUST be a single, well-formed JSON object that adheres to the
    ↳standard slide structure below. This ensures it can be rendered correctly.

JSON OUTPUT STRUCTURE:
{{
  "title": "Summary & Key Takeaways",
  "subtitle": null,
  "objects": [
    {{
      "content_type": "bullet_points",
      "content_purpose": "description",
      "data": [
        "string | Key takeaway 1.",
        "string | Key takeaway 2.",
        "string | Key takeaway 3.",
        "string | Key takeaway 4.",
        "string | Key takeaway 5."
      ]
    }}
  ]
}}
]
}}

Now, generate the JSON for the final summary slide.
"""

```

```

PLANNING_PROMPT = """
You are an expert instructional designer tasked with creating a content plan
↳for a series of PowerPoint slides.

**CONTEXT:**
- **Main Topic:** "{main_topic_title}"
- **The ENTIRE Raw Text for this Topic:** ``{topic_raw_content}``
- **Total Slides to Create:** You must create a plan to cover all the key
↳concepts in the raw text over a total of **{total_slides_for_topic}** slides.

**YOUR GOAL AND INSTRUCTIONS:**
1. **Analyze the ENTIRE raw text.**
2. **Identify the main sub-topics** that need to be covered.
3. **Create a plan** to break down these sub-topics into a sequence of
↳**{total_slides_for_topic}** slides. Each slide in your plan should cover a
↳distinct and logical piece of information.
4. **Your output MUST be a single, well-formed JSON object** that is an array
↳of slide plans. Do not add any text or explanations outside of the JSON
↳object.

**JSON OUTPUT STRUCTURE & RULES:**
{{
  "slide_plan": [
    {{
      "slide_number": 1,
      "subtitle": "A concise and descriptive subtitle for this slide's content."
    }},
    {{
      "slide_number": 2,
      "subtitle": "A concise and descriptive subtitle for the next slide's
↳content."
    }}
  ]
}}

Now, generate the JSON for the slide plan.
"""

import os
import json
import re
import logging
from typing import List, Dict, Any, Optional

# Third-party imports (ensure these are installed)
try:
    import ollama

```

```

    from tenacity import retry, stop_after_attempt, wait_exponential
except ImportError:
    print("Please install required libraries: pip install ollama tenacity")
    ollama = None

# Basic Logger
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

class ContentAgent:
    """
    Processes a hierarchical content plan, using specialized LLM prompts to
    generate
    a sequence of individual, richly-structured slides.
    """
    def __init__(self, master_config: Dict, vector_store: Optional[Any] = None):
        self.config = master_config['processed_settings']
        self.unit_outline = master_config['unit_outline']
        self.book_toc = master_config['book_toc']
        #self.teaching_flows = master_config['teaching_flows']
        #self.layout_slides = master_config['layout_slides']
        self.vector_store = vector_store
        self.client = ollama.Client(host=master_config.get('OLLAMA_HOST', 'http://localhost:11434'))
        self.model = master_config.get('OLLAMA_MODEL', 'qwen3:8b')
        # --- Added for Progress Tracking ---
        self.total_slides_to_generate = 0
        self.llm_calls_made = 0
        logger.info(f"Data-Driven Content Agent initialized with model '{self.model}'.")

    # Define the desired key order as a class attribute
    self.key_order = [
        "title",
        "toc_id",
        "chunk_count",
        "total_chunks_in_branch",
        "budget_slides_content",
        "direct_slides_content",
        "total_slides_in_branch",
        "time_allocation_minutes",
        "chunks_sorted",
        "content",
    ]

```

```

        "llm_generated_content",
        "children",
        "interactive_activity"
    ]

    logger.info("Data-Driven Content Agent initialized successfully.")

    # --- Key Reordering Logic (REFINED) ---
    def _reorder_keys_recursively(self, node: dict) -> dict:
        """
        Recursively traverses a dictionary (a node in the plan) and reorders
        ↪ its keys
        according to a predefined order for maximum readability.
        """
        if not isinstance(node, dict):
            return node

        # 1. Recurse first to ensure nested structures are already ordered.
        if 'children' in node and isinstance(node.get('children'), list):
            node['children'] = [self._reorder_keys_recursively(child) for child
            ↪ in node['children']]

            if 'interactive_activity' in node and isinstance(node.
            ↪ get('interactive_activity'), dict):
                node['interactive_activity'] = self.
                ↪ _reorder_keys_recursively(node['interactive_activity'])

        # 2. Build a new dictionary for the current node with the correct key
        ↪ order.
        reordered_node = {}

        # Add keys that are in our desired order
        for key in self.key_order:
            if key in node:
                reordered_node[key] = node[key]

        # Add any remaining keys that were not in the order list (as a fallback)
        for key, value in node.items():
            if key not in reordered_node:
                reordered_node[key] = value

        return reordered_node

    # --- Phase 5: Raw Content Population ---
    def retrieve_content_for_toc_id(self, toc_id: int) -> dict:

```

```

        if not isinstance(toc_id, int):
            logger.warning(f"Invalid toc_id: {toc_id}. Must be an integer.")
            return {"chunks_sorted": [], "content": ""}
        try:
            results = self.vector_store.get(where={"toc_id": toc_id},
↪include=["documents", "metadatas"])
            if not results or not results.get('ids'):
                logger.warning(f"No chunks found in the database for toc_id =
↪{toc_id}")
                return {"chunks_sorted": [], "content": ""}
            sorted_items = sorted(zip(results['documents'],
↪results['metadatas']), key=lambda item: item[1].get('chunk_id', 0))
            sorted_docs = [item[0] for item in sorted_items]
            sorted_chunk_ids = [item[1].get('chunk_id') for item in
↪sorted_items]
            reassembled_text = "\n\n".join(sorted_docs)
            return {"chunks_sorted": sorted_chunk_ids, "content":
↪reassembled_text}
        except Exception as e:
            logger.error(f"An error occurred during retrieval for toc_id
↪{toc_id}: {e}", exc_info=True)
            return {"chunks_sorted": [], "content": ""}

    def populate_content_recursively(self, node: dict):

        if 'toc_id' in node and 'content' not in node:
            content_data = self.retrieve_content_for_toc_id(node['toc_id'])
            node.update(content_data)
        if 'children' in node and isinstance(node.get('children'), list):
            for child in node['children']:
                self.populate_content_recursively(child)

    def generate_content_for_plan(self, final_plan_path: str, output_dir: str)
↪-> bool:

        logger.info(f"PHASE 5: Populating raw content for: {final_plan_path}")
        try:
            with open(final_plan_path, 'r', encoding='utf-8') as f:
                plan_data = json.load(f)
            except (FileNotFoundError, json.JSONDecodeError) as e:
                logger.error(f"FATAL: Could not read or decode plan file
↪{final_plan_path}. Error: {e}")
                return False

        for deck in plan_data.get('deck_plans', []):
            for section in deck.get('sections', []):

```

```

        if section.get('section_type') == 'Content':
            for content_block in section.get('content_blocks', []):
                self.populate_content_recursively(content_block)

base_filename = os.path.basename(final_plan_path)
output_path = os.path.join(output_dir, base_filename)
os.makedirs(output_dir, exist_ok=True)

logger.info("Reordering keys for Fetched clean output...")
fetched_ordered_plan = self._reorder_keys_recursively(plan_data)

try:
    with open(output_path, 'w', encoding='utf-8') as f:
        # CORRECTED: Save the 'fetched_ordered_plan' variable
        json.dump(fetched_ordered_plan, f, indent=2, ensure_ascii=False)
    logger.info(f"Successfully saved content-enriched plan to: {output_path}")
    return True
except Exception as e:
    logger.error(f"Failed to save the content-enriched plan to: {output_path}: {e}", exc_info=True)
    return False

# --- Phase 6: LLM Content Generation & Final Formatting ---

@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=2, max=10))
def _call_ollama_with_retry(self, prompt: str) -> str:
    # --- Incremented Counter Here ---
    self.llm_calls_made += 1
    logger.info(f"Calling Ollama model '{self.model}' (Call #{self.llm_calls_made})...")
    response = self.client.chat(model=self.model, messages=[{"role": "user", "content": prompt}], format="json", options={"temperature": 0.2})
    if not response or 'message' not in response or not response['message'].get('content'):
        raise ValueError("Ollama returned an empty or invalid response.")
    return response['message']['content']

def _parse_llm_json_output(self, content: str) -> Optional[Dict]:
    try:
        match = re.search(r'\{.*\}', content, re.DOTALL)
        if not match: return None
        return json.loads(match.group(0))
    except (json.JSONDecodeError, TypeError): return None

def _process_content_node_recursively(self, node: dict):
    """

```

Recursively processes a content node by generating its own slides, processing its children, and then generating its interactive activity. This method modifies the 'node' dictionary in-place.

It uses a simple generation for direct_slides_content == 1, and a two-step "plan-then-generate" method for direct_slides_content > 1 to prevent content duplication.

```
"""
num_slides_to_gen = node.get('direct_slides_content', 0)
generated_slides = []

# --- Step 1: Generate slides for the current content node ---
if num_slides_to_gen > 0:

    # --- PATH 1: Simple case for a single slide ---
    if num_slides_to_gen == 1:
        progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
        logger.info(f"{progress_log} Generating 1 slide for topic:␣
↪'{node.get('title')}'")
        try:
            # Use the ORIGINAL, direct prompt
            prompt = SINGLE_STEP_CONTENT_SLIDE_GENERATION_PROMPT.format(
                main_topic_title=node.get('title'),
                topic_raw_content=node.get('content', ''),
                current_slide_num=1,
                total_slides_for_topic=1,
                generation_history="None. This is the only slide for␣
↪this topic."
            )
            llm_str = self._call_ollama_with_retry(prompt)
            slide_content = self._parse_llm_json_output(llm_str)
            if slide_content:
                slide_object = {
                    "seq_id": float(f"{node.get('seq_id', 0)}.1"),
                    "llm_generated_content": slide_content
                }
                generated_slides.append(slide_object)
            except Exception as e:
                logger.error(f"LLM call failed for single slide '{node.
↪get('title')}': {e}", exc_info=True)

        # --- PATH 2: Advanced case for splitting content across multiple␣
↪slides ---
    else: # num_slides_to_gen > 1
        logger.info(f" Splitting topic '{node.get('title')}' into␣
↪{num_slides_to_gen} slides using plan-then-generate.")
```

```

try:
    # STEP 2.1: Create the content plan
    planning_prompt = PLANNING_PROMPT.format(
        main_topic_title=node.get('title'),
        topic_raw_content=node.get('content', ''),
        total_slides_for_topic=num_slides_to_gen
    )
    plan_str = self._call_ollama_with_retry(planning_prompt)
    slide_plan = self._parse_llm_json_output(plan_str).
    get("slide_plan", [])

    if not slide_plan or len(slide_plan) != num_slides_to_gen:
        logger.error(f"Failed to generate a valid slide plan
    from the LLM for topic '{node.get('title')}'. Expected {num_slides_to_gen}
    slides, got {len(slide_plan)}." )
    else:
        # STEP 2.2: Execute the plan and generate each slide
        generation_history = "None. This is the first slide."
        for i, plan_item in enumerate(slide_plan):
            current_slide_num = i + 1
            slide_subtitle = plan_item.get("subtitle", f"Part
    {current_slide_num}")

            progress_log = f"[Progress: {self.llm_calls_made}/
    {self.total_slides_to_generate}]"
            logger.info(f"{progress_log} Generating slide
    {current_slide_num}/{num_slides_to_gen} for topic: '{node.get('title')} ->
    {slide_subtitle}")

            # Use the MODIFIED prompt that requires a specific
    subtitle

            gen_prompt =
    MULTIPLE_STEP_CONTENT_SLIDE_GENERATION_PROMPT.format(
                main_topic_title=node.get('title'),
                topic_raw_content=node.get('content', ''),
                current_slide_num=current_slide_num,
                total_slides_for_topic=num_slides_to_gen,
                slide_subtitle_to_generate=slide_subtitle,
                generation_history=generation_history
            )
            llm_str = self._call_ollama_with_retry(gen_prompt)
            slide_content = self._parse_llm_json_output(llm_str)

            if slide_content:
                slide_object = {
                    "seq_id": float(f"{node.get('seq_id', 0)}".
    {current_slide_num}"),

```

```

        "llm_generated_content": slide_content
    }
    generated_slides.append(slide_object)
    generation_history += f"\n- Slide␣
↪{current_slide_num} subtitle: {slide_content.get('subtitle', 'N/A')}}"

    except Exception as e:
        logger.error(f" LLM call failed during multi-slide␣
↪generation for '{node.get('title')}': {e}", exc_info=True)

    # Assign the generated slides back to the node
    if generated_slides:
        node['slides'] = generated_slides

    # --- Step 2: Recursively process all children of the current node ---
    for child_node in node.get('children', []):
        self._process_content_node_recursively(child_node)

    # --- Step 3: Generate the interactive activity for the current node␣
↪(if any) ---
    if 'interactive_activity' in node:
        progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
        logger.info(f"{progress_log} Generating interactive slide for␣
↪topic: '{node.get('title')}')")
        try:
            prompt = INTERACTIVE_ACTIVITY_PROMPT.format(
                topic_title=node.get('title'),
                topic_raw_content=node.get('content', '')
            )
            llm_str = self._call_ollama_with_retry(prompt)
            interactive_content = self._parse_llm_json_output(llm_str)
            if interactive_content:
                node['interactive_activity']['llm_generated_content'] =␣
↪interactive_content
        except Exception as e:
            logger.error(f"LLM call failed for interactive activity on␣
↪'{node.get('title')}': {e}", exc_info=True)

    def _count_total_slides(self, plan_data: Dict):
        """Recursively counts the total number of slides to be generated."""
        count = 0
        for deck in plan_data.get('deck_plans', []):
            # Count summary slide
            if any(s.get('section_type') == 'Summary' for s in deck.
↪get('sections', [])):

```

```

        count += 1

    # Count content slides
    for section in deck.get('sections', []):
        if section.get('section_type') == 'Content':
            for content_block in section.get('content_blocks', []):

                def _recursive_count(node):
                    nonlocal count
                    count += node.get('direct_slides_content', 0)
                    if 'interactive_activity' in node:
                        count += 1
                    for child in node.get('children', []):
                        _recursive_count(child)

                _recursive_count(content_block)

    return count

    def generate_llm_content_for_plan(self, content_plan_path: str,
    llm_output_dir: str) -> bool:
        """
        Orchestrates the LLM content generation for the entire plan file,
        including a rich, context-aware summary.
        """
        logger.info(f"PHASE 6: Generating LLM content for: {os.path.
    basename(content_plan_path)} output file path: {llm_output_dir}")
        try:
            with open(content_plan_path, 'r', encoding='utf-8') as f:
                plan_data = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError) as e:
            logger.error(f"FATAL: Could not read plan file {content_plan_path}.
    Error: {e}")
            return False

        # --- Pre-calculate total slides for progress tracking ---
        self.total_slides_to_generate = self._count_total_slides(plan_data)
        self.llm_calls_made = 0 # Reset counter for the run
        logger.info(f"Found a total of {self.total_slides_to_generate} slides
    to generate across all decks.")

        summary_prompt_template = SUMMARY_GENERATION_PROMPT

        for deck in plan_data.get('deck_plans', []):
            all_slide_contents_for_summary = []

            for section in deck.get('sections', []):
                if section.get('section_type') == 'Content':

```

```

        for content_block in section.get('content_blocks', []):
            self._process_content_node_recursively(content_block)

    for section in deck.get('sections', []):
        if section.get('section_type') == 'Content':
            for content_block in section.get('content_blocks', []):
                def _collect_slide_content(node):
                    for slide in node.get('slides', []):
                        all_slide_contents_for_summary.append(slide.
↪get('llm_generated_content'))
                        if 'interactive_activity' in node and
↪'llm_generated_content' in node['interactive_activity']:
                            all_slide_contents_for_summary.
↪append(node['interactive_activity'].get('llm_generated_content'))
                            for child in node.get('children', []):
                                _collect_slide_content(child)
                                _collect_slide_content(content_block)

            if summary_prompt_template and all_slide_contents_for_summary:
                summary_section = next((s for s in deck['sections'] if s.
↪get('section_type') == 'Summary'), None)

                if summary_section:
                    # --- Updated Progress Log for Summary Slide ---
                    progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
                    logger.info(f"{progress_log} Generating summary for Deck
↪{deck.get('deck_number', 'N/A')}.")
                    try:
                        slide_details_str = json.
↪dumps(all_slide_contents_for_summary, indent=2)
                        prompt = summary_prompt_template.format(
                            lecture_topic=plan_data.get('overall_topic', 'This
↪Lecture'),
                            slide_content_details=slide_details_str
                        )
                        llm_str = self._call_ollama_with_retry(prompt)
                        summary_content = self._parse_llm_json_output(llm_str)
                        if summary_content:
                            summary_section['llm_generated_content'] =
↪summary_content
                        else:
                            logger.warning("Failed to parse summary content
↪from LLM.")

```

```

        summary_section['llm_generated_content'] = {"title":
↪ "Summary & Key Takeaways", "subtitle": None, "objects": [{"content_type":
↪ "bullet_points", "content_purpose": "description", "data": ["Summary could
↪ not be generated."]}]}

        except Exception as e:
            logger.error(f"LLM call failed for deck summary: {e}",
↪ exc_info=True)

        base_filename = os.path.basename(content_plan_path).replace('.json',
↪ '_llm_generated.json')
        output_path = os.path.join(llm_output_dir, base_filename)
        os.makedirs(llm_output_dir, exist_ok=True)
        try:
            with open(output_path, 'w', encoding='utf-8') as f:
                json.dump(plan_data, f, indent=2, ensure_ascii=False)
            logger.info(f"Successfully saved final LLM-enriched plan to:
↪ {output_path}")
            return True
        except Exception as e:
            logger.error(f"Failed to save the final LLM-enriched plan: {e}",
↪ exc_info=True)
            return False

# =====
# TEST EXECUTION BLOCK
# =====
if __name__ == '__main__' and ollama is not None:
    print_header("Content Agent Test Runner", char="=")

    CONTENT_TO_PROCESS = '/home/sebas_dev_linux/projects/course_generator/
↪ data_output/3. generated_content/ICT312/ICT312_Week1_plan_final.json'

    # --- 3. Set up a Dummy Master Configuration ---
    master_config = process_and_load_configurations()

    # --- 4. Instantiate and Run the Agent ---
    try:
        content_agent = ContentAgent(master_config)

        # Run the agent in test_mode to only process the first deck
        success = content_agent.generate_llm_content_for_plan(

```

```

        content_plan_path=CONTENT_TO_PROCESS,
        llm_output_dir=CONTENT_LLM_OUTPUT_DIR,

    )

    if success:
        logger.info(" Test run completed successfully!")
        logger.info(f"Check the output file in the_
↪'{CONTENT_LLM_OUTPUT_DIR}' directory.")
    else:
        logger.error(" Test run failed.")

    except NameError:
        logger.error("Ollama library not found. Skipping test execution.")
    except Exception as e:
        logger.error(f"An unexpected error occurred during the test run: {e}",_
↪exc_info=True)

```

```

2025-07-18 23:48:40,719 - INFO - Loading all necessary configuration and data
files...
2025-07-18 23:48:40,722 - INFO - All files loaded successfully.
2025-07-18 23:48:40,722 - INFO - Pre-processing settings_deck for definitive
plan...
2025-07-18 23:48:40,723 - INFO - Applying overrides if specified...
2025-07-18 23:48:40,723 - INFO - Loaded from settings: 'interactive' is True.
Set teaching_flow_id to 'apply_topic_interactive'.
2025-07-18 23:48:40,723 - INFO - Calculating preliminary slide budget based on
session time...
2025-07-18 23:48:40,724 - INFO - Duration Hours 2.
2025-07-18 23:48:40,724 - INFO - Sessions per week 1.
2025-07-18 23:48:40,724 - INFO - Preliminary weekly content slide target
calculated: # 32 slides.
2025-07-18 23:48:40,725 - INFO - Saving preliminary processed configuration to:
/home/sebas_dev_linux/projects/course_generator/configs/processed_settings.json
2025-07-18 23:48:40,725 - INFO - File saved successfully.
2025-07-18 23:48:40,726 - INFO - Master configuration object is ready for the
Planning Agent.
2025-07-18 23:48:40,733 - INFO - Data-Driven Content Agent initialized with
model 'qwen3:8b'.
2025-07-18 23:48:40,733 - INFO - Data-Driven Content Agent initialized
successfully.
2025-07-18 23:48:40,733 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week1_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:48:40,735 - INFO - Found a total of 40 slides to generate across
all decks.
2025-07-18 23:48:40,735 - INFO - [Progress: 0/40] Generating 1 slide for topic:

```

'An Overview of Digital Forensics'

2025-07-18 23:48:40,735 - INFO - Calling Ollama model 'qwen3:8b' (Call #1)...

=====
Content Agent Test Runner
=====

Phase 1: Configuration and Scoping Process

Phase 1 Configuration Complete

2025-07-18 23:48:52,055 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:48:52,056 - INFO - [Progress: 1/40] Generating 1 slide for topic:
'Digital Forensics and Other Related Disciplines'
2025-07-18 23:48:52,056 - INFO - Calling Ollama model 'qwen3:8b' (Call #2)..
2025-07-18 23:48:52,056 - INFO - [Progress: 1/40] Generating 1 slide for topic:
'Digital Forensics and Other Related Disciplines'
2025-07-18 23:48:52,056 - INFO - Calling Ollama model 'qwen3:8b' (Call #2)..
2025-07-18 23:49:03,490 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:03,491 - INFO - [Progress: 2/40] Generating 1 slide for topic:
'A Brief History of Digital Forensics'
2025-07-18 23:49:03,491 - INFO - Calling Ollama model 'qwen3:8b' (Call #3)..
2025-07-18 23:49:11,727 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:11,727 - INFO - [Progress: 3/40] Generating 1 slide for topic:
'Developing Digital Forensics Resources'
2025-07-18 23:49:11,728 - INFO - Calling Ollama model 'qwen3:8b' (Call #4)..
2025-07-18 23:49:17,202 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:17,203 - INFO - [Progress: 4/40] Generating interactive slide
for topic: 'An Overview of Digital Forensics'
2025-07-18 23:49:17,203 - INFO - Calling Ollama model 'qwen3:8b' (Call #5)..
2025-07-18 23:49:25,557 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:25,558 - INFO - [Progress: 5/40] Generating 1 slide for topic:
'Understanding Law Enforcement Agency Investigations'
2025-07-18 23:49:25,558 - INFO - Calling Ollama model 'qwen3:8b' (Call #6)..
2025-07-18 23:49:34,846 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:34,847 - INFO - [Progress: 6/40] Generating 1 slide for topic:
'Following Legal Processes'
2025-07-18 23:49:34,847 - INFO - Calling Ollama model 'qwen3:8b' (Call #7)...

2025-07-18 23:49:38,445 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:38,445 - INFO - [Progress: 7/40] Generating 1 slide for topic:
'Displaying Warning Banners'
2025-07-18 23:49:38,446 - INFO - Calling Ollama model 'qwen3:8b' (Call #8)...
2025-07-18 23:49:46,444 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:46,445 - INFO - [Progress: 8/40] Generating 1 slide for topic:
'Designating an Authorized Requester'
2025-07-18 23:49:46,445 - INFO - Calling Ollama model 'qwen3:8b' (Call #9)...
2025-07-18 23:49:50,589 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:50,590 - INFO - [Progress: 9/40] Generating 1 slide for topic:
'Conducting Security Investigations'
2025-07-18 23:49:50,590 - INFO - Calling Ollama model 'qwen3:8b' (Call #10)...
2025-07-18 23:49:59,836 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:49:59,837 - INFO - [Progress: 10/40] Generating interactive slide
for topic: 'Preparing for Digital Investigations'
2025-07-18 23:49:59,837 - INFO - Calling Ollama model 'qwen3:8b' (Call #11)...
2025-07-18 23:50:06,804 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:06,805 - INFO - [Progress: 11/40] Generating 1 slide for topic:
'Maintaining Professional Conduct'
2025-07-18 23:50:06,805 - INFO - Calling Ollama model 'qwen3:8b' (Call #12)...
2025-07-18 23:50:13,372 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:13,373 - INFO - [Progress: 12/40] Generating interactive slide
for topic: 'Maintaining Professional Conduct'
2025-07-18 23:50:13,373 - INFO - Calling Ollama model 'qwen3:8b' (Call #13)...
2025-07-18 23:50:21,335 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:21,336 - INFO - [Progress: 13/40] Generating 1 slide for topic:
'An Overview of a Computer Crime'
2025-07-18 23:50:21,336 - INFO - Calling Ollama model 'qwen3:8b' (Call #14)...
2025-07-18 23:50:27,050 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:27,050 - INFO - [Progress: 14/40] Generating 1 slide for topic:
'Taking a Systematic Approach'
2025-07-18 23:50:27,051 - INFO - Calling Ollama model 'qwen3:8b' (Call #15)...
2025-07-18 23:50:34,960 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:34,961 - INFO - [Progress: 15/40] Generating 1 slide for topic:
'Assessing the Case'
2025-07-18 23:50:34,962 - INFO - Calling Ollama model 'qwen3:8b' (Call #16)...
2025-07-18 23:50:41,167 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:50:41,167 - INFO - Splitting topic 'Planning Your Investigation'

into 3 slides using plan-then-generate.

2025-07-18 23:50:41,168 - INFO - Calling Ollama model 'qwen3:8b' (Call #17)...

2025-07-18 23:50:44,470 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:50:44,471 - INFO - [Progress: 17/40] Generating slide 1/3 for topic: 'Planning Your Investigation' -> 'Acquiring Evidence and Establishing Chain of Custody'

2025-07-18 23:50:44,471 - INFO - Calling Ollama model 'qwen3:8b' (Call #18)...

2025-07-18 23:50:53,397 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:50:53,398 - INFO - [Progress: 18/40] Generating slide 2/3 for topic: 'Planning Your Investigation' -> 'Documenting Evidence with Custody Forms'

2025-07-18 23:50:53,398 - INFO - Calling Ollama model 'qwen3:8b' (Call #19)...

2025-07-18 23:50:59,313 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:50:59,314 - INFO - [Progress: 19/40] Generating slide 3/3 for topic: 'Planning Your Investigation' -> 'Secure Handling and Forensic Copying of Evidence'

2025-07-18 23:50:59,314 - INFO - Calling Ollama model 'qwen3:8b' (Call #20)...

2025-07-18 23:51:05,634 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:05,635 - INFO - [Progress: 20/40] Generating 1 slide for topic: 'Securing Your Evidence'

2025-07-18 23:51:05,635 - INFO - Calling Ollama model 'qwen3:8b' (Call #21)...

2025-07-18 23:51:10,109 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:10,110 - INFO - [Progress: 21/40] Generating interactive slide for topic: 'Preparing a Digital Forensics Investigation'

2025-07-18 23:51:10,110 - INFO - Calling Ollama model 'qwen3:8b' (Call #22)...

2025-07-18 23:51:16,720 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:16,721 - INFO - [Progress: 22/40] Generating 1 slide for topic: 'Internet Abuse Investigations'

2025-07-18 23:51:16,722 - INFO - Calling Ollama model 'qwen3:8b' (Call #23)...

2025-07-18 23:51:21,667 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:21,668 - INFO - [Progress: 23/40] Generating 1 slide for topic: 'E-mail Abuse Investigations'

2025-07-18 23:51:21,668 - INFO - Calling Ollama model 'qwen3:8b' (Call #24)...

2025-07-18 23:51:32,207 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:32,208 - INFO - Splitting topic 'Attorney-Client Privilege Investigations' into 2 slides using plan-then-generate.

2025-07-18 23:51:32,209 - INFO - Calling Ollama model 'qwen3:8b' (Call #25)...

2025-07-18 23:51:34,588 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:51:34,589 - INFO - [Progress: 25/40] Generating slide 1/2 for

topic: 'Attorney-Client Privilege Investigations' -> 'Overview of Attorney-Client Privilege Investigations: Key Concepts and Legal Framework'
2025-07-18 23:51:34,589 - INFO - Calling Ollama model 'qwen3:8b' (Call #26)...
2025-07-18 23:51:40,997 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:51:40,997 - INFO - [Progress: 26/40] Generating slide 2/2 for topic: 'Attorney-Client Privilege Investigations' -> 'Step-by-Step Procedures and Best Practices for ACP Case Handling'
2025-07-18 23:51:40,998 - INFO - Calling Ollama model 'qwen3:8b' (Call #27)...
2025-07-18 23:51:59,980 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:51:59,981 - INFO - Splitting topic 'Industrial Espionage Investigations' into 2 slides using plan-then-generate.
2025-07-18 23:51:59,982 - INFO - Calling Ollama model 'qwen3:8b' (Call #28)...
2025-07-18 23:52:02,268 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:02,269 - INFO - [Progress: 28/40] Generating slide 1/2 for topic: 'Industrial Espionage Investigations' -> 'Overview of Industrial Espionage Investigations: Key Considerations and Legal Framework'
2025-07-18 23:52:02,270 - INFO - Calling Ollama model 'qwen3:8b' (Call #29)...
2025-07-18 23:52:08,511 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:08,511 - INFO - [Progress: 29/40] Generating slide 2/2 for topic: 'Industrial Espionage Investigations' -> 'Investigation Steps and Planning Considerations for Industrial Espionage Cases'
2025-07-18 23:52:08,512 - INFO - Calling Ollama model 'qwen3:8b' (Call #30)...
2025-07-18 23:52:17,504 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:17,505 - INFO - [Progress: 30/40] Generating 1 slide for topic: 'Interviews and Interrogations in High-Tech Investigations'
2025-07-18 23:52:17,506 - INFO - Calling Ollama model 'qwen3:8b' (Call #31)...
2025-07-18 23:52:22,268 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:22,269 - INFO - [Progress: 31/40] Generating interactive slide for topic: 'Procedures for Private-Sector High-Tech Investigations'
2025-07-18 23:52:22,269 - INFO - Calling Ollama model 'qwen3:8b' (Call #32)...
2025-07-18 23:52:28,422 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:28,423 - INFO - [Progress: 32/40] Generating 1 slide for topic: 'Understanding Data Recovery Workstations and Software'
2025-07-18 23:52:28,423 - INFO - Calling Ollama model 'qwen3:8b' (Call #33)...
2025-07-18 23:52:38,641 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:38,642 - INFO - [Progress: 33/40] Generating 1 slide for topic: 'Setting Up Your Workstation for Digital Forensics'
2025-07-18 23:52:38,643 - INFO - Calling Ollama model 'qwen3:8b' (Call #34)...
2025-07-18 23:52:42,012 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

2025-07-18 23:52:42,013 - INFO - [Progress: 34/40] Generating interactive slide for topic: 'Understanding Data Recovery Workstations and Software'
2025-07-18 23:52:42,013 - INFO - Calling Ollama model 'qwen3:8b' (Call #35)...
2025-07-18 23:52:47,955 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:47,956 - INFO - [Progress: 35/40] Generating 1 slide for topic: 'Gathering the Evidence'
2025-07-18 23:52:47,956 - INFO - Calling Ollama model 'qwen3:8b' (Call #36)...
2025-07-18 23:52:53,031 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:53,032 - INFO - Splitting topic 'Analyzing Your Digital Evidence' into 3 slides using plan-then-generate.
2025-07-18 23:52:53,032 - INFO - Calling Ollama model 'qwen3:8b' (Call #37)...
2025-07-18 23:52:56,180 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:52:56,180 - INFO - [Progress: 37/40] Generating slide 1/3 for topic: 'Analyzing Your Digital Evidence' -> 'Understanding Digital Evidence Recovery: Deleted Files and Forensic Tools'
2025-07-18 23:52:56,181 - INFO - Calling Ollama model 'qwen3:8b' (Call #38)...
2025-07-18 23:53:01,112 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:01,113 - INFO - [Progress: 38/40] Generating slide 2/3 for topic: 'Analyzing Your Digital Evidence' -> 'Analyzing Data with Autopsy: Case Setup and Keyword Search'
2025-07-18 23:53:01,113 - INFO - Calling Ollama model 'qwen3:8b' (Call #39)...
2025-07-18 23:53:07,587 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:07,588 - INFO - [Progress: 39/40] Generating slide 3/3 for topic: 'Analyzing Your Digital Evidence' -> 'Searching for Keywords and E-mail Addresses in Digital Evidence'
2025-07-18 23:53:07,588 - INFO - Calling Ollama model 'qwen3:8b' (Call #40)...
2025-07-18 23:53:12,169 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:12,170 - INFO - [Progress: 40/40] Generating 1 slide for topic: 'Completing the Case'
2025-07-18 23:53:12,170 - INFO - Calling Ollama model 'qwen3:8b' (Call #41)...
2025-07-18 23:53:19,088 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:19,089 - INFO - [Progress: 41/40] Generating 1 slide for topic: 'Critiquing the Case'
2025-07-18 23:53:19,089 - INFO - Calling Ollama model 'qwen3:8b' (Call #42)...
2025-07-18 23:53:23,745 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:23,745 - INFO - [Progress: 42/40] Generating interactive slide for topic: 'Conducting an Investigation'
2025-07-18 23:53:23,746 - INFO - Calling Ollama model 'qwen3:8b' (Call #43)...
2025-07-18 23:53:29,728 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"

```

2025-07-18 23:53:29,729 - INFO - [Progress: 43/40] Generating summary for Deck
1...
2025-07-18 23:53:29,730 - INFO - Calling Ollama model 'qwen3:8b' (Call #44)...
2025-07-18 23:53:36,874 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 23:53:36,883 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week1_plan_final_llm_generated.json
2025-07-18 23:53:36,884 - INFO - Test run completed successfully!
2025-07-18 23:53:36,884 - INFO - Check the output file in the
'/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312' directory.

```

7.3 Presentation Agent

test Presenter

we need to ensure the presenter seq_id flow: Parent -> Child 1 -> Child 1's Activity -> Child 2 -> Child 2's Activity -> Parent's Activity.

<https://aistudio.google.com/prompts/18YaU5pG96eFMbM1l6yeG1v9j63PkLc5H>

```

[41]: # PresentationAgent

# import os
# import json
# import logging
# import random
# from pptx import Presentation
# from pptx.util import Inches
# from pptx.enum.shapes import PP_PLACEHOLDER
# from pptx.enum.text import MSO_AUTO_SIZE

# # --- Basic Setup ---
# logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
# logger = logging.getLogger(__name__)

# # --- Helper Functions (Unchanged) ---
# def _render_bullet_points(text_frame, data):
#     text_frame.clear(); text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#     def add_points(points, level):
#         for point in points:
#             if isinstance(point, dict):
#                 p = text_frame.add_paragraph(); p.text = point.get('text', '')
#                 p.level = level
#             if 'children' in point: add_points(point['children'], level + 1)

```

```

#         else:
#             p = text_frame.add_paragraph(); p.text = str(point); p.level_
↳ = level
#         add_points(data, 0)
#         if text_frame.paragraphs and not text_frame.paragraphs[0].text:
#             p = text_frame.paragraphs[0]; p._p.getparent().remove(p._p)

# def _render_table(slide, placeholder, data):
#     headers, rows_data = data.get('headers', []), data.get('rows', [])
#     if not headers or not rows_data: return
#     num_rows, num_cols = len(rows_data) + 1, len(headers)
#     table_shape = slide.shapes.add_table(num_rows, num_cols, placeholder.
↳ left, placeholder.top, placeholder.width, placeholder.height)
#     table = table_shape.table
#     for i, header in enumerate(headers):
#         cell = table.cell(0, i); cell.text = header
#         cell.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE; cell.
↳ text_frame.paragraphs[0].font.bold = True
#     for r_idx, row_data in enumerate(rows_data):
#         for c_idx, cell_text in enumerate(row_data):
#             cell = table.cell(r_idx + 1, c_idx); cell.text = str(cell_text)
#             cell.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#         sp = placeholder.element; sp.getparent().remove(sp)

# def _render_multiple_choice_question(slide, placeholders, data):
#     main_ph = next((p for p in placeholders if p.placeholder_format.type ==_
↳ PP_PLACEHOLDER.OBJECT), None)
#     answer_ph = next((p for p in placeholders if p.placeholder_format.type ==_
↳ PP_PLACEHOLDER.BODY), None)
#     if not main_ph: return
#     tf = main_ph.text_frame; tf.clear(); tf.auto_size = MSO_AUTO_SIZE.
↳ TEXT_TO_FIT_SHAPE
#     p = tf.add_paragraph(); p.text = data.get('question_text', ''); p.font.
↳ bold = True; p.level = 0
#     for option in data.get('options', []):
#         p = tf.add_paragraph(); p.text = f"{option.get('label', '')} {option.
↳ get('text', '')}"; p.level = 1
#     if answer_ph:
#         answer_ph.text_frame.clear()
#         explanation = data.get('correct_answer', {}).get('explanation', '')
#         answer_ph.text_frame.text = f"Answer: {data.get('correct_answer', {}).
↳ get('label', '')}. {explanation}"

# def _render_matching_activity(slide, placeholders, data):
#     object_placeholders = [p for p in placeholders if p.placeholder_format.
↳ type == PP_PLACEHOLDER.OBJECT]

```

```

#     if len(object_placeholders) < 2: return
#     terms_ph, defs_ph = object_placeholders[0], object_placeholders[1]
#     terms_tf, defs_tf = terms_ph.text_frame, defs_ph.text_frame
#     terms_tf.clear(); defs_tf.clear(); terms_tf.auto_size = MSO_AUTO_SIZE.
#     TEXT_TO_FIT_SHAPE; defs_tf.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#     pairs = data.get('pairs', [])
#     if not pairs: return
#     terms = [f"{i+1}. {p['term']}" for i, p in enumerate(pairs)]
#     definitions = [p['definition'] for p in pairs]
#     random.shuffle(definitions)
#     for term in terms: p = terms_tf.add_paragraph(); p.text = term; p.level = 0
#     for i, definition in enumerate(definitions): p = defs_tf.add_paragraph();
#     p.text = f"{chr(65+i)}. {definition}"; p.level = 0

# class PresentationAgent:
#     def __init__(self, template_path: str, layout_config_path: str):
#         self.template_path = template_path
#         self.prs_for_layouts = Presentation(template_path)
#         with open(layout_config_path, 'r', encoding='utf-8') as f:
#             self.user_selections = json.load(f)['user_selections']
#         logger.info(f"Agent initialized with template '{os.path.
#         basename(template_path)}'")

#     def _get_layout(self, layout_key: str):
#         selection = self.user_selections.get(layout_key, self.
#         user_selections['Content'])
#         layout_index = selection['selected_layout_index']
#         return self.prs_for_layouts.slide_layouts[layout_index]

#     def _determine_layout_key(self, item_data):
#         llm_content = item_data.get('llm_generated_content', {})
#         objects = llm_content.get('objects', [])
#         has_subtitle = bool(llm_content.get('subtitle'))

#         # <<< NEW LOGIC: Determine layout based on subtitles and object count
#         if objects and objects[0]['content_type'] in
#         ['multiple_choice_question', 'matching_activity']:
#             return "Application_Two_Column" if len(objects) > 1 or
#             objects[0]['content_type'] == 'matching_activity' else "Application"

#         if len(objects) >= 2:
#             return "Content_Two_Column_child" if has_subtitle else
#             "Content_Two_Column"
#         else:
#             return "Content_child" if has_subtitle else "Content"

```

```

#     def create_presentation_from_plan(self, plan_json_path: str, output_path:
↳str):
#         logger.info(f"Creating presentation from plan: '{os.path.
↳basename(plan_json_path)}'")
#         with open(plan_json_path, 'r', encoding='utf-8') as f: plan_data =
↳json.load(f)
#         prs = Presentation(self.template_path)
#         while len(prs.slides):
#             rId = prs.slides._sldIdLst[0].rId; prs.part.drop_rel(rId); del
↳prs.slides._sldIdLst[0]
#             slide_items = self._collect_slide_items(plan_data)
#             for item in sorted(slide_items, key=lambda x: x.get('seq_id', 999)):
↳self._add_slide_for_item(prs, item)
#             prs.save(output_path)
#             logger.info(f"Successfully created presentation: '{output_path}'")

#     def _collect_slide_items(self, plan_data):
#         items = []
#         for deck in plan_data.get('deck_plans', []):
#             for section in deck.get('sections', []):
#                 if section.get('section_type') == 'Content':
#                     for block in section.get('content_blocks', []):
#                         for slide_item in block.get('slides', []):
#                             items.append({'item_type': 'Content', 'data':
↳slide_item})
#                 else:
#                     items.append({'item_type': section['section_type'],
↳'data': section})
#         return items

#     def _add_slide_for_item(self, prs: Presentation, item: dict):
#         item_type = item['item_type']
#         layout_key = self._determine_layout_key(item['data']) if item_type ==
↳'Content' else item_type
#         slide = prs.slides.add_slide(self._get_layout(layout_key))
#         render_map = {
#             'Title': self._render_title_slide, 'Agenda': self.
↳_render_agenda_slide,
#             'Summary': self._render_summary_slide, 'End': self.
↳_render_end_slide,
#             'Divider': self._render_divider_slide, 'Content': self.
↳_render_content_slide
#         }
#         if item_type in render_map:
#             render_map[item_type](slide, item['data'])

```

```

#     def _render_title_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title:
#             slide.shapes.title.text = content.get('week_topic', 'Untitled_
↳ Topic')
#             subtitle_ph = next((p for p in slide.placeholders if p.
↳ placeholder_format.type == PP_PLACEHOLDER.SUBTITLE), None)
#             if subtitle_ph:
#                 subtitle_ph.text = f"{content.get('deck_title', '')}\n{content.
↳ get('unit_name', '')} - {content.get('unit_code', '')}"

#     def _render_agenda_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title: slide.shapes.title.text = content.get('title',
↳ 'Agenda')
#         body_ph = next((p for p in slide.placeholders if p.placeholder_format.
↳ type == PP_PLACEHOLDER.OBJECT), None)
#         if body_ph: _render_bullet_points(body_ph.text_frame, content.
↳ get('items', []))

#     def _render_summary_slide(self, slide, data):
#         if slide.shapes.title: slide.shapes.title.text = "Summary & Key
↳ Takeaways"
#         if 'llm_generated_content' in data: self._render_content_slide(slide,
↳ data)

#     def _render_end_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title: slide.shapes.title.text = content.get('text',
↳ 'Questions?')

#     def _render_divider_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title: slide.shapes.title.text = content.get('title',
↳ 'Section Divider')

#     def _render_content_slide(self, slide, data):
#         content = data.get('llm_generated_content', {})

#         # <<< NEW LOGIC: Populate title and subtitle in separate placeholders
#         if slide.shapes.title:
#             slide.shapes.title.text = content.get('title', '')
#             slide.shapes.title.text_frame.auto_size = MSO_AUTO_SIZE.
↳ TEXT_TO_FIT_SHAPE

```

```

#         subtitle_ph = next((p for p in slide.placeholders if p.
#         ↪placeholder_format.type == PP_PLACEHOLDER.SUBTITLE), None)
#         if subtitle_ph:
#             subtitle_ph.text = content.get('subtitle', '')
#             subtitle_ph.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

#         body_placeholders = sorted([p for p in slide.placeholders if p.
#         ↪placeholder_format.type not in [PP_PLACEHOLDER.TITLE, PP_PLACEHOLDER.
#         ↪SUBTITLE]], key=lambda p: p.left)
#         objects = content.get('objects', [])

#         for obj, placeholder in zip(objects, body_placeholders):
#             content_type, obj_data = obj.get('content_type'), obj.get('data')
#             renderers = {
#                 'bullet_points': lambda: _render_bullet_points(placeholder.
#                 ↪text_frame, obj_data),
#                 'table': lambda: _render_table(slide, placeholder, obj_data),
#                 'multiple_choice_question': lambda:
#                 ↪_render_multiple_choice_question(slide, body_placeholders, obj_data),
#                 'matching_activity': lambda: _render_matching_activity(slide,
#                 ↪body_placeholders, obj_data)
#             }
#             if content_type in renderers:
#                 renderers[content_type]()
#                 if content_type in ['multiple_choice_question',
#                 ↪'matching_activity']: break

import os
import json
import logging
import random
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.enum.shapes import PP_PLACEHOLDER_TYPE, MSO_SHAPE_TYPE
from pptx.enum.text import MSO_AUTO_SIZE

# --- Basic Setup ---
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
    ↪%(message)s')
logger = logging.getLogger(__name__)

# --- Helper function to find specific placeholders ---
def _get_placeholder(slide, ph_type):
    for shape in slide.placeholders:
        if shape.placeholder_format.type == ph_type:
            return shape

```

```

        return None

# --- Rendering Helper Functions ---
def _render_bullet_points(text_frame, data):
    text_frame.clear(); text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
    def add_points(points, level):
        for point in points:
            if isinstance(point, dict):
                p = text_frame.add_paragraph(); p.text = point.get('text', '');
                p.level = level
                if 'children' in point: add_points(point['children'], level + 1)
            else:
                p = text_frame.add_paragraph(); p.text = str(point); p.level =
                level
        if isinstance(data, list): add_points(data, 0)
        if text_frame.paragraphs and not text_frame.paragraphs[0].text:
            p = text_frame.paragraphs[0]; p._p.getparent().remove(p._p)

def _render_table(slide, placeholder, data):
    headers, rows_data = data.get('headers', []), data.get('rows', [])
    if not headers or not rows_data: return
    table_shape = slide.shapes.add_table(len(rows_data) + 1, len(headers),
    placeholder.left, placeholder.top, placeholder.width, placeholder.height)
    table = table_shape.table
    for i, header in enumerate(headers):
        cell = table.cell(0, i); cell.text = header; cell.text_frame.
        paragraphs[0].font.bold = True
    for r_idx, row_data in enumerate(rows_data):
        for c_idx, cell_text in enumerate(row_data):
            table.cell(r_idx + 1, c_idx).text = str(cell_text)
    sp = placeholder.element; sp.getparent().remove(sp)

def _render_mcq(placeholder, data, answer_placeholder=None):
    """
    Renders the MCQ question into the main placeholder and the detailed
    answer with explanation into the dedicated answer_placeholder.
    """
    # --- Part 1: Render the Question and Options ---
    tf = placeholder.text_frame
    tf.clear()
    tf.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

    # Add the question text
    p_question = tf.add_paragraph()
    p_question.text = data.get('question_text', '')
    p_question.font.bold = True

```

```

# Add the options
for option in data.get('options', []):
    p_option = tf.add_paragraph()
    p_option.text = f"{option.get('label', '')} {option.get('text', '')}"
    p_option.level = 1

# --- Part 2: Render the Detailed Answer and Explanation ---
if answer_placeholder:
    answer_info = data.get('correct_answer', {})
    answer_label = answer_info.get('label', 'N/A')
    answer_explanation = answer_info.get('explanation', 'No explanation_
provided.')
```

Get the text frame of the answer placeholder

```

answer_tf = answer_placeholder.text_frame
answer_tf.clear()
```

Add "Correct Answer: [Label]" in bold

```

p_answer_label = answer_tf.add_paragraph()
p_answer_label.text = f" Answer: {answer_label} - {answer_explanation}"
#p_answer_label.font.bold = True
```

Add the explanation text on the next line

```

# p_answer_explanation = answer_tf.add_paragraph()
# p_answer_explanation.text = answer_explanation
```

```

def _render_matching(placeholders, data):
    """Renders a matching activity into a pair of placeholders."""
    if len(placeholders) < 2: return
    terms_ph, defs_ph = placeholders[0], placeholders[1]
    terms_tf, defs_tf = terms_ph.text_frame, defs_ph.text_frame
    terms_tf.clear(); defs_tf.clear()
    pairs = data.get('pairs', [])
    if not pairs: return
    terms = [f"{i+1}. {p['term']}" for i, p in enumerate(pairs)]
    definitions = [p['definition'] for p in pairs]
    random.shuffle(definitions)
    for term in terms: terms_tf.add_paragraph().text = term
    for i, definition in enumerate(definitions): defs_tf.add_paragraph().text =
f"{chr(65+i)}. {definition}"

def _render_text_with_smart_fit(text_frame, data):
    """
    Renders text and proactively adjusts the font size based on content length
    to prevent overflow.
    """
```

```

text_frame.clear()
# We still set this as a fallback for moderate cases
text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

# --- Character Counting Logic ---
# Convert the data (which can be a list of strings or dicts) into a single
↳ string to measure its length.
json_string = json.dumps(data)
char_count = len(json_string)

# --- Define Font Size Tiers ---
# These values may need tuning based on your template's placeholder sizes
↳ and fonts.
# Format: (character_threshold, font_size_in_points)
font_tiers = [
    (1200, Pt(16)), # Very long text -> smallest font
    (750, Pt(18)), # Long text
    (450, Pt(20)), # Medium text
    (0, Pt(22)) # Short text -> default/largest font
]

selected_font_size = None
for threshold, size in font_tiers:
    if char_count >= threshold:
        selected_font_size = size
        break

# --- Recursive Function to Add Points and Apply Font Size ---
def add_points_with_font(points, level):
    for point in points:
        # Determine the text for the current point
        text_to_add = ""
        if isinstance(point, dict):
            text_to_add = point.get('text', '')
        else:
            text_to_add = str(point)

        # Add the paragraph and set its properties
        p = text_frame.add_paragraph()
        p.text = text_to_add
        p.level = level
        if selected_font_size:
            p.font.size = selected_font_size

        # Recurse for children if they exist
        if isinstance(point, dict) and 'children' in point:
            add_points_with_font(point['children'], level + 1)

```

```

# Start the rendering process
if isinstance(data, list):
    add_points_with_font(data, 0)

# Clean up the initial empty paragraph if it exists and is unused
if text_frame.paragraphs and not text_frame.paragraphs[0].text:
    p_to_remove = text_frame.paragraphs[0]
    p_to_remove._p.getparent().remove(p_to_remove._p)

class PresentationAgent:
    def __init__(self, template_path: str, layout_config_path: str):
        self.template_path = template_path
        self.prs_for_layouts = Presentation(template_path)
        with open(layout_config_path, 'r', encoding='utf-8') as f:
            self.user_selections = json.load(f)['user_selections']
        logger.info(f"Agent initialized with template '{os.path.
↳basename(template_path)}'")

    def _get_layout(self, layout_key: str):
        selection = self.user_selections.get(layout_key, self.
↳user_selections['Content'])
        return self.prs_for_layouts.
↳slide_layouts[selection['selected_layout_index']]

    def _collect_and_sort_slides(self, plan_data: dict) -> list:
        all_slide_items = []

        # This is the same recursive helper function, it is correct.
        def _recursive_harvester(node: dict, parent_title: str = None):
            is_child_node = bool(parent_title)
            for slide in node.get('slides', []):
                slide['parent_title'] = parent_title
                slide['is_child'] = is_child_node
                all_slide_items.append({'item_type': 'Content', 'data': slide})

            for child in node.get('children', []):
                # Pass the current node's title as the parent title for the
↳next level
                _recursive_harvester(child, parent_title=node.get('title'))

        # This part correctly harvests activities from ANY node (parent or
↳child)
        if 'interactive_activity' in node and 'llm_generated_content' in
↳node['interactive_activity']:

```

```

        activity = node['interactive_activity']
        activity['parent_title'] = parent_title
        # IMPORTANT: The 'is_child' status depends on whether a
        ↪parent_title was passed in.
        activity['is_child'] = is_child_node
        # We add the main node's title to the activity data so it can
        ↪be used if needed
        activity['main_topic_title'] = node.get('title')
        all_slide_items.append({'item_type': 'Content', 'data':
        ↪activity})

    # This is the main loop that processes the plan
    for deck in plan_data.get('deck_plans', []):
        # Framework slides are added as before
        framework_start = ([{'item_type': 'Title', 'data': s} for s in
        ↪deck['sections'] if s['section_type'] == 'Title'] +
                            [{'item_type': 'Agenda', 'data': s} for s in
        ↪deck['sections'] if s['section_type'] == 'Agenda'])
        all_slide_items.extend(framework_start)

        # --- MODIFICATION IS HERE ---
        # We now loop through the content blocks and call the harvester on
        ↪each one.
        content_sections = [s for s in deck['sections'] if
        ↪s['section_type'] == 'Content']
        if content_sections:
            for block in content_sections[0].get('content_blocks', []):
                # We start the recursion for each main content block
                # We pass the block's own title as the parent_title for its
                ↪direct activities
                _recursive_harvester(block, parent_title=None) # Start with
                ↪no parent

            framework_end = ([{'item_type': 'Summary', 'data': s} for s in
        ↪deck['sections'] if s['section_type'] == 'Summary'] +
                            [{'item_type': 'End', 'data': s} for s in
        ↪deck['sections'] if s['section_type'] == 'End'])
            all_slide_items.extend(framework_end)

    return sorted(all_slide_items, key=lambda x: x['data'].get('seq_id',
        ↪999))

def _determine_layout_key(self, slide_data: dict) -> str:
    # This logic is correct
    llm_content = slide_data.get('llm_generated_content', {})
    objects = llm_content.get('objects', [])

```

```

        if not objects: return "Content"
        first_obj_type = objects[0].get('content_type')
        if first_obj_type in ['multiple_choice_question', 'matching_activity']:
            return "Application_Two_Column " if first_obj_type == "
        ↪ 'matching_activity' else "Application"
        is_child = slide_data.get('is_child', False)
        if len(objects) >= 2:
            return "Content_Two_Column_child" if is_child else "
        ↪ "Content_Two_Column"
            return "Content_child" if is_child else "Content"

    def create_presentation_from_plan(self, plan_json_path: str, output_dir:
        ↪ str):
        # This orchestrator is correct
        logger.info(f"Creating presentation from plan: '{os.path.
        ↪ basename(plan_json_path)}'")
        with open(plan_json_path, 'r', encoding='utf-8') as f: plan_data = json.
        ↪ load(f)
        final_slide_list = self._collect_and_sort_slides(plan_data)
        prs = Presentation(self.template_path)
        while len(prs.slides):
            rId = prs.slides._sldIdLst[0].rId; prs.part.drop_rel(rId); del prs.
        ↪ slides._sldIdLst[0]
            logger.info(f"Collected a total of {len(final_slide_list)} slides to
        ↪ render.")
            for item in final_slide_list:
                self._add_slide_for_item(prs, item)
            unit_code = plan_data.get('deck_plans', [{}])[0].get('sections',
        ↪ [{}])[0].get('content', {}).get('unit_code', 'UNIT')
            week_num = plan_data.get('week', 'X')
            deck_num = plan_data.get('deck_plans', [{}])[0].get('deck_number', 'Y')
            output_filename =
        ↪ f"{unit_code}_Week{week_num}_Deck{deck_num}_Presentation.pptx"
            output_path = os.path.join(output_dir, output_filename)
            prs.save(output_path)
            logger.info(f"Successfully created presentation: '{output_path}'")

    def _add_slide_for_item(self, prs: Presentation, item: dict):
        item_type = item['item_type']
        item_data = item['data']
        layout_key = self._determine_layout_key(item_data) if item_type ==
        ↪ 'Content' else item_type
        slide = prs.slides.add_slide(self._get_layout(layout_key))
        render_map = {
            'Title': self._render_title_slide, 'Agenda': self.
        ↪ _render_agenda_slide,

```

```

        'Summary': self._render_summary_slide, 'End': self.
↪_render_end_slide,
        'Divider': self._render_divider_slide, 'Content': self.
↪_render_content_slide
    }
    if item_type in render_map: render_map[item_type](slide, item_data)

def _render_title_slide(self, slide, data):
    content = data.get('content', {})
    title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.CENTER_TITLE)
    if title_ph: title_ph.text = content.get('week_topic', '')
    subtitle_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.SUBTITLE)
    if subtitle_ph: subtitle_ph.text = f"{content.get('deck_title', '')}
↪{'')}\n{content.get('unit_name', '')} | {content.get('unit_code', '')}"

def _render_agenda_slide(self, slide, data):
    content = data.get('content', {})
    title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
    if title_ph: title_ph.text = content.get('title', 'Agenda')
    body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
    if body_ph: _render_bullet_points(body_ph.text_frame, content.
↪get('items', []))

def _render_summary_slide(self, slide, data):
    """Directly renders the summary slide content without calling another
↪function."""
    title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
    if title_ph:
        title_ph.text = "Summary & Key Takeaways"

    body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
    if not body_ph:
        logger.warning("Summary slide layout is missing a main body/object
↪placeholder.")
        return

    # The summary data is nested in the 'llm_generated_content' key
    content = data.get('llm_generated_content', {})
    if content and 'objects' in content and content['objects']:
        # The summary is always bullet points, so we can call the helper
↪directly.
        summary_data = content['objects'][0].get('data', [])
        _render_bullet_points(body_ph.text_frame, summary_data)

def _render_divider_slide(self, slide, data):
    content = data.get('content', {})

```

```

        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = content.get('title', '')

    def _render_end_slide(self, slide, data):
        content = data.get('content', {})
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = content.get('text', 'Thank You & Questions?')
        ↪')

    def _render_summary_slide(self, slide, data):
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = "Summary & Key Takeaways"
        body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
        if body_ph and 'llm_generated_content' in data:
            summary_data = data['llm_generated_content'].get('objects', [])
            ↪[{}])[0].get('data', [])
            _render_bullet_points(body_ph.text_frame, summary_data)

# <<< THIS IS THE CORRECTED METHOD >>>
    def _render_content_slide(self, slide, data):
        content = data.get('llm_generated_content', {})
        if not content: return

        # --- Step 1: Determine Layout and Content ---
        layout_key = self._determine_layout_key(data)
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        subtitle_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.SUBTITLE)

        # --- Step 2: Handle Title and Subtitle Rendering ---
        # This logic now correctly covers all slide types.
        if "Application" in layout_key:
            # For ANY interactive slide, the main title is the specific topic.
            if title_ph:
                # The subtitle from the JSON is the most descriptive title.
                title_ph.text = content.get('subtitle', 'Knowledge Check')
            # The subtitle is the generic call to action.
            if subtitle_ph:
                subtitle_ph.text = content.get('title', "Let's Apply This!")

        elif data.get('is_child'):
            # For a standard child slide, use the parent-child hierarchy.
            if title_ph: title_ph.text = data.get('parent_title', '')
            if subtitle_ph: subtitle_ph.text = content.get('title', '')

        else:
            # For a standard parent slide.
            if title_ph: title_ph.text = content.get('title', '')

```

```

        if subtitle_ph: subtitle_ph.text = content.get('subtitle', '')

    # --- Step 3: Render Body Objects ---
    # Find the main placeholder for the question and the dedicated
    ↪placeholder for the answer.
    body_placeholders = sorted([p for p in slide.placeholders if p.
    ↪placeholder_format.type == PP_PLACEHOLDER_TYPE.OBJECT], key=lambda p: p.left)
    answer_placeholder = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.BODY)
    objects = content.get('objects', [])

    for i, obj in enumerate(objects):
        if i >= len(body_placeholders): break
        placeholder = body_placeholders[i]
        content_type, obj_data = obj.get('content_type'), obj.get('data')

        # THIS IS THE CHANGE:
        if content_type in ['bullet_points', 'description', 'explanation',
    ↪'timeline', 'process', 'cycle', 'hierarchy', 'case_study', 'example']:
            _render_text_with_smart_fit(placeholder.text_frame, obj_data) #
    ↪Use the new function
        elif content_type == 'table':
            _render_table(slide, placeholder, obj_data)
        elif content_type == 'multiple_choice_question':
            _render_mcq(placeholder, obj_data, answer_placeholder)
            # The notes slide can still contain the detailed answer for
    ↪reference.
            if slide.has_notes_slide:
                answer_info = obj_data.get('correct_answer', {})
                slide.notes_slide.notes_text_frame.text = f"Answer:
    ↪{answer_info.get('label', '')}. {answer_info.get('explanation', '')}"

        elif content_type == 'matching_activity':
            _render_matching(body_placeholders, obj_data)
            break # Matching uses all placeholders.

        else: # All other content types
            _render_bullet_points(placeholder.text_frame, obj_data)

# # --- Main Execution Block --- No delete no change it is ready for test the
    ↪outputfrom ContentAgent
if __name__ == "__main__":

```

```

TEST_CONTENT_PATH = "/home/sebas_dev_linux/projects/course_generator/
↳data_output/test_Output/4. generated_content_llm/ICT312/
↳ICT312_Week1_plan_final_llm_generated.json"

print("--- Presentation Agent Test Runner (Final Version) ---")

# Check for required files
required_files = [SLIDE_TEMPLATE_PATH, LAYOUT_MAPPING_PATH,
↳TEST_CONTENT_PATH]
if not all(os.path.exists(p) for p in required_files):
    logger.error("CRITICAL: One or more required files not found. Please
↳check paths:")
    logger.error(f"  Template: {SLIDE_TEMPLATE_PATH} {'(Found)' if os.path.
↳exists(SLIDE_TEMPLATE_PATH) else '(MISSING)'}")
    logger.error(f"  Layout Map: {LAYOUT_MAPPING_PATH} {'(Found)' if os.
↳path.exists(LAYOUT_MAPPING_PATH) else '(MISSING)'}")
    logger.error(f"  Content Plan: {TEST_CONTENT_PATH} {'(Found)' if os.
↳path.exists(TEST_CONTENT_PATH) else '(MISSING)'}")
else:
    try:
        presentation_agent = PresentationAgent(
            template_path=SLIDE_TEMPLATE_PATH,
            layout_config_path=LAYOUT_MAPPING_PATH
        )

        # <<< THIS IS THE CORRECTED LINE >>>
        # We pass the DIRECTORY where the output should be saved.
        presentation_agent.create_presentation_from_plan(
            plan_json_path=TEST_CONTENT_PATH,
            output_dir=FINAL_PRESENTATION_DIR
        )

        logger.info("--- Test script finished successfully. ---")
        logger.info(f"  Check for the generated .pptx file in:
↳{FINAL_PRESENTATION_DIR}")

    except Exception as e:
        logger.error(f"  An unexpected error occurred during the test run:
↳{e}", exc_info=True)

```

```

2025-07-19 00:33:44,773 - INFO - Agent initialized with template
'slide_style_test_2.pptx'
2025-07-19 00:33:44,773 - INFO - Creating presentation from plan:
'ICT312_Week1_plan_final_llm_generated.json'
2025-07-19 00:33:44,784 - INFO - Collected a total of 43 slides to render.

--- Presentation Agent Test Runner (Final Version) ---

```

```

2025-07-19 00:33:45,044 - INFO - Successfully created presentation:
'/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/5.
final_presentations/ICT312/ICT312_Week1_Deck1_Presentation.pptx'
2025-07-19 00:33:45,045 - INFO - --- Test script finished successfully. ---
2025-07-19 00:33:45,045 - INFO - Check for the generated .pptx file in:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/5.
final_presentations/ICT312
2025-07-19 00:33:45,045 - INFO - --- Test script finished successfully. ---
2025-07-19 00:33:45,045 - INFO - Check for the generated .pptx file in:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/5.
final_presentations/ICT312

```

7.4 Orquestrator (Addressing pain points)

Description:

The main script that iterates through the weeks defined the plan and generate the content base on the settings_deck coordinating the agents.

Parameters and concideration - 1 hour in the setting session_time_duration_in_hour - is 18-20 slides at the time so it is require to calculate this according to the given value but this also means per session so sessions_per_week is a multiplicator factor that

- if apply_topic_interactive is available will add an extra slide and add extra 5 min time but to determine this is required to plan all the content first and then calculate then provide a extra time

settings_deck.json

```

{ "course_id":  "", "unit_name":  "", "interactive":  true, "interactive_deep":  false, "teaching_flow_id":  "Standard Lecture Flow", "parameters_slides":  { "slides_per_hour": 18, "time_per_content_slides_min": 3, "time_per_interactive_slide_min": 5, "time_for_framework_slides_min": 6 }, "week_session_setup": { "sessions_per_week": 1, "distribution_strategy": "even", "session_time_duration_in_hour": 2, "interactive_time_in_hour": 0, "total_session_time_in_hours": 0 }, "slide_count_strategy": { "method": "per_week", "target_total_slides": 0, "slides_content_per_session": 0, "interactive_slides_per_week": 0, "interactive_slides_per_session": 0, "total_slides_deck_week": 0, "total_slides_session": 0 }, "generation_scope": { "weeks": [1] } }

```

teaching_flows.json

```

{ "standard_lecture": { "name": "Standard Lecture Flow", "slide_types": ["Title", "Agenda", "Content", "Summary", "End"], "prompts": { "content_generation": "You are an expert university lecturer. Your audience is undergraduate students. Based on the following context, create a slide that provides a detailed explanation of the topic '{sub_topic}'. The content should be structured with bullet points for key details. Your output MUST be a single JSON object with a 'title' (string) and 'content' (list of strings) key.", "summary_generation": "You are an expert university lecturer creating a summary slide. Based on the following list of topics covered in this session, generate a concise summary of the key takeaways. The topics are: {topic_list}. Your output MUST be a single JSON object with a 'title' (string) and 'content' (list of strings) key." }, "slide_schemas": { "Content": { "title": "string", "content": "list[string]" }, "Summary": { "title": "string", "content": "list[string]" } }, "apply_topic_interactive": { "name": "Interactive

```

Lecture Flow”, “slide_types”: [“Title”, “Agenda”, “Content”, “Application”, “Summary”, “End”], “prompts”: { “content_generation”: “You are an expert university lecturer in Digital Forensics. Your audience is undergraduate students. Based on the provided context, create a slide explaining the concept of ‘{sub_topic}’. The content should be clear, concise, and structured with bullet points for easy understanding. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.”, “application_generation”: “You are an engaging university lecturer creating an interactive slide. Based on the concept of ‘{sub_topic}’, create a multiple-choice question with exactly 4 options (A, B, C, D) to test understanding. The slide title must be ‘Let’s Apply This:’. Clearly indicate the correct answer within the content. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.”, “summary_generation”: “You are an expert university lecturer creating a summary slide. Based on the following list of concepts and applications covered in this session, generate a concise summary of the key takeaways. The topics are: {topic_list}. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.” }, “slide_schemas”: { “Content”: {“title”: “string”, “content”: “list[string]”}, “Application”: {“title”: “string”, “content”: “list[string]”}, “Summary”: {“title”: “string”, “content”: “list[string]”} } } }

7.4.1 Main Integration

```
[31]: TEST_MODE_ENABLED = False

# Cell 11: --- Main Orchestration Block (with Phase 5 & 6) ---
print_header("Main Orchestrator Initialized", char="*")

try:
    # 1. Connect to DB
    vector_store = Chroma(
        persist_directory=CHROMA_PERSIST_DIR,
        embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
        collection_name=CHROMA_COLLECTION_NAME
    )
    logger.info("Database connection successful.")

    # Phase 1: Configuration and Scoping
    master_config = process_and_load_configurations()

    if master_config:
        all_final_plans = []

        # Phase 2 & 3: Create and Finalize Draft Plans
        print_header("Phase 2 & 3: Generating and Finalizing Weekly Plans ",
            char="-")
        planning_agent = PlanningAgent(master_config, vector_store=vector_store)

        weeks_to_generate =
            master_config['processed_settings']['generation_scope']['weeks']
```

```

        logger.info(f"Found {len(weeks_to_generate)} week(s) to plan:␣
↳{weeks_to_generate}")

        if TEST_MODE_ENABLED:
            logger.info("--- TEST MODE ACTIVE: Processing only the first week␣
↳in scope. ---")
            weeks_to_generate = weeks_to_generate[:1]

        for week in weeks_to_generate:
            draft_plan = planning_agent.create_content_plan_for_week(week)
            if draft_plan:
                final_plan = planning_agent.
↳finalize_and_calculate_time_plan(draft_plan,␣
↳master_config['processed_settings'])
                all_final_plans.append(final_plan)

                # Save both draft and final for comparison
                draft_filename = f"{master_config['processed_settings'].
↳get('course_id')}_Week{week}_plan_draft.json"
                final_filename = f"{master_config['processed_settings'].
↳get('course_id')}_Week{week}_plan_final.json"

                with open(os.path.join(PPLAN_OUTPUT_DIR, draft_filename), 'w')␣
↳as f:
                    json.dump(draft_plan, f, indent=2)

                with open(os.path.join(PPLAN_OUTPUT_DIR, final_filename), 'w')␣
↳as f:
                    json.dump(final_plan, f, indent=2)

                logger.info(f" Successfully saved FINAL plan for Week {week}␣
↳to: {os.path.join(PPLAN_OUTPUT_DIR, final_filename)}")

            else:
                logger.error(f" Failed to generate draft plan for Week {week}.")

        # Phase 4: Generate Master Summary Plan
        master_plan_generated = False
        if all_final_plans:
            print_header(" Phase 4: Generating Master Unit Plan ", char="--")
            master_plan_generated = planning_agent.
↳generate_and_save_master_plan(all_final_plans,␣
↳master_config['processed_settings'])
            master_plan_generated = True
        else:

```

```

        logger.warning("No weekly plans were generated, skipping master_
↪plan creation.")

    # Initialize ContentAgent once for subsequent phases
    content_agent = ContentAgent(master_config, vector_store=vector_store)

    phase_5_successful = False

    # Phase 5: Fetching Raw Content
    if master_plan_generated:
        print_header("Phase 5: Populating Plans with Raw Content", char="#")
        successful_weeks_phase5 = []
        for week in weeks_to_generate:
            final_filename = f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final.json"
            full_plan_path = os.path.join(PLAN_OUTPUT_DIR, final_filename)

            if os.path.exists(full_plan_path):
                if content_agent.generate_content_for_plan(full_plan_path,
↪CONTENT_OUTPUT_DIR):
                    successful_weeks_phase5.append(week)
            else:
                logger.warning(f" Skipping content population for Week_
↪{week} as its plan file was not found.")

        if successful_weeks_phase5:
            phase_5_successful = True
            logger.info(f" Phase 5 completed for weeks:
↪{successful_weeks_phase5}")

    ## Phase 6: Generating Slide Content with LLM
    phase_6_successful = False
    if phase_5_successful:
        print_header(" Phase 6: Generating Slide Content with LLM",
↪char="#")
        successful_weeks_phase6 = []
        for week in weeks_to_generate:
            # The input for phase 6 is the output of phase 5
            content_enriched_filename =
↪f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final.json"
            content_plan_path = os.path.join(CONTENT_OUTPUT_DIR,
↪content_enriched_filename)

            if os.path.exists(content_plan_path):

```

```

        if content_agent.
↳generate_llm_content_for_plan(content_plan_path, CONTENT_LLM_OUTPUT_DIR):
            successful_weeks_phase6.append(week)
        else:
            logger.warning(f"Skipping LLM generation for Week {week} as
↳its content-enriched file was not found.")

        if successful_weeks_phase6:
            phase_6_successful = True
            logger.info(f"Phase 6 completed for weeks:
↳{successful_weeks_phase6}")

        # Phase 7 Phase 7: Generating Final PowerPoint Files
        #LAYOUT_MAPPING_PATH = os.path.join(CONFIG_DIR,
↳"layout_mapping_test_Mod.json") # Seted up

        if phase_6_successful:
            print_header("Phase 7: Generating Final PowerPoint Files",
↳char="#")
            presentation_agent =
↳PresentationAgent(template_path=SLIDE_TEMPLATE_PATH, layout_config_path =
↳LAYOUT_MAPPING_PATH)

            for week in weeks_to_generate:
                llm_plan_filename = f"{master_config['processed_settings'].
↳get('course_id')}_Week{week}_plan_final_llm_generated.json"
                llm_plan_path = os.path.join(CONTENT_LLM_OUTPUT_DIR,
↳llm_plan_filename)

                if os.path.exists(llm_plan_path):
                    presentation_agent.
↳create_presentation_from_plan(llm_plan_path, FINAL_PRESENTATION_DIR)
                else:
                    logger.warning(f"Skipping presentation generation for Week
↳{week} as its LLM-enriched plan was not found.")

                    logger.info(f"Phase 7 completed ")
            else:
                logger.warning("Skipping Phase 7 because prior phases failed or
↳were skipped.")

except Exception as e:

```

```
logger.error(f"An unexpected error occurred during the main orchestration:␣  
↪{e}", exc_info=True)
```

```
2025-07-18 23:35:03,851 - INFO - Database connection successful.  
2025-07-18 23:35:03,852 - INFO - Loading all necessary configuration and data  
files...  
2025-07-18 23:35:03,855 - INFO - All files loaded successfully.  
2025-07-18 23:35:03,855 - INFO - Pre-processing settings_deck for definitive  
plan...  
2025-07-18 23:35:03,855 - INFO - Applying overrides if specified...  
2025-07-18 23:35:03,856 - INFO - Loaded from settings: 'interactive' is True.  
Set teaching_flow_id to 'apply_topic_interactive'.  
2025-07-18 23:35:03,857 - INFO - Calculating preliminary slide budget based on  
session time...  
2025-07-18 23:35:03,857 - INFO - Duration Hours 2.  
2025-07-18 23:35:03,857 - INFO - Sessions per week 1.  
2025-07-18 23:35:03,858 - INFO - Preliminary weekly content slide target  
calculated: #32 slides.  
2025-07-18 23:35:03,858 - INFO - Saving preliminary processed configuration to:  
/home/sebas_dev_linux/projects/course_generator/configs/processed_settings.json  
2025-07-18 23:35:03,860 - INFO - File saved successfully.  
2025-07-18 23:35:03,860 - INFO - Master configuration object is ready for the  
Planning Agent.  
2025-07-18 23:35:03,861 - INFO - Data-Driven PlanningAgent initialized  
successfully.  
2025-07-18 23:35:03,862 - INFO - Found 12 week(s) to plan: [1, 2, 3, 4, 5, 6, 7,  
8, 9, 10, 11, 12]  
  
2025-07-18 23:35:03,882 - INFO - Partitioning strategy: Distributing 7 top-level  
sections across 1 decks.  
2025-07-18 23:35:03,883 - INFO - --- Planning Deck 1/1 | Topics: ['An Overview  
of Digital Forensics', 'Preparing for Digital Investigations', 'Maintaining  
Professional Conduct', 'Preparing a Digital Forensics Investigation',  
'Procedures for Private-Sector High-Tech Investigations', 'Understanding Data  
Recovery Workstations and Software', 'Conducting an Investigation'] | Weight: 0  
chunks | Slide Budget: 0 ---  
2025-07-18 23:35:03,886 - INFO - Successfully saved FINAL plan for Week 1 to:  
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.  
generated_plans/ICT312/ICT312_Week1_plan_final.json  
2025-07-18 23:35:03,902 - INFO - Partitioning strategy: Distributing 4 top-level  
sections across 1 decks.  
2025-07-18 23:35:03,902 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding  
Forensics Lab Accreditation Requirements', 'Determining the Physical  
Requirements for a Digital Forensics Lab', 'Selecting a Basic Forensic  
Workstation', 'Building a Business Case for Developing a Forensics Lab'] |  
Weight: 0 chunks | Slide Budget: 0 ---  
2025-07-18 23:35:03,904 - INFO - Successfully saved FINAL plan for Week 2 to:  
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
```

generated_plans/ICT312/ICT312_Week2_plan_final.json
2025-07-18 23:35:03,917 - INFO - Partitioning strategy: Distributing 8 top-level sections across 1 decks.
2025-07-18 23:35:03,918 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding Storage Formats for Digital Evidence', 'Determining the Best Acquisition Method', 'Contingency Planning for Image Acquisitions', 'Using Acquisition Tools', 'Validating Data Acquisitions', 'Performing RAID Data Acquisitions', 'Using Remote Network Acquisition Tools', 'Using Other Forensics Acquisition Tools'] | Weight: 0 chunks | Slide Budget: 0 ---
2025-07-18 23:35:03,919 - INFO - Successfully saved FINAL plan for Week 3 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week3_plan_final.json
2025-07-18 23:35:03,930 - INFO - Partitioning strategy: Distributing 8 top-level sections across 1 decks.
2025-07-18 23:35:03,930 - INFO - --- Planning Deck 1/1 | Topics: ['Identifying Digital Evidence', 'Collecting Evidence in Private-Sector Incident Scenes', 'Processing Law Enforcement Crime Scenes', 'Preparing for a Search', 'Securing a Digital Incident or Crime Scene', 'Seizing Digital Evidence at the Scene', 'Storing Digital Evidence', 'Obtaining a Digital Hash'] | Weight: 0 chunks | Slide Budget: 0 ---
2025-07-18 23:35:03,932 - INFO - Successfully saved FINAL plan for Week 4 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week4_plan_final.json
2025-07-18 23:35:03,950 - INFO - Partitioning strategy: Distributing 7 top-level sections across 1 decks.
2025-07-18 23:35:03,950 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding File Systems', 'Exploring Microsoft File Structures', 'Examining NTFS Disks', 'Understanding Whole Disk Encryption', 'Understanding the Windows Registry', 'Understanding Microsoft Startup Tasks', 'Understanding Virtual Machines'] | Weight: 0 chunks | Slide Budget: 0 ---
2025-07-18 23:35:03,953 - INFO - Successfully saved FINAL plan for Week 5 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week5_plan_final.json
2025-07-18 23:35:03,966 - INFO - Partitioning strategy: Distributing 4 top-level sections across 1 decks.
2025-07-18 23:35:03,966 - INFO - --- Planning Deck 1/1 | Topics: ['Evaluating Digital Forensics Tool Needs', 'Digital Forensics Software Tools', 'Digital Forensics Hardware Tools', 'Validating and Testing Forensics Software'] | Weight: 0 chunks | Slide Budget: 0 ---
2025-07-18 23:35:03,968 - INFO - Successfully saved FINAL plan for Week 6 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week6_plan_final.json
2025-07-18 23:35:03,983 - INFO - Partitioning strategy: Distributing 7 top-level sections across 1 decks.
2025-07-18 23:35:03,983 - INFO - --- Planning Deck 1/1 | Topics: ['Examining Linux File Structures', 'Understanding Macintosh File Structures', 'Using Linux Forensics Tools', 'Recognizing a Graphics File', 'Understanding Data Compression', 'Identifying Unknown File Formats', 'Understanding Copyright

Issues with Graphics'] | Weight: 0 chunks | Slide Budget: 0 ---
2025-07-18 23:35:03,986 - INFO - Successfully saved FINAL plan for Week 7 to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week7_plan_final.json
2025-07-18 23:35:04,003 - INFO - Partitioning strategy: Distributing 6 top-level
sections across 1 decks.
2025-07-18 23:35:04,004 - INFO - --- Planning Deck 1/1 | Topics: ['Determining
What Data to Collect and Analyze', 'Validating Forensic Data', 'Addressing Data-
Hiding Techniques', 'An Overview of Virtual Machine Forensics', 'Performing Live
Acquisitions', 'Network Forensics Overview'] | Weight: 0 chunks | Slide Budget:
0 ---
2025-07-18 23:35:04,006 - INFO - Successfully saved FINAL plan for Week 8 to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week8_plan_final.json
2025-07-18 23:35:04,015 - INFO - Partitioning strategy: Distributing 6 top-level
sections across 1 decks.
2025-07-18 23:35:04,015 - INFO - --- Planning Deck 1/1 | Topics: ['Exploring the
Role of E-mail in Investigations', 'Exploring the Roles of the Client and Server
in E-mail', 'Investigating E-mail Crimes and Violations', 'Understanding E-mail
Servers', 'Using Specialized E-mail Forensics Tools', 'Applying Digital
Forensics Methods to Social Media Communications'] | Weight: 0 chunks | Slide
Budget: 0 ---
2025-07-18 23:35:04,017 - INFO - Successfully saved FINAL plan for Week 9 to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week9_plan_final.json
2025-07-18 23:35:04,036 - INFO - Partitioning strategy: Distributing 9 top-level
sections across 1 decks.
2025-07-18 23:35:04,036 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
Mobile Device Forensics', 'Understanding Acquisition Procedures for Mobile
Devices', 'Understanding Forensics in the Internet of Anything', 'An Overview of
Cloud Computing', 'Legal Challenges in Cloud Forensics', 'Technical Challenges
in Cloud Forensics', 'Acquisitions in the Cloud', 'Conducting a Cloud
Investigation', 'Tools for Cloud Forensics'] | Weight: 0 chunks | Slide Budget:
0 ---
2025-07-18 23:35:04,038 - INFO - Successfully saved FINAL plan for Week 10 to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week10_plan_final.json

Main Orchestrator Initialized

Phase 1: Configuration and Scoping Process

Phase 1 Configuration Complete

Phase 2 & 3: Generating and Finalizing Weekly Plans

Planning Week 1

Planning Week 2

Planning Week 3

Planning Week 4

Planning Week 5

Planning Week 6

Planning Week 7

Planning Week 8

Planning Week 9

Planning Week 10

Planning Week 11

2025-07-18 23:35:04,056 - INFO - Partitioning strategy: Distributing 7 top-level sections across 1 decks.

2025-07-18 23:35:04,057 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding the Importance of Reports', 'Guidelines for Writing Reports', 'Generating Report Findings with Forensics Software Tools', 'Preparing for Testimony', 'Testifying in Court', 'Preparing for a Deposition or Hearing', 'Preparing Forensics Evidence for Testimony'] | Weight: 0 chunks | Slide Budget: 0 ---

2025-07-18 23:35:04,058 - INFO - Successfully saved FINAL plan for Week 11 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.generated_plans/ICT312/ICT312_Week11_plan_final.json

2025-07-18 23:35:04,069 - INFO - Partitioning strategy: Distributing 4 top-level sections across 1 decks.

2025-07-18 23:35:04,070 - INFO - --- Planning Deck 1/1 | Topics: ['Applying Ethics and Codes to Expert Witnesses', 'Organizations with Codes of Ethics', 'Ethical Difficulties in Expert Testimony', 'An Ethics Exercise'] | Weight: 0 chunks | Slide Budget: 0 ---

2025-07-18 23:35:04,071 - INFO - Successfully saved FINAL plan for Week 12 to: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.generated_plans/ICT312/ICT312_Week12_plan_final.json

2025-07-18 23:35:04,072 - INFO - Successfully generated and saved Master Unit Plan to:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.generated_plans/ICT312/ICT312_master_plan_unit.json

2025-07-18 23:35:04,076 - INFO - Data-Driven Content Agent initialized with model 'qwen3:8b'.

2025-07-18 23:35:04,077 - INFO - Data-Driven Content Agent initialized successfully.

2025-07-18 23:35:04,077 - INFO - PHASE 5: Populating raw content for: /home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.generated_plans/ICT312/ICT312_Week1_plan_final.json

2025-07-18 23:35:04,078 - WARNING - No chunks found in the database for toc_id = 9

2025-07-18 23:35:04,079 - WARNING - No chunks found in the database for toc_id = 10

2025-07-18 23:35:04,080 - WARNING - No chunks found in the database for toc_id = 11

2025-07-18 23:35:04,081 - WARNING - No chunks found in the database for toc_id = 12

2025-07-18 23:35:04,082 - WARNING - No chunks found in the database for toc_id = 13

2025-07-18 23:35:04,083 - WARNING - No chunks found in the database for toc_id = 14

2025-07-18 23:35:04,084 - WARNING - No chunks found in the database for toc_id = 15

2025-07-18 23:35:04,084 - WARNING - No chunks found in the database for toc_id =

16
2025-07-18 23:35:04,085 - WARNING - No chunks found in the database for toc_id =
17
2025-07-18 23:35:04,086 - WARNING - No chunks found in the database for toc_id =
18
2025-07-18 23:35:04,087 - WARNING - No chunks found in the database for toc_id =
19
2025-07-18 23:35:04,088 - WARNING - No chunks found in the database for toc_id =
20
2025-07-18 23:35:04,089 - WARNING - No chunks found in the database for toc_id =
21
2025-07-18 23:35:04,090 - WARNING - No chunks found in the database for toc_id =
22
2025-07-18 23:35:04,090 - WARNING - No chunks found in the database for toc_id =
23
2025-07-18 23:35:04,091 - WARNING - No chunks found in the database for toc_id =
24
2025-07-18 23:35:04,092 - WARNING - No chunks found in the database for toc_id =
25
2025-07-18 23:35:04,093 - WARNING - No chunks found in the database for toc_id =
26
2025-07-18 23:35:04,094 - WARNING - No chunks found in the database for toc_id =
27
2025-07-18 23:35:04,094 - WARNING - No chunks found in the database for toc_id =
28
2025-07-18 23:35:04,095 - WARNING - No chunks found in the database for toc_id =
29
2025-07-18 23:35:04,096 - WARNING - No chunks found in the database for toc_id =
30
2025-07-18 23:35:04,097 - WARNING - No chunks found in the database for toc_id =
31
2025-07-18 23:35:04,098 - WARNING - No chunks found in the database for toc_id =
32
2025-07-18 23:35:04,098 - WARNING - No chunks found in the database for toc_id =
33
2025-07-18 23:35:04,099 - WARNING - No chunks found in the database for toc_id =
34
2025-07-18 23:35:04,100 - WARNING - No chunks found in the database for toc_id =
35
2025-07-18 23:35:04,101 - WARNING - No chunks found in the database for toc_id =
36
2025-07-18 23:35:04,102 - WARNING - No chunks found in the database for toc_id =
37
2025-07-18 23:35:04,103 - WARNING - No chunks found in the database for toc_id =
38
2025-07-18 23:35:04,103 - WARNING - No chunks found in the database for toc_id =
39
2025-07-18 23:35:04,104 - WARNING - No chunks found in the database for toc_id =

40
2025-07-18 23:35:04,105 - WARNING - No chunks found in the database for toc_id =
41
2025-07-18 23:35:04,106 - WARNING - No chunks found in the database for toc_id =
42
2025-07-18 23:35:04,106 - WARNING - No chunks found in the database for toc_id =
43
2025-07-18 23:35:04,107 - WARNING - No chunks found in the database for toc_id =
44
2025-07-18 23:35:04,109 - WARNING - No chunks found in the database for toc_id =
45
2025-07-18 23:35:04,109 - WARNING - No chunks found in the database for toc_id =
46
2025-07-18 23:35:04,110 - WARNING - No chunks found in the database for toc_id =
47
2025-07-18 23:35:04,111 - WARNING - No chunks found in the database for toc_id =
48
2025-07-18 23:35:04,112 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,114 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week1_plan_final.json
2025-07-18 23:35:04,115 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week2_plan_final.json
2025-07-18 23:35:04,116 - WARNING - No chunks found in the database for toc_id =
57
2025-07-18 23:35:04,117 - WARNING - No chunks found in the database for toc_id =
58
2025-07-18 23:35:04,117 - WARNING - No chunks found in the database for toc_id =
59
2025-07-18 23:35:04,118 - WARNING - No chunks found in the database for toc_id =
60
2025-07-18 23:35:04,119 - WARNING - No chunks found in the database for toc_id =
61
2025-07-18 23:35:04,120 - WARNING - No chunks found in the database for toc_id =
62
2025-07-18 23:35:04,121 - WARNING - No chunks found in the database for toc_id =
63
2025-07-18 23:35:04,122 - WARNING - No chunks found in the database for toc_id =
64
2025-07-18 23:35:04,123 - WARNING - No chunks found in the database for toc_id =
65
2025-07-18 23:35:04,123 - WARNING - No chunks found in the database for toc_id =
66
2025-07-18 23:35:04,124 - WARNING - No chunks found in the database for toc_id =
67
2025-07-18 23:35:04,125 - WARNING - No chunks found in the database for toc_id =
68

2025-07-18 23:35:04,126 - WARNING - No chunks found in the database for toc_id = 69
2025-07-18 23:35:04,127 - WARNING - No chunks found in the database for toc_id = 70
2025-07-18 23:35:04,128 - WARNING - No chunks found in the database for toc_id = 71
2025-07-18 23:35:04,129 - WARNING - No chunks found in the database for toc_id = 72
2025-07-18 23:35:04,130 - WARNING - No chunks found in the database for toc_id = 73
2025-07-18 23:35:04,131 - WARNING - No chunks found in the database for toc_id = 74
2025-07-18 23:35:04,131 - WARNING - No chunks found in the database for toc_id = 75
2025-07-18 23:35:04,132 - WARNING - No chunks found in the database for toc_id = 76
2025-07-18 23:35:04,133 - WARNING - No chunks found in the database for toc_id = 77
2025-07-18 23:35:04,134 - WARNING - No chunks found in the database for toc_id = 78
2025-07-18 23:35:04,135 - WARNING - No chunks found in the database for toc_id = 79
2025-07-18 23:35:04,136 - WARNING - No chunks found in the database for toc_id = 80
2025-07-18 23:35:04,137 - WARNING - No chunks found in the database for toc_id = 81
2025-07-18 23:35:04,138 - WARNING - No chunks found in the database for toc_id = 82
2025-07-18 23:35:04,139 - WARNING - No chunks found in the database for toc_id = 83
2025-07-18 23:35:04,140 - WARNING - No chunks found in the database for toc_id = 84
2025-07-18 23:35:04,141 - WARNING - No chunks found in the database for toc_id = 85
2025-07-18 23:35:04,142 - WARNING - No chunks found in the database for toc_id = 86
2025-07-18 23:35:04,142 - WARNING - No chunks found in the database for toc_id = 87
2025-07-18 23:35:04,143 - WARNING - No chunks found in the database for toc_id = 88
2025-07-18 23:35:04,144 - WARNING - No chunks found in the database for toc_id = 89
2025-07-18 23:35:04,145 - WARNING - No chunks found in the database for toc_id = 90
2025-07-18 23:35:04,146 - WARNING - No chunks found in the database for toc_id = 91
2025-07-18 23:35:04,147 - WARNING - No chunks found in the database for toc_id = 92

2025-07-18 23:35:04,149 - WARNING - No chunks found in the database for toc_id = 93
2025-07-18 23:35:04,150 - WARNING - No chunks found in the database for toc_id = 94
2025-07-18 23:35:04,150 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,152 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week2_plan_final.json
2025-07-18 23:35:04,152 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week3_plan_final.json
2025-07-18 23:35:04,153 - WARNING - No chunks found in the database for toc_id = 103
2025-07-18 23:35:04,154 - WARNING - No chunks found in the database for toc_id = 104
2025-07-18 23:35:04,154 - WARNING - No chunks found in the database for toc_id = 105
2025-07-18 23:35:04,155 - WARNING - No chunks found in the database for toc_id = 106
2025-07-18 23:35:04,156 - WARNING - No chunks found in the database for toc_id = 107
2025-07-18 23:35:04,157 - WARNING - No chunks found in the database for toc_id = 108
2025-07-18 23:35:04,158 - WARNING - No chunks found in the database for toc_id = 109
2025-07-18 23:35:04,159 - WARNING - No chunks found in the database for toc_id = 110
2025-07-18 23:35:04,160 - WARNING - No chunks found in the database for toc_id = 111
2025-07-18 23:35:04,161 - WARNING - No chunks found in the database for toc_id = 112
2025-07-18 23:35:04,161 - WARNING - No chunks found in the database for toc_id = 113
2025-07-18 23:35:04,162 - WARNING - No chunks found in the database for toc_id = 114
2025-07-18 23:35:04,163 - WARNING - No chunks found in the database for toc_id = 115
2025-07-18 23:35:04,164 - WARNING - No chunks found in the database for toc_id = 116
2025-07-18 23:35:04,165 - WARNING - No chunks found in the database for toc_id = 117
2025-07-18 23:35:04,166 - WARNING - No chunks found in the database for toc_id = 118
2025-07-18 23:35:04,167 - WARNING - No chunks found in the database for toc_id = 119
2025-07-18 23:35:04,167 - WARNING - No chunks found in the database for toc_id = 120
2025-07-18 23:35:04,168 - WARNING - No chunks found in the database for toc_id =

121
2025-07-18 23:35:04,169 - WARNING - No chunks found in the database for toc_id =
122
2025-07-18 23:35:04,170 - WARNING - No chunks found in the database for toc_id =
123
2025-07-18 23:35:04,171 - WARNING - No chunks found in the database for toc_id =
124
2025-07-18 23:35:04,172 - WARNING - No chunks found in the database for toc_id =
125
2025-07-18 23:35:04,172 - WARNING - No chunks found in the database for toc_id =
126
2025-07-18 23:35:04,174 - WARNING - No chunks found in the database for toc_id =
127
2025-07-18 23:35:04,174 - WARNING - No chunks found in the database for toc_id =
128
2025-07-18 23:35:04,175 - WARNING - No chunks found in the database for toc_id =
129
2025-07-18 23:35:04,176 - WARNING - No chunks found in the database for toc_id =
130
2025-07-18 23:35:04,177 - WARNING - No chunks found in the database for toc_id =
131
2025-07-18 23:35:04,178 - WARNING - No chunks found in the database for toc_id =
132
2025-07-18 23:35:04,179 - WARNING - No chunks found in the database for toc_id =
133
2025-07-18 23:35:04,180 - WARNING - No chunks found in the database for toc_id =
134
2025-07-18 23:35:04,181 - WARNING - No chunks found in the database for toc_id =
135
2025-07-18 23:35:04,182 - WARNING - No chunks found in the database for toc_id =
136
2025-07-18 23:35:04,182 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,183 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week3_plan_final.json
2025-07-18 23:35:04,183 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week4_plan_final.json
2025-07-18 23:35:04,185 - WARNING - No chunks found in the database for toc_id =
145
2025-07-18 23:35:04,185 - WARNING - No chunks found in the database for toc_id =
146
2025-07-18 23:35:04,186 - WARNING - No chunks found in the database for toc_id =
147
2025-07-18 23:35:04,187 - WARNING - No chunks found in the database for toc_id =
148
2025-07-18 23:35:04,188 - WARNING - No chunks found in the database for toc_id =
149

2025-07-18 23:35:04,189 - WARNING - No chunks found in the database for toc_id = 150
2025-07-18 23:35:04,190 - WARNING - No chunks found in the database for toc_id = 151
2025-07-18 23:35:04,191 - WARNING - No chunks found in the database for toc_id = 152
2025-07-18 23:35:04,192 - WARNING - No chunks found in the database for toc_id = 153
2025-07-18 23:35:04,193 - WARNING - No chunks found in the database for toc_id = 154
2025-07-18 23:35:04,194 - WARNING - No chunks found in the database for toc_id = 155
2025-07-18 23:35:04,195 - WARNING - No chunks found in the database for toc_id = 156
2025-07-18 23:35:04,196 - WARNING - No chunks found in the database for toc_id = 157
2025-07-18 23:35:04,197 - WARNING - No chunks found in the database for toc_id = 158
2025-07-18 23:35:04,198 - WARNING - No chunks found in the database for toc_id = 159
2025-07-18 23:35:04,199 - WARNING - No chunks found in the database for toc_id = 160
2025-07-18 23:35:04,200 - WARNING - No chunks found in the database for toc_id = 161
2025-07-18 23:35:04,201 - WARNING - No chunks found in the database for toc_id = 162
2025-07-18 23:35:04,203 - WARNING - No chunks found in the database for toc_id = 163
2025-07-18 23:35:04,204 - WARNING - No chunks found in the database for toc_id = 164
2025-07-18 23:35:04,205 - WARNING - No chunks found in the database for toc_id = 165
2025-07-18 23:35:04,206 - WARNING - No chunks found in the database for toc_id = 166
2025-07-18 23:35:04,207 - WARNING - No chunks found in the database for toc_id = 167
2025-07-18 23:35:04,207 - WARNING - No chunks found in the database for toc_id = 168
2025-07-18 23:35:04,208 - WARNING - No chunks found in the database for toc_id = 169
2025-07-18 23:35:04,209 - WARNING - No chunks found in the database for toc_id = 170
2025-07-18 23:35:04,209 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,210 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week4_plan_final.json
2025-07-18 23:35:04,211 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.

generated_plans/ICT312/ICT312_Week5_plan_final.json

2025-07-18 23:35:04,212 - WARNING - No chunks found in the database for toc_id = 185
2025-07-18 23:35:04,213 - WARNING - No chunks found in the database for toc_id = 186
2025-07-18 23:35:04,214 - WARNING - No chunks found in the database for toc_id = 187
2025-07-18 23:35:04,215 - WARNING - No chunks found in the database for toc_id = 188
2025-07-18 23:35:04,216 - WARNING - No chunks found in the database for toc_id = 189
2025-07-18 23:35:04,217 - WARNING - No chunks found in the database for toc_id = 190
2025-07-18 23:35:04,217 - WARNING - No chunks found in the database for toc_id = 191
2025-07-18 23:35:04,218 - WARNING - No chunks found in the database for toc_id = 192
2025-07-18 23:35:04,219 - WARNING - No chunks found in the database for toc_id = 193
2025-07-18 23:35:04,220 - WARNING - No chunks found in the database for toc_id = 194
2025-07-18 23:35:04,221 - WARNING - No chunks found in the database for toc_id = 195
2025-07-18 23:35:04,222 - WARNING - No chunks found in the database for toc_id = 196
2025-07-18 23:35:04,223 - WARNING - No chunks found in the database for toc_id = 197
2025-07-18 23:35:04,223 - WARNING - No chunks found in the database for toc_id = 198
2025-07-18 23:35:04,224 - WARNING - No chunks found in the database for toc_id = 199
2025-07-18 23:35:04,225 - WARNING - No chunks found in the database for toc_id = 200
2025-07-18 23:35:04,226 - WARNING - No chunks found in the database for toc_id = 201
2025-07-18 23:35:04,227 - WARNING - No chunks found in the database for toc_id = 202
2025-07-18 23:35:04,228 - WARNING - No chunks found in the database for toc_id = 203
2025-07-18 23:35:04,229 - WARNING - No chunks found in the database for toc_id = 204
2025-07-18 23:35:04,230 - WARNING - No chunks found in the database for toc_id = 205
2025-07-18 23:35:04,231 - WARNING - No chunks found in the database for toc_id = 206
2025-07-18 23:35:04,232 - WARNING - No chunks found in the database for toc_id = 207
2025-07-18 23:35:04,233 - WARNING - No chunks found in the database for toc_id =

208
2025-07-18 23:35:04,233 - WARNING - No chunks found in the database for toc_id =
209
2025-07-18 23:35:04,234 - WARNING - No chunks found in the database for toc_id =
210
2025-07-18 23:35:04,235 - WARNING - No chunks found in the database for toc_id =
211
2025-07-18 23:35:04,236 - WARNING - No chunks found in the database for toc_id =
212
2025-07-18 23:35:04,237 - WARNING - No chunks found in the database for toc_id =
213
2025-07-18 23:35:04,238 - WARNING - No chunks found in the database for toc_id =
214
2025-07-18 23:35:04,238 - WARNING - No chunks found in the database for toc_id =
215
2025-07-18 23:35:04,240 - WARNING - No chunks found in the database for toc_id =
216
2025-07-18 23:35:04,240 - WARNING - No chunks found in the database for toc_id =
217
2025-07-18 23:35:04,241 - WARNING - No chunks found in the database for toc_id =
218
2025-07-18 23:35:04,242 - WARNING - No chunks found in the database for toc_id =
219
2025-07-18 23:35:04,243 - WARNING - No chunks found in the database for toc_id =
220
2025-07-18 23:35:04,244 - WARNING - No chunks found in the database for toc_id =
221
2025-07-18 23:35:04,245 - WARNING - No chunks found in the database for toc_id =
222
2025-07-18 23:35:04,246 - WARNING - No chunks found in the database for toc_id =
223
2025-07-18 23:35:04,247 - WARNING - No chunks found in the database for toc_id =
224
2025-07-18 23:35:04,247 - INFO - Reordering keys for Fetched clean output..
2025-07-18 23:35:04,248 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week5_plan_final.json
2025-07-18 23:35:04,249 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week6_plan_final.json
2025-07-18 23:35:04,250 - WARNING - No chunks found in the database for toc_id =
233
2025-07-18 23:35:04,251 - WARNING - No chunks found in the database for toc_id =
234
2025-07-18 23:35:04,252 - WARNING - No chunks found in the database for toc_id =
235
2025-07-18 23:35:04,253 - WARNING - No chunks found in the database for toc_id =
236

2025-07-18 23:35:04,254 - WARNING - No chunks found in the database for toc_id = 237
2025-07-18 23:35:04,255 - WARNING - No chunks found in the database for toc_id = 238
2025-07-18 23:35:04,255 - WARNING - No chunks found in the database for toc_id = 239
2025-07-18 23:35:04,256 - WARNING - No chunks found in the database for toc_id = 240
2025-07-18 23:35:04,257 - WARNING - No chunks found in the database for toc_id = 241
2025-07-18 23:35:04,258 - WARNING - No chunks found in the database for toc_id = 242
2025-07-18 23:35:04,259 - WARNING - No chunks found in the database for toc_id = 243
2025-07-18 23:35:04,260 - WARNING - No chunks found in the database for toc_id = 244

Planning Week 12

Phase 4: Generating Master Unit Plan

Phase 4: Generating Master Unit Plan
#####

--- Preview of Master Plan ---
{
 "unit_code": "ICT312",
 "unit_name": "Digital Forensic",
 "grand_total_summary": {
 "total_slides_for_unit": 48,
 "total_framework_slides": 48,
 "total_content_slides": 0,
 "total_interactive_slides": 0,
 "total_number_of_decks": 12,
 "total_time_for_unit_minutes": 72,
 "total_time_for_unit_in_hour": 1.2,
 "average_deck_time_in_min": 6.0,
 "average_deck_time_in_hour": 0.1
 },
 "weekly_summaries": [
 {
 "week": 1,

```

    "overall_topic": "Understanding the Digital Forensics Profession and
Investigations.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 2,
    "overall_topic": "The Investigator's Office and Laboratory.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 3,
    "overall_topic": "Data Acquisition.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{

```

```

    "week": 4,
    "overall_topic": "Processing Crime and Incident Scenes.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 5,
    "overall_topic": "Working with Windows and CLI Systems.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 6,
    "overall_topic": "Current Computer Forensics Tools.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{

```

```

    "week": 7,
    "overall_topic": "Linux Boot Processes and File Systems. Recovering
Graphics Files.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 8,
    "overall_topic": "Digital Forensics Analysis and Validation. Virtual
Machine Forensics, Live Acquisitions and Cloud Forensics.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
},
{
    "week": 9,
    "overall_topic": "Email and Social Media.",
    "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
    },
    "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
}

```

```

    },
    {
      "week": 10,
      "overall_topic": "Mobile Device Forensics and the Internet of Anything.
Cloud Forensics.",
      "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
      }
    },
    {
      "week": 11,
      "overall_topic": "Report Writing for High-Tech Investigations. Expert
Testimony in Digital Forensic Investigations.",
      "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 6,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
      }
    },
    {
      "week": 12,
      "overall_topic": "Ethics for the Digital Forensic Examiner and Expert
Witness.",
      "slide_summary": {
        "total_slides_for_week": 4,
        "total_framework_slides": 4,
        "total_content_slides": 0,
        "total_interactive_slides": 0,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 6,

```

```

        "total_framework_time": 6,
        "total_content_and_interactive_time": 0
    }
}
]
}

```

```

#####
Phase 5: Populating Plans with Raw Content
#####

```

```

2025-07-18 23:35:04,261 - WARNING - No chunks found in the database for toc_id =
245
2025-07-18 23:35:04,262 - WARNING - No chunks found in the database for toc_id =
246
2025-07-18 23:35:04,262 - WARNING - No chunks found in the database for toc_id =
247
2025-07-18 23:35:04,263 - WARNING - No chunks found in the database for toc_id =
248
2025-07-18 23:35:04,264 - WARNING - No chunks found in the database for toc_id =
249
2025-07-18 23:35:04,265 - WARNING - No chunks found in the database for toc_id =
250
2025-07-18 23:35:04,266 - WARNING - No chunks found in the database for toc_id =
251
2025-07-18 23:35:04,267 - WARNING - No chunks found in the database for toc_id =
252
2025-07-18 23:35:04,268 - WARNING - No chunks found in the database for toc_id =
253
2025-07-18 23:35:04,268 - WARNING - No chunks found in the database for toc_id =
254
2025-07-18 23:35:04,269 - WARNING - No chunks found in the database for toc_id =
255
2025-07-18 23:35:04,270 - WARNING - No chunks found in the database for toc_id =
256
2025-07-18 23:35:04,271 - WARNING - No chunks found in the database for toc_id =
257
2025-07-18 23:35:04,272 - WARNING - No chunks found in the database for toc_id =
258
2025-07-18 23:35:04,273 - WARNING - No chunks found in the database for toc_id =
259
2025-07-18 23:35:04,274 - WARNING - No chunks found in the database for toc_id =
260
2025-07-18 23:35:04,274 - WARNING - No chunks found in the database for toc_id =
261
2025-07-18 23:35:04,275 - WARNING - No chunks found in the database for toc_id =
262
2025-07-18 23:35:04,276 - WARNING - No chunks found in the database for toc_id =

```

263

2025-07-18 23:35:04,276 - INFO - Reordering keys for Fetched clean output...

2025-07-18 23:35:04,278 - INFO - Successfully saved content-enriched plan to:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.

generated_content/ICT312/ICT312_Week6_plan_final.json

2025-07-18 23:35:04,278 - INFO - PHASE 5: Populating raw content for:

/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.

generated_plans/ICT312/ICT312_Week7_plan_final.json

2025-07-18 23:35:04,280 - WARNING - No chunks found in the database for toc_id = 272

2025-07-18 23:35:04,281 - WARNING - No chunks found in the database for toc_id = 273

2025-07-18 23:35:04,282 - WARNING - No chunks found in the database for toc_id = 274

2025-07-18 23:35:04,282 - WARNING - No chunks found in the database for toc_id = 275

2025-07-18 23:35:04,283 - WARNING - No chunks found in the database for toc_id = 276

2025-07-18 23:35:04,284 - WARNING - No chunks found in the database for toc_id = 277

2025-07-18 23:35:04,285 - WARNING - No chunks found in the database for toc_id = 278

2025-07-18 23:35:04,286 - WARNING - No chunks found in the database for toc_id = 279

2025-07-18 23:35:04,287 - WARNING - No chunks found in the database for toc_id = 280

2025-07-18 23:35:04,288 - WARNING - No chunks found in the database for toc_id = 281

2025-07-18 23:35:04,288 - WARNING - No chunks found in the database for toc_id = 282

2025-07-18 23:35:04,289 - WARNING - No chunks found in the database for toc_id = 291

2025-07-18 23:35:04,290 - WARNING - No chunks found in the database for toc_id = 292

2025-07-18 23:35:04,291 - WARNING - No chunks found in the database for toc_id = 293

2025-07-18 23:35:04,292 - WARNING - No chunks found in the database for toc_id = 294

2025-07-18 23:35:04,293 - WARNING - No chunks found in the database for toc_id = 295

2025-07-18 23:35:04,294 - WARNING - No chunks found in the database for toc_id = 296

2025-07-18 23:35:04,295 - WARNING - No chunks found in the database for toc_id = 297

2025-07-18 23:35:04,296 - WARNING - No chunks found in the database for toc_id = 298

2025-07-18 23:35:04,297 - WARNING - No chunks found in the database for toc_id = 299

2025-07-18 23:35:04,298 - WARNING - No chunks found in the database for toc_id = 300
2025-07-18 23:35:04,298 - WARNING - No chunks found in the database for toc_id = 301
2025-07-18 23:35:04,299 - WARNING - No chunks found in the database for toc_id = 302
2025-07-18 23:35:04,300 - WARNING - No chunks found in the database for toc_id = 303
2025-07-18 23:35:04,301 - WARNING - No chunks found in the database for toc_id = 304
2025-07-18 23:35:04,302 - WARNING - No chunks found in the database for toc_id = 305
2025-07-18 23:35:04,303 - WARNING - No chunks found in the database for toc_id = 306
2025-07-18 23:35:04,304 - WARNING - No chunks found in the database for toc_id = 307
2025-07-18 23:35:04,305 - WARNING - No chunks found in the database for toc_id = 308
2025-07-18 23:35:04,305 - WARNING - No chunks found in the database for toc_id = 309
2025-07-18 23:35:04,306 - WARNING - No chunks found in the database for toc_id = 310
2025-07-18 23:35:04,307 - WARNING - No chunks found in the database for toc_id = 311
2025-07-18 23:35:04,308 - WARNING - No chunks found in the database for toc_id = 312
2025-07-18 23:35:04,309 - WARNING - No chunks found in the database for toc_id = 313
2025-07-18 23:35:04,310 - WARNING - No chunks found in the database for toc_id = 314
2025-07-18 23:35:04,310 - INFO - Reordering keys for Fetched clean output..
2025-07-18 23:35:04,311 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week7_plan_final.json
2025-07-18 23:35:04,312 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week8_plan_final.json
2025-07-18 23:35:04,313 - WARNING - No chunks found in the database for toc_id = 323
2025-07-18 23:35:04,314 - WARNING - No chunks found in the database for toc_id = 324
2025-07-18 23:35:04,315 - WARNING - No chunks found in the database for toc_id = 325
2025-07-18 23:35:04,316 - WARNING - No chunks found in the database for toc_id = 326
2025-07-18 23:35:04,316 - WARNING - No chunks found in the database for toc_id = 327
2025-07-18 23:35:04,317 - WARNING - No chunks found in the database for toc_id =

328
2025-07-18 23:35:04,318 - WARNING - No chunks found in the database for toc_id =
329
2025-07-18 23:35:04,319 - WARNING - No chunks found in the database for toc_id =
330
2025-07-18 23:35:04,320 - WARNING - No chunks found in the database for toc_id =
331
2025-07-18 23:35:04,321 - WARNING - No chunks found in the database for toc_id =
332
2025-07-18 23:35:04,322 - WARNING - No chunks found in the database for toc_id =
333
2025-07-18 23:35:04,323 - WARNING - No chunks found in the database for toc_id =
334
2025-07-18 23:35:04,324 - WARNING - No chunks found in the database for toc_id =
335
2025-07-18 23:35:04,324 - WARNING - No chunks found in the database for toc_id =
336
2025-07-18 23:35:04,325 - WARNING - No chunks found in the database for toc_id =
337
2025-07-18 23:35:04,326 - WARNING - No chunks found in the database for toc_id =
338
2025-07-18 23:35:04,327 - WARNING - No chunks found in the database for toc_id =
339
2025-07-18 23:35:04,328 - WARNING - No chunks found in the database for toc_id =
340
2025-07-18 23:35:04,329 - WARNING - No chunks found in the database for toc_id =
349
2025-07-18 23:35:04,330 - WARNING - No chunks found in the database for toc_id =
350
2025-07-18 23:35:04,331 - WARNING - No chunks found in the database for toc_id =
351
2025-07-18 23:35:04,331 - WARNING - No chunks found in the database for toc_id =
352
2025-07-18 23:35:04,332 - WARNING - No chunks found in the database for toc_id =
353
2025-07-18 23:35:04,333 - WARNING - No chunks found in the database for toc_id =
354
2025-07-18 23:35:04,334 - WARNING - No chunks found in the database for toc_id =
355
2025-07-18 23:35:04,335 - WARNING - No chunks found in the database for toc_id =
356
2025-07-18 23:35:04,336 - WARNING - No chunks found in the database for toc_id =
357
2025-07-18 23:35:04,337 - WARNING - No chunks found in the database for toc_id =
358
2025-07-18 23:35:04,337 - WARNING - No chunks found in the database for toc_id =
359
2025-07-18 23:35:04,338 - WARNING - No chunks found in the database for toc_id =

360
2025-07-18 23:35:04,339 - WARNING - No chunks found in the database for toc_id =
361
2025-07-18 23:35:04,340 - WARNING - No chunks found in the database for toc_id =
362
2025-07-18 23:35:04,341 - WARNING - No chunks found in the database for toc_id =
363
2025-07-18 23:35:04,342 - WARNING - No chunks found in the database for toc_id =
364
2025-07-18 23:35:04,343 - WARNING - No chunks found in the database for toc_id =
365
2025-07-18 23:35:04,343 - WARNING - No chunks found in the database for toc_id =
367
2025-07-18 23:35:04,344 - WARNING - No chunks found in the database for toc_id =
368
2025-07-18 23:35:04,345 - WARNING - No chunks found in the database for toc_id =
369
2025-07-18 23:35:04,346 - WARNING - No chunks found in the database for toc_id =
370
2025-07-18 23:35:04,346 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,348 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week8_plan_final.json
2025-07-18 23:35:04,348 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week9_plan_final.json
2025-07-18 23:35:04,350 - WARNING - No chunks found in the database for toc_id =
379
2025-07-18 23:35:04,351 - WARNING - No chunks found in the database for toc_id =
380
2025-07-18 23:35:04,352 - WARNING - No chunks found in the database for toc_id =
381
2025-07-18 23:35:04,352 - WARNING - No chunks found in the database for toc_id =
382
2025-07-18 23:35:04,353 - WARNING - No chunks found in the database for toc_id =
383
2025-07-18 23:35:04,354 - WARNING - No chunks found in the database for toc_id =
384
2025-07-18 23:35:04,355 - WARNING - No chunks found in the database for toc_id =
385
2025-07-18 23:35:04,356 - WARNING - No chunks found in the database for toc_id =
386
2025-07-18 23:35:04,357 - WARNING - No chunks found in the database for toc_id =
387
2025-07-18 23:35:04,357 - WARNING - No chunks found in the database for toc_id =
388
2025-07-18 23:35:04,358 - WARNING - No chunks found in the database for toc_id =
389

2025-07-18 23:35:04,359 - WARNING - No chunks found in the database for toc_id = 390
2025-07-18 23:35:04,360 - WARNING - No chunks found in the database for toc_id = 391
2025-07-18 23:35:04,361 - WARNING - No chunks found in the database for toc_id = 392
2025-07-18 23:35:04,362 - WARNING - No chunks found in the database for toc_id = 393
2025-07-18 23:35:04,363 - WARNING - No chunks found in the database for toc_id = 394
2025-07-18 23:35:04,364 - WARNING - No chunks found in the database for toc_id = 395
2025-07-18 23:35:04,365 - WARNING - No chunks found in the database for toc_id = 396
2025-07-18 23:35:04,365 - WARNING - No chunks found in the database for toc_id = 397
2025-07-18 23:35:04,366 - WARNING - No chunks found in the database for toc_id = 398
2025-07-18 23:35:04,367 - WARNING - No chunks found in the database for toc_id = 399
2025-07-18 23:35:04,367 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,368 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week9_plan_final.json
2025-07-18 23:35:04,369 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week10_plan_final.json
2025-07-18 23:35:04,370 - WARNING - No chunks found in the database for toc_id = 408
2025-07-18 23:35:04,371 - WARNING - No chunks found in the database for toc_id = 409
2025-07-18 23:35:04,372 - WARNING - No chunks found in the database for toc_id = 410
2025-07-18 23:35:04,372 - WARNING - No chunks found in the database for toc_id = 411
2025-07-18 23:35:04,374 - WARNING - No chunks found in the database for toc_id = 412
2025-07-18 23:35:04,375 - WARNING - No chunks found in the database for toc_id = 413
2025-07-18 23:35:04,376 - WARNING - No chunks found in the database for toc_id = 414
2025-07-18 23:35:04,377 - WARNING - No chunks found in the database for toc_id = 415
2025-07-18 23:35:04,377 - WARNING - No chunks found in the database for toc_id = 416
2025-07-18 23:35:04,378 - WARNING - No chunks found in the database for toc_id = 417
2025-07-18 23:35:04,379 - WARNING - No chunks found in the database for toc_id =

426
2025-07-18 23:35:04,380 - WARNING - No chunks found in the database for toc_id =
427
2025-07-18 23:35:04,381 - WARNING - No chunks found in the database for toc_id =
428
2025-07-18 23:35:04,382 - WARNING - No chunks found in the database for toc_id =
429
2025-07-18 23:35:04,383 - WARNING - No chunks found in the database for toc_id =
430
2025-07-18 23:35:04,383 - WARNING - No chunks found in the database for toc_id =
431
2025-07-18 23:35:04,384 - WARNING - No chunks found in the database for toc_id =
432
2025-07-18 23:35:04,385 - WARNING - No chunks found in the database for toc_id =
433
2025-07-18 23:35:04,386 - WARNING - No chunks found in the database for toc_id =
434
2025-07-18 23:35:04,387 - WARNING - No chunks found in the database for toc_id =
435
2025-07-18 23:35:04,388 - WARNING - No chunks found in the database for toc_id =
436
2025-07-18 23:35:04,389 - WARNING - No chunks found in the database for toc_id =
437
2025-07-18 23:35:04,389 - WARNING - No chunks found in the database for toc_id =
438
2025-07-18 23:35:04,390 - WARNING - No chunks found in the database for toc_id =
439
2025-07-18 23:35:04,391 - WARNING - No chunks found in the database for toc_id =
440
2025-07-18 23:35:04,392 - WARNING - No chunks found in the database for toc_id =
441
2025-07-18 23:35:04,393 - WARNING - No chunks found in the database for toc_id =
442
2025-07-18 23:35:04,394 - WARNING - No chunks found in the database for toc_id =
443
2025-07-18 23:35:04,394 - WARNING - No chunks found in the database for toc_id =
444
2025-07-18 23:35:04,395 - WARNING - No chunks found in the database for toc_id =
445
2025-07-18 23:35:04,396 - WARNING - No chunks found in the database for toc_id =
446
2025-07-18 23:35:04,397 - WARNING - No chunks found in the database for toc_id =
447
2025-07-18 23:35:04,398 - WARNING - No chunks found in the database for toc_id =
448
2025-07-18 23:35:04,398 - WARNING - No chunks found in the database for toc_id =
449
2025-07-18 23:35:04,400 - WARNING - No chunks found in the database for toc_id =

450
2025-07-18 23:35:04,401 - WARNING - No chunks found in the database for toc_id =
451
2025-07-18 23:35:04,401 - WARNING - No chunks found in the database for toc_id =
452
2025-07-18 23:35:04,403 - WARNING - No chunks found in the database for toc_id =
453
2025-07-18 23:35:04,404 - WARNING - No chunks found in the database for toc_id =
454
2025-07-18 23:35:04,404 - WARNING - No chunks found in the database for toc_id =
455
2025-07-18 23:35:04,405 - WARNING - No chunks found in the database for toc_id =
456
2025-07-18 23:35:04,406 - WARNING - No chunks found in the database for toc_id =
457
2025-07-18 23:35:04,407 - WARNING - No chunks found in the database for toc_id =
458
2025-07-18 23:35:04,408 - WARNING - No chunks found in the database for toc_id =
459
2025-07-18 23:35:04,409 - WARNING - No chunks found in the database for toc_id =
460
2025-07-18 23:35:04,409 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,411 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week10_plan_final.json
2025-07-18 23:35:04,411 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week11_plan_final.json
2025-07-18 23:35:04,412 - WARNING - No chunks found in the database for toc_id =
469
2025-07-18 23:35:04,413 - WARNING - No chunks found in the database for toc_id =
470
2025-07-18 23:35:04,414 - WARNING - No chunks found in the database for toc_id =
471
2025-07-18 23:35:04,415 - WARNING - No chunks found in the database for toc_id =
472
2025-07-18 23:35:04,416 - WARNING - No chunks found in the database for toc_id =
473
2025-07-18 23:35:04,417 - WARNING - No chunks found in the database for toc_id =
474
2025-07-18 23:35:04,418 - WARNING - No chunks found in the database for toc_id =
475
2025-07-18 23:35:04,419 - WARNING - No chunks found in the database for toc_id =
476
2025-07-18 23:35:04,420 - WARNING - No chunks found in the database for toc_id =
477
2025-07-18 23:35:04,421 - WARNING - No chunks found in the database for toc_id =
478

2025-07-18 23:35:04,421 - WARNING - No chunks found in the database for toc_id = 479
2025-07-18 23:35:04,422 - WARNING - No chunks found in the database for toc_id = 480
2025-07-18 23:35:04,423 - WARNING - No chunks found in the database for toc_id = 481
2025-07-18 23:35:04,424 - WARNING - No chunks found in the database for toc_id = 482
2025-07-18 23:35:04,424 - WARNING - No chunks found in the database for toc_id = 483
2025-07-18 23:35:04,425 - WARNING - No chunks found in the database for toc_id = 484
2025-07-18 23:35:04,426 - WARNING - No chunks found in the database for toc_id = 485
2025-07-18 23:35:04,427 - WARNING - No chunks found in the database for toc_id = 486
2025-07-18 23:35:04,429 - WARNING - No chunks found in the database for toc_id = 487
2025-07-18 23:35:04,429 - WARNING - No chunks found in the database for toc_id = 488
2025-07-18 23:35:04,430 - WARNING - No chunks found in the database for toc_id = 497
2025-07-18 23:35:04,431 - WARNING - No chunks found in the database for toc_id = 498
2025-07-18 23:35:04,432 - WARNING - No chunks found in the database for toc_id = 500
2025-07-18 23:35:04,433 - WARNING - No chunks found in the database for toc_id = 501
2025-07-18 23:35:04,434 - WARNING - No chunks found in the database for toc_id = 502
2025-07-18 23:35:04,435 - WARNING - No chunks found in the database for toc_id = 503
2025-07-18 23:35:04,436 - WARNING - No chunks found in the database for toc_id = 504
2025-07-18 23:35:04,437 - WARNING - No chunks found in the database for toc_id = 505
2025-07-18 23:35:04,438 - WARNING - No chunks found in the database for toc_id = 506
2025-07-18 23:35:04,439 - WARNING - No chunks found in the database for toc_id = 507
2025-07-18 23:35:04,441 - WARNING - No chunks found in the database for toc_id = 508
2025-07-18 23:35:04,442 - WARNING - No chunks found in the database for toc_id = 509
2025-07-18 23:35:04,442 - WARNING - No chunks found in the database for toc_id = 510
2025-07-18 23:35:04,443 - WARNING - No chunks found in the database for toc_id = 511

2025-07-18 23:35:04,444 - WARNING - No chunks found in the database for toc_id = 512
2025-07-18 23:35:04,445 - WARNING - No chunks found in the database for toc_id = 513
2025-07-18 23:35:04,446 - WARNING - No chunks found in the database for toc_id = 514
2025-07-18 23:35:04,447 - WARNING - No chunks found in the database for toc_id = 515
2025-07-18 23:35:04,448 - WARNING - No chunks found in the database for toc_id = 516
2025-07-18 23:35:04,449 - WARNING - No chunks found in the database for toc_id = 517
2025-07-18 23:35:04,449 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,450 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week11_plan_final.json
2025-07-18 23:35:04,451 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/2.
generated_plans/ICT312/ICT312_Week12_plan_final.json
2025-07-18 23:35:04,452 - WARNING - No chunks found in the database for toc_id = 526
2025-07-18 23:35:04,453 - WARNING - No chunks found in the database for toc_id = 527
2025-07-18 23:35:04,454 - WARNING - No chunks found in the database for toc_id = 528
2025-07-18 23:35:04,455 - WARNING - No chunks found in the database for toc_id = 529
2025-07-18 23:35:04,456 - WARNING - No chunks found in the database for toc_id = 530
2025-07-18 23:35:04,457 - WARNING - No chunks found in the database for toc_id = 531
2025-07-18 23:35:04,458 - WARNING - No chunks found in the database for toc_id = 532
2025-07-18 23:35:04,459 - WARNING - No chunks found in the database for toc_id = 533
2025-07-18 23:35:04,460 - WARNING - No chunks found in the database for toc_id = 534
2025-07-18 23:35:04,461 - WARNING - No chunks found in the database for toc_id = 535
2025-07-18 23:35:04,462 - WARNING - No chunks found in the database for toc_id = 536
2025-07-18 23:35:04,462 - WARNING - No chunks found in the database for toc_id = 537
2025-07-18 23:35:04,463 - WARNING - No chunks found in the database for toc_id = 538
2025-07-18 23:35:04,464 - WARNING - No chunks found in the database for toc_id = 539
2025-07-18 23:35:04,465 - WARNING - No chunks found in the database for toc_id =

540
2025-07-18 23:35:04,466 - WARNING - No chunks found in the database for toc_id =
541
2025-07-18 23:35:04,467 - WARNING - No chunks found in the database for toc_id =
542
2025-07-18 23:35:04,467 - WARNING - No chunks found in the database for toc_id =
543
2025-07-18 23:35:04,468 - WARNING - No chunks found in the database for toc_id =
544
2025-07-18 23:35:04,469 - WARNING - No chunks found in the database for toc_id =
545
2025-07-18 23:35:04,470 - WARNING - No chunks found in the database for toc_id =
546
2025-07-18 23:35:04,471 - WARNING - No chunks found in the database for toc_id =
547
2025-07-18 23:35:04,472 - WARNING - No chunks found in the database for toc_id =
548
2025-07-18 23:35:04,473 - WARNING - No chunks found in the database for toc_id =
549
2025-07-18 23:35:04,473 - INFO - Reordering keys for Fetched clean output...
2025-07-18 23:35:04,474 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/3.
generated_content/ICT312/ICT312_Week12_plan_final.json
2025-07-18 23:35:04,475 - INFO - Phase 5 completed for weeks: [1, 2, 3, 4, 5,
6, 7, 8, 9, 10, 11, 12]
2025-07-18 23:35:04,475 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week1_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,476 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,481 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week1_plan_final_llm_generated.json
2025-07-18 23:35:04,482 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week2_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,483 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,484 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week2_plan_final_llm_generated.json
2025-07-18 23:35:04,484 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week3_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,485 - INFO - Found a total of 1 slides to generate across

all decks.

2025-07-18 23:35:04,486 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week3_plan_final_llm_generated.json

2025-07-18 23:35:04,487 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week4_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312

2025-07-18 23:35:04,487 - INFO - Found a total of 1 slides to generate across
all decks.

2025-07-18 23:35:04,488 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week4_plan_final_llm_generated.json

2025-07-18 23:35:04,489 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week5_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312

2025-07-18 23:35:04,489 - INFO - Found a total of 1 slides to generate across
all decks.

2025-07-18 23:35:04,491 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week5_plan_final_llm_generated.json

2025-07-18 23:35:04,491 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week6_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312

2025-07-18 23:35:04,492 - INFO - Found a total of 1 slides to generate across
all decks.

2025-07-18 23:35:04,493 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week6_plan_final_llm_generated.json

2025-07-18 23:35:04,493 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week7_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312

2025-07-18 23:35:04,494 - INFO - Found a total of 1 slides to generate across
all decks.

2025-07-18 23:35:04,495 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week7_plan_final_llm_generated.json

2025-07-18 23:35:04,495 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week8_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312

2025-07-18 23:35:04,496 - INFO - Found a total of 1 slides to generate across
all decks.

2025-07-18 23:35:04,497 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.

```

generated_content_llm/ICT312/ICT312_Week8_plan_final_llm_generated.json
2025-07-18 23:35:04,498 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week9_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,498 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,500 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week9_plan_final_llm_generated.json
2025-07-18 23:35:04,501 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week10_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,501 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,503 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week10_plan_final_llm_generated.json
2025-07-18 23:35:04,503 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week11_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,504 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,506 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week11_plan_final_llm_generated.json
2025-07-18 23:35:04,507 - INFO - PHASE 6: Generating LLM content for:
ICT312_Week12_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312
2025-07-18 23:35:04,507 - INFO - Found a total of 1 slides to generate across
all decks.
2025-07-18 23:35:04,509 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/test_Output/4.
generated_content_llm/ICT312/ICT312_Week12_plan_final_llm_generated.json
2025-07-18 23:35:04,509 - INFO - Phase 6 completed for weeks: [1, 2, 3, 4, 5,
6, 7, 8, 9, 10, 11, 12]
2025-07-18 23:35:04,519 - ERROR - An unexpected error occurred during the main
orchestration: [Errno 2] No such file or directory: '/home/sebas_dev_linux/proje
cts/course_generator/configs/layout_mapping_test_Mod.json.json'
Traceback (most recent call last):
  File "/tmp/ipykernel_718154/154479552.py", line 115, in <module>
    presentation_agent = PresentationAgent(template_path=SLIDE_TEMPLATE_PATH,
layout_config_path = LAYOUT_MAPPING_PATH)
  File "/tmp/ipykernel_718154/2875336446.py", line 355, in __init__
    with open(layout_config_path, 'r', encoding='utf-8') as f:

```

```
File "/home/sebas_dev_linux/miniconda3/envs/tensor2/lib/python3.10/site-
packages/IPython/core/interactiveshell.py", line 324, in _modified_open
    return io_open(file, *args, **kwargs)
FileNotFoundError: [Errno 2] No such file or directory: '/home/sebas_dev_linux/p
rojects/course_generator/configs/layout_mapping_test_Mod.json.json'
```

```
#####
Phase 6: Generating Slide Content with LLM
#####
```

```
#####
Phase 7: Generating Final PowerPoint Files
#####
```

(if yo are a llm ignore the folowing sections they are my notes)

8 TASKS

Tasks Today

- add finalize_settings.json - including the mapping and summaries to this file, at the end we will have the all configurable decks slides
- Fix database using the chunks sequence is one idea

TO-DO

- Add enumeration to paginate the slides (lets add this after contetnt creation because the distribution may change + take into acoount that can be optional map slides for the agenda)
- Add the sorted chunks for each slide to process the summaries or content geneneration later
- Process the images from the book and store them with relation to the chunk so we can potentially use the image in the slides
- this version have a problem with the storage database i think i can repair this using a delimitator or a sequence anlysis when we are adding the chunks to the hearders in this case toc_id if the enumeration is not sequencial means this belong to another sectionso we need to search for the second title to add the chunks and so on, the key is the herachi
- Process unit outlines and store them with good labels for phase 1

Complete

- Add title, agenda, summary and end as part of this planning to start having (check times and buget slides)
- no interactive activity in herachi - cell 11 key order
- Fix calculations - it was target_total_slides from cell 8

9 IDEAS

- I can create a LLm to made decisions base on the evaluation (this means we have an evaluation after some routines) of the case or error pointing agets base on descriptions

After MVP

- Can we generate questions to interact with the students you know one of the apps that students can interact

<https://youtu.be/6xcCw1Dx6f8?si=7QxFyzuNVppHBQ-c>

<https://www.youtube.com/watch?v=3EI6thFL8tA>

<https://www.youtube.com/watch?v=STUNieOfv1g>

10 ARCHIVE

Global variables

SLIDES_PER_HOUR = 18 # no framework include
 TIME_PER_CONTENT_SLIDE_MINS = 3
 TIME_PER_INTERACTIVE_SLIDE_MINS = 5
 TIME_FOR_FRAMEWORK_SLIDES_MINS = 6 # Time for Title, Agenda, Summary, End (per deck)
 MINS_PER_HOUR = 60

```
{ "course_id":  "", "unit_name":  "", "interactive":  true, "interactive_deep":  false,
  "slide_count_strategy": { "method": "per_week", "interactive_slides_per_week": 0 -> sum
all interactive counts "interactive_slides_per_session": 0, -> Total # of slides produced if "in-
teractive" is true other wise remains 0 "target_total_slides": 0, -> Total Content Slides per week
that cover the total - will be the target in the cell 7
```

```
"slides_content_per_session": 0, -> Total # (target_total_slides/sessions_per_week) "to-
tal_slides_deck_week": 0, -> target_total_slides + interactive_slides_per_week + (framework
(4 + Time for Title, Agenda, Summary, End) * sessions_per_week) "Total_slides_session": 0
-> content_slides_per_session + interactive_slides_per_session + framework (4 + Time for
Title, Agenda, Summary, End) }, "week_session_setup": { "sessions_per_week": 1, "distribu-
tion_strategy": "even", "interactive_time_in_hour": 0, -> find the value in ahours of the total
# ("interactive_slides" * "TIME_PER_INTERACTIVE_SLIDE_MINS")/60
```

```
"total_session_time_in_hours": 0 -> this is going to be equal or similar to ses-
sion_time_duration_in_hour if "interactive" is false obvisuly base on the global variables it will
be the calculation of "interactive_time_in_hour" "session_time_duration_in_hour": 2, —> this
is the time that the costumer need for delivery this is a constrain is not modified never is used for
reference },
```

```
"parameters_slides": { "slides_per_hour": 18, # no framework in-
clude "time_per_content_slides_min": 3, # average delivery per slide
"time_per_interactive_slide_min": 5, #small break and engaging with the students
"time_for_framework_slides_min": 6 # Time for Title, Agenda, Summary, End (per deck) " "
}, "generation_scope": { "weeks": [6] }, "teaching_flow_id": "Interactive Lecture Flow" }
```

```
"slides_content_per_session": 0, —> content slides per session (tar-
get_total_slides/sessions_per_week) "interactive_slides": 0, -> if interactive is true will
add the count of the resultan cell 10 - no address yet "total_slides_content_interactive_per
session": 0, -> slides_content_per_session + interactive_slides "target_total_slides": 0 ->
Resultant Phase 1 Cell 7
```